Royaume du Maroc
Université Sidi Mohamed Ben Abdellah
École Nationale des Sciences
Appliquées de Fès

Filière : Génie des Systèmes Communicants et Sécurité Informatique (GSCSI)

Rapport de stage de Projet de Fin d'Année

`demo_cli`

Un garde-fou pour agents de code IA :
prédire l'effet d'une commande, récupérer, tracer

Réalisé par

Nizar Yartaoui

Encadrante académique
Mme Kaouthar Chetioui
ENSA Fès

Encadrant professionnel
M. Younes Lakhouane
WePwn

Organisme d'accueil : **WePwn**

Période du stage : du 15 juin 2026 au 15 août 2026

Année universitaire 2025–2026

À mes parents, pour leur soutien et leurs sacrifices,

à ma famille et à mes amis,

à tous ceux qui m'ont appris à ne rien tenir pour acquis.

Remerciements

Je tiens à remercier **M. Younes Lakhouane**, fondateur de WePwn et mon encadrant professionnel, pour la confiance qu'il m'a accordée en me remettant `demo_cli` en version bêta, et pour la liberté qu'il m'a laissée d'en reprendre l'architecture, de l'attaquer et de la consolider.

J'adresse mes remerciements à **Mme Kaouthar Chetioui**, mon encadrante académique à l'ENSA de Fès, pour son suivi et ses conseils tout au long de ce projet.

Je remercie également l'ensemble du corps professoral de la filière Génie des Systèmes Communicants et Sécurité Informatique, ainsi que les membres du jury qui ont accepté d'évaluer ce travail.

Enfin, je remercie ma famille et mes proches pour leur soutien constant.

Résumé

Les agents de code fondés sur l'intelligence artificielle, comme Claude Code et Codex, exécutent eux-mêmes des commandes shell sur la machine du développeur ; une erreur de leur part peut supprimer des fichiers, vider une base de données ou réécrire un historique distant. Ce rapport présente le travail mené chez WePwn sur `demo_cli`, un garde-fou qui intercepte chaque commande d'un agent *avant* son exécution, en prédit l'effet et rend une décision : la laisser passer, la rendre réversible en copiant d'abord ce qu'elle va détruire, la placer sous un filet d'exécution, ou la soumettre à une personne et la refuser si rien ne peut être récupéré ; chaque décision laisse un reçu dans un registre chaîné.

Partant d'une version bêta qui reconnaissait les commandes par des expressions régulières, le stage a conduit à un moteur qui analyse la ligne et simule ses effets sur une vue virtuelle du système de fichiers, à une table de règles écrite comme des données dont chaque exemple est un test, à une politique et un décideur uniques, et à deux filets d'exécution : un garde d'appels système fondé sur ptrace sous Linux et un système de fichiers WinFsp sous Windows. L'ancien chemin à expressions régulières ne décide plus aucune ligne POSIX, et un second frontal pour PowerShell partage le même moteur. Le travail a été mené partie par partie, avec des tests qui échouent sur le code d'avant et un portail de 922 commandes qui interdit qu'une décision devienne plus permissive sans arbitrage. Un travail offensif a mis en évidence et corrigé des vulnérabilités, dont une tâche Windows élevée qui exécutait du code modifiable par l'agent.

Mots-clés : agents de code IA, sécurité, analyse de commandes shell, simulation, récupération, ptrace, WinFsp, PowerShell, journal chaîné.

Abstract

AI coding agents such as Claude Code and Codex run shell commands on the developer's machine themselves; a single mistake can delete files, empty a database or rewrite a remote history. This report presents the work carried out at WePwn on `demo_cli`, a safety guard that intercepts every command an agent wants to run *before* it runs, predicts its effect and returns one decision: let it through, make it reversible by first copying what it will destroy, run it under a runtime net, or hand it to a person and refuse it when nothing can be recovered. Every decision leaves a receipt in a hash-chained ledger.

Starting from a beta that recognised commands with regular expressions, the internship produced an engine that parses the line and simulates its effects on a virtual view of the file system, a rule table written as data in which every example is a test, a single policy and a single decider, and two runtime nets: a ptrace-based system-call guard on Linux and a WinFsp file system on Windows. The old regular-expression path no longer decides any POSIX line, and a second front end for PowerShell shares the same engine. The work was done one part at a time, with tests that fail on the previous code and a 922-command decision gate that forbids any decision from becoming looser without an explicit ruling. Offensive work found and fixed vulnerabilities, among them an elevated Windows task that ran code the agent could modify.

Keywords: AI coding agents, security, shell command analysis, simulation, recovery, ptrace, WinFsp, PowerShell, hash-chained ledger.

Table des matières

Remerciements ii

Résumé iii

Abstract iv

Liste des abréviations et glossaire xiv

Introduction générale 1

1 Contexte général du projet 3

1.1 L'organisme d'accueil : WePwn . . . 3

1.1.1 Présentation . . . 3

1.1.2 Fiche signalétique . . . 4

1.1.3 Cadre du stage . . . 4

1.2 Point de départ et contribution . . . 4

1.2.1 La version bêta remise au début du stage . . . 4

1.2.2 Ce que le stage a apporté . . . 5

1.3 Problématique : des agents qui exécutent des commandes . . . 6

1.3.1 Un agent n'écrit plus seulement du code, il l'exécute . . . 6

1.3.2 Trois incidents publics . . . 6

1.3.3 Pourquoi le problème est difficile . . . 7

1.3.4 Énoncé du problème . . . 8

1.4 Objectifs du projet . . . 8

1.5 Méthodologie de travail . . . 8

1.5.1 Le risque propre à un outil de sécurité . . . 9

1.5.2 Une partie à la fois . . . 9

1.5.3 Les portes d'un atterrissage . . . 10

1.5.4 Le message de commit comme compte rendu . . . 10

1.6 Planning 11
1.6.1 Déroulement du stage 11
1.6.2 Chronologie des parties 11

2 État de l'art et étude de l'existant 14
2.1 Les agents de code et leurs points d'extension 14
2.1.1 Claude Code : le *hook* PreToolUse 14
2.1.2 Codex : un *hook* qui ne sait que refuser 14
2.1.3 Cursor 15
2.2 Les demandes de permission 15
2.3 Les bacs à sable 16
2.4 Points de restauration, gestion de versions et sauvegardes 17
2.4.1 Les points de restauration des agents 17
2.4.2 La gestion de versions 17
2.4.3 Les sauvegardes 17
2.5 L'analyse statique des commandes shell 18
2.6 La surveillance à l'exécution 19
2.7 Synthèse et positionnement de `demo_cli` 20

3 Analyse et conception 22
3.1 Analyse des besoins 22
3.1.1 Acteurs 22
3.1.2 Besoins fonctionnels 22
3.1.3 Besoins non fonctionnels 22
3.2 Modèle de menace et invariant 22
3.2.1 Contre quoi `demo_cli` protège 22
3.2.2 L'invariant 24
3.2.3 Fermé pour les décisions, ouvert pour les pannes 24
3.3 Architecture d'ensemble 24
3.3.1 Le flux d'une commande 25
3.3.2 Ce que l'outil voit 26
3.4 Les principes qui gouvernent les décisions 27
3.4.1 Analyser et simuler plutôt que reconnaître des motifs 27

3.4.2 L'inconnu n'est pas sûr : « non couvert » et « non compris » 27
3.4.3 Jamais un REVERSIBLE non mérité 30
3.4.4 Pendant la migration, la réponse la plus stricte gagne 30
3.4.5 Les fichiers de `demo_cli` : le plan de contrôle 31
3.4.6 Demander ou refuser 32
3.4.7 Les faits sur les outils sont vérifiés 32
3.5 Conception du moteur 33
3.5.1 Vue d'ensemble 33
3.5.2 Les étapes 33
3.5.3 Le langage des effets 34
3.5.4 La table de règles, écrite comme des données 34
3.5.5 Le budget 36
3.6 La politique et le décideur unique 36
3.6.1 Une seule politique 36
3.6.2 La table de décision 36
3.6.3 Le décideur 36
3.6.4 Six décisions, trois postures 37
3.6.5 Les escalades sans surface 38
3.7 Deux systèmes, un décideur 39
3.7.1 Linux : pourquoi ptrace 40
3.7.2 Windows : pourquoi un montage WinFsp 41
3.7.3 Points de contrôle : utiles, mais optionnels 41
3.8 Choix de conception et alternatives écartées 42

4 Réalisation 45

4.1 Environnement de réalisation 45
4.1.1 Organisation du code 46
4.2 Les portes 46
4.2.1 Le *hook* Claude Code 46
4.2.2 La porte fichier 48
4.2.3 Codex et Cursor 48
4.2.4 Le garde du shell 48

4.3 Le moteur et ses familles d'outils 48
4.3.1 Des parties courtes, chacune avec ses tests 48
4.3.2 Les familles d'outils 49
4.3.3 Quelques détails qui comptent 51
4.4 La récupération et les reçus 51
4.4.1 Captures ciblées et groupes 51
4.4.2 L'aperçu SQL 52
4.4.3 Le registre de reçus 52
4.5 Les filets d'exécution 53
4.5.1 Linux : le garde d'appels système 53
4.5.2 Windows : le montage WinFsp 54
4.5.3 Ce que les deux filets ne voient pas 56
4.6 Le côté Windows 57
4.6.1 Parité entre les deux systèmes 57
4.6.2 Une leçon de la partie T : juger ce qui s'exécute 58
4.6.3 PowerShell avant C3 58
4.7 C3 : le second frontal du moteur 58
4.7.1 L'idée : un frontal, pas un second garde-fou 58
4.7.2 Organisation du code 59
4.7.3 Fonctionnement 60
4.7.4 Les arbitrages PS1–PS5 61
4.7.5 Validation 62
4.8 M2.3 : la fin du pouvoir des anciennes règles 62
4.8.1 M2.3a : les lignes non lues quittent les anciennes règles 62
4.8.2 M2.3b : l'ancien chemin ne décide plus aucune ligne POSIX 63

5 Sécurité, validation et limites 64
5.1 Le travail offensif 64
5.1.1 Trois façons de chercher 64
5.1.2 L'audit de reprise 64
5.1.3 La batterie red-team 65
5.1.4 Les vulnérabilités du stage 67

5.2 Cas d'étude : la tâche Windows qui exécutait du code modifiable 70
5.2.1 Le contexte 70
5.2.2 La faille 70
5.2.3 Trois conceptions écartées 70
5.2.4 La correction 71
5.2.5 La vérification 72
5.3 Stratégie de test 72
5.3.1 Les familles de tests 72
5.3.2 Un test qui échoue sur le code d'avant 73
5.3.3 Les pièges des tests eux-mêmes 74
5.4 Le portail de décision 74
5.4.1 Principe 74
5.4.2 Les mouvements, partie par partie 75
5.4.3 Quand le portail ne suffit plus : le corpus élargi (M2.3b) 76
5.4.4 Deux références, une par système 76
5.5 Résultats 76
5.6 Limites assumées 76

Conclusion générale et perspectives **79**

Bibliographie et webographie **81**

A Exemple de reçu **83**

B Extrait de la table de règles **85**

C Extraits de tests **86**

D Captures d'écran **88**

E L'architecture complète **90**
E.1 La carte 90
E.2 Où sont les lignes 92
E.3 Le trajet d'une commande dans le code 93
E.4 Les dépendances vont dans un seul sens 93

E.5 Carte des modules 94

Table des figures

1.1 La méthode : le cycle d'une partie et les portes à franchir avant l'atterrissage . 9

1.2 Chronologie des parties livrées, en jours du cycle documenté (d'après l'historique git) 12

3.1 Le flux d'ensemble : de la commande de l'agent au reçu 25

3.2 Ce que l'outil voit : chaque couche et ce qu'elle couvre, selon l'endroit où la commande agit 26

3.3 L'intérieur du moteur, sur un exemple réel (sortie du moteur au commit `7d2a2aa`) 33

3.4 La table de décision : où l'effet tombe × quelle sorte d'effet 37

3.5 Linux et Windows : un seul décideur, deux filets d'exécution 39

4.1 Sous Windows : `demo_cli setup` protège un projet (déplacement, verrouillage, tâche, montage), puis `demo_cli teardown` défait chaque étape 55

4.2 Le montage actif (`doctor`) : un `Remove-Item` tapé dans le projet est capturé par le montage, qui ne lit aucune commande (entrée « delete a.txt » du journal de récupération) 56

4.3 C3 : deux frontaux, un seul moteur 59

5.1 La tâche durcie : refus d'écriture et de modification pour un utilisateur normal 72

E.1 La carte complète de `demo_cli` : les 80 modules, groupés par rôle, avec leur nombre de lignes au commit `7d2a2aa`. Les familles du moteur sont regroupées (un fichier de règles, un fichier d'opérations, parfois un fichier de données générées) ; le détail est au tableau E.2. C3, en tirets, est décrit à la section 4.7. . 91

E.2 Lignes de code par couche, au commit `7d2a2aa` (même découpage que la figure E.1) 92

E.3 Le trajet d'une commande dans le code, du *hook* au reçu (fonctions réelles de `guard.py` et des modules qu'il appelle) 93

Liste des tableaux

1.1 Fiche signalétique de WePwn 4
1.2 De la version bêta à l'état actuel (commit `7d2a2aa`, 9 octobre 2026) 5
1.3 Lignes où une lecture naïve se trompe (extraits de `tests/test_command_model.py` et des commits I1a, W0) 7
1.4 Objectifs du stage et critère de vérification de chacun 8

2.1 Les protections existantes face aux critères du problème 20

3.1 Besoins fonctionnels 23
3.2 Besoins non fonctionnels 23
3.3 Le langage des effets (`engine/effects.py`) 35
3.4 L'ordre des branches de `decide.py` 38
3.5 Choix de conception, alternatives et justifications 42

4.1 Environnement et outils 45
4.2 Traduction des décisions par le *hook* Claude Code 47
4.3 Les parties du moteur et ce que chacune a apporté 49
4.4 Les familles d'outils du moteur (partie M2.2) 49
4.5 Appels système surveillés par le garde Linux 53
4.6 Parité Linux / Windows au commit de référence 57
4.7 Lignes PowerShell : l'ancien chemin et C3 (le projet contient a.txt et b.txt) . . 58
4.8 Les modules de C3 59

5.1 La batterie red-team #001–#010 et son issue 65
5.2 Vulnérabilités corrigées 67
5.3 Vulnérabilités et constats ouverts 69
5.4 W-SEC : chaque maillon de la chaîne d'exécution, avant et après 71
5.5 Tests de partie rejoués sur le commit qui précède leur partie 74
5.6 Mouvements du portail de décision déclarés par partie (messages de commit) . 75

5.7 Résultats chiffrés au commit de référence `7d2a2aa` 77

D.1 Démonstrations reproductibles 88

E.1 `demo_cli` en chiffres, au commit de référence `7d2a2aa` 90

E.2 Les 12 familles du moteur, chargées à la demande 92

E.3 Carte des modules : rôle et taille (lignes, commit `7d2a2aa`) 94

Liste des abréviations et glossaire

Abréviations

Sigle	Signification
ACL	*Access Control List* — liste de contrôle d'accès (Windows/NTFS)
API	*Application Programming Interface*
AST	*Abstract Syntax Tree* — arbre syntaxique abstrait
CLI	*Command-Line Interface* — interface en ligne de commande
CPU	*Central Processing Unit*
ENSAF	École Nationale des Sciences Appliquées de Fès
FS	*File System* — système de fichiers
FUSE	*Filesystem in Userspace* (Linux)
GSCSI	Génie des Systèmes Communicants et Sécurité Informatique
HTTP	*HyperText Transfer Protocol*
IA	Intelligence artificielle
IaC	*Infrastructure as Code* (Terraform, Pulumi…)
LLM	*Large Language Model* — grand modèle de langage
PaaS	*Platform as a Service*
PFA	Projet de Fin d'Année
PITR	*Point-In-Time Recovery* — restauration à un instant donné
POSIX	*Portable Operating System Interface* — ici, le langage des shells `sh`/`bash`
SaaS	*Software as a Service*
SDDL	*Security Descriptor Definition Language* (Windows)
SQL	*Structured Query Language*
UAC	*User Account Control* — élévation de privilèges sous Windows
VCS	*Version Control System* — gestion de versions (git)

Sigle	Signification
WSL	*Windows Subsystem for Linux*

Glossaire

Agent de code IA
: Programme fondé sur un LLM qui lit et modifie un projet et exécute des commandes shell pour le compte d'un développeur (Claude Code, Codex…).

Porte
: Point par lequel une action de l'agent atteint `demo_cli` avant de s'exécuter : *hook* de l'agent, porte d'édition de fichiers, garde du shell, commande `demo_cli check`.

Effet
: Ce qu'une commande fait au monde, dans un vocabulaire commun : DELETE, OVERWRITE, TRUNCATE, MOVE, CREATE, APPEND, RUN, SEND, LATER…

Surface distante
: Ce qu'aucune copie locale ne peut restaurer : base de données distante, historique git d'un serveur, ressource cloud, conteneur, message envoyé…

Non couvert
: (*uncovered*) Un outil que la table de règles ne décrit pas encore.

Non compris
: (*not understood*) Une partie de ligne que le moteur ne peut pas lire avant l'exécution (valeur inconnue, construction non modélisée).

Point de récupération
: Copie, prise *avant* l'exécution, de ce que la commande va détruire ; `demo_cli undo <id>` la restaure.

Filet d'exécution
: Mécanisme qui observe la commande *pendant* qu'elle tourne et sauvegarde chaque fichier au moment où il est détruit : garde d'appels système (Linux), montage WinFsp (Windows).

Plan de contrôle
: Les fichiers de `demo_cli` lui-même (configuration, registre, réserve de récupération, configuration des *hooks*) : un agent ne peut pas les modifier.

Reçu
: Entrée du registre décrivant une décision ; chaque reçu contient l'empreinte du précédent (chaîne SHA-256).

Portail de décision

Comparaison, sur un corpus de 922 commandes, de chaque décision avant et après une modification.

Arbitrage

(*ruling*) Décision écrite et datée qui tranche un cas de politique avant son implémentation.

Introduction générale

Les agents de code fondés sur l'intelligence artificielle ont changé de nature. Ils ne se contentent plus de proposer du code : ils l'exécutent. Claude Code ou Codex lisent un projet, modifient des fichiers et lancent des commandes dans un shell, avec les droits du développeur et sur sa machine. Un agent qui se trompe ne produit plus une suggestion maladroite, mais une suppression, un écrasement, une base de données vidée ou un historique réécrit sur un serveur. Plusieurs incidents publics l'ont montré, et les protections existantes — demandes de permission, bacs à sable, points de restauration, gestion de versions — en couvrent chacune une partie seulement.

C'est dans ce contexte que s'inscrit ce stage de Projet de Fin d'Année, effectué au sein de WePwn, entreprise marocaine de sécurité offensive. Il porte sur `demo_cli`, un garde-fou pour agents de code que WePwn avait publié en version bêta. L'idée de départ de cette bêta est simple et forte : avant qu'une commande destructrice ne s'exécute, copier ce qu'elle va détruire, pour que l'erreur reste réversible ; et quand rien ne peut être copié, ne pas le laisser passer sans qu'une personne l'ait vu. Sa limite l'était tout autant : elle lisait les commandes avec des expressions régulières.

La question qui guide ce travail est donc la suivante : *comment, avant qu'une commande écrite par un agent ne s'exécute, en prédire l'effet assez exactement pour laisser passer ce qui est sans danger, rendre récupérable ce qui peut l'être, et arrêter ou soumettre à une personne le reste — sans jamais revendiquer une protection qu'on n'a pas prise, et de la même manière sous Linux et sous Windows ?*

La réponse construite pendant le stage tient en quelques principes : remplacer les motifs par un *moteur* qui analyse la ligne et en simule les effets sur une vue virtuelle du système de fichiers ; réunir toute la politique dans un seul module et toutes les décisions dans un seul décideur ; placer, sous ce que le moteur ne sait pas lire, un *filet d'exécution* — un garde d'appels système sous Linux, un système de fichiers WinFsp sous Windows ; refuser à l'agent toute action sur les fichiers de l'outil ; et faire évoluer l'ensemble avec une méthode qui interdit qu'une décision devienne plus permissive sans qu'on l'ait voulu. Le travail a aussi été offensif : chercher, comme dans un programme de *bug bounty*, comment contourner ce qui venait d'être construit.

Ce rapport est organisé en cinq chapitres. Le premier présente l'organisme d'accueil, le point de départ du stage, la problématique, les objectifs et la méthode. Le deuxième étudie les protections existantes et situe `demo_cli` par rapport à elles. Le troisième, le plus important,

expose l'analyse et la conception : besoins, modèle de menace, architecture, et surtout les principes — et leurs raisons — qui gouvernent chaque décision. Le quatrième décrit la réalisation, couche par couche, y compris le côté Windows et le second frontal du moteur pour PowerShell. Le cinquième rend compte du travail de sécurité, de la validation et des limites. Une conclusion dresse le bilan et ouvre des perspectives.

CHAPITRE 1

Contexte général du projet

Ce chapitre situe le travail. Il présente l'organisme d'accueil, WePwn, puis le point de départ concret du stage : un outil, `demo_cli`, remis en version bêta. Il pose ensuite le problème que cet outil traite — des agents d'intelligence artificielle qui exécutent eux-mêmes des commandes sur la machine d'un développeur —, fixe les objectifs, et décrit la méthode de travail qui a permis de faire évoluer un logiciel de sécurité sans jamais le rendre, même un instant, moins sûr.

1.1 L'organisme d'accueil : WePwn



WePwn

Sécurité offensive · programmes de *bug bounty* · divulgation coordonnée de vulnérabilités · outils de sécurité

1.1.1 Présentation

WePwn est une entreprise marocaine de sécurité offensive, basée à Casablanca. Son activité d'origine est le *bug bounty* : une entreprise ouvre un programme, des chercheurs en sécurité indépendants testent ses services, et chaque vulnérabilité validée est récompensée. WePwn se présente comme la mise en relation entre des entreprises et des chercheurs, afin que les premières soient averties des vulnérabilités identifiées chez elles [1]. Selon l'organisme, elle a été la première plateforme de *bug bounty* au Maroc [1] ; les programmes qu'elle héberge rémunèrent les chercheurs en dirhams. Un portail dédié permet aux chercheurs de rejoindre ces programmes [2].

Autour de cette activité, WePwn propose un service de divulgation de vulnérabilités géré de bout en bout [3] : tri des signalements, évaluation de la sévérité, validation, rapport détaillé, conseils de correction et pilotage du programme dans la durée. Ce métier — recevoir des attaques réelles, les reproduire, en mesurer l'impact et accompagner la correction — a naturellement conduit l'entreprise vers un second axe : les *pratiques* et les *outils* de sécurité. `demo_cli` appartient à cet axe.

1. Affirmation de l'organisme d'accueil, reprise telle quelle.

1.1.2 Fiche signalétique

Table 1.1. Fiche signalétique de WePwn

Raison sociale	WePwn
Domaine	Cybersécurité — sécurité offensive
Activités	Programmes de *bug bounty* ; divulgation de vulnérabilités gérée ; pratiques et outils de sécurité
Fondateur	M. Younes Lakhouane
Site web	`https://wepwn.ma`
Siège	Casablanca
Effectif	plus de 10 personnes

1.1.3 Cadre du stage

Le stage s'est déroulé au sein de WePwn, sous l'encadrement professionnel de M. Younes Lakhouane, fondateur de l'entreprise, et sous l'encadrement académique de Mme Kaouthar Chetioui (ENSA de Fès). Il s'est déroulé du 15 juin 2026 au 15 août 2026. Le sujet porte sur `demo_cli`, un outil de WePwn publié en source ouverte sous licence MIT [4], que l'entreprise destine aux équipes qui confient du travail à des agents de code IA.

La double culture de l'entreprise explique la forme du travail demandé : construire l'outil *et* l'attaquer. Une partie du stage a consisté à développer et à architecturer ; une autre, à chercher comment contourner ce qui venait d'être construit, avec les méthodes du *bug bounty* — lire le code, fabriquer des commandes piégées, fouiller les traces. Le chapitre 5 rend compte de cette seconde partie.

1.2 Point de départ et contribution

1.2.1 La version bêta remise au début du stage

Le stage ne part pas d'une page blanche. WePwn avait publié `demo_cli` en version 0.4.0-beta.8 (19 juillet 2026) [4]. Cette bêta posait déjà l'idée qui reste au cœur du projet, et qu'il faut attribuer à son point d'origine :

L'invariant hérité de la bêta

Une action qui modifie quelque chose doit être récupérable *et* conforme au contexte déclaré ; sinon elle est escaladée — jamais autorisée en silence, jamais présentée à tort comme « récupérée ».

De cet invariant découlait une posture inhabituelle pour un outil de sécurité : *récupérer plutôt que bloquer*. Quand `demo_cli` peut prouver qu'il a copié la cible réelle d'une commande destructrice, il la copie et laisse l'agent continuer ; une commande `demo_cli undo <id>` remet les fichiers en place. Il ne bloque que ce qui ne peut pas être récupéré (un `terraform destroy`, un `git push -force`). La bêta s'intégrait à Claude Code et à Cursor, savait prévisualiser une requête SQL destructrice et laissait de chaque décision un reçu chaîné.

Elle annonçait aussi, honnêtement, ses limites. Son journal des modifications le dit en une phrase : l'analyse des commandes shell repose sur des motifs, « un arbre syntaxique complet est sur la feuille de route ». Concrètement, le classifieur tenait en 340 lignes et 9 expressions régulières. Le tableau 1.2 compare cette bêta à l'état du projet au commit de référence `7d2a2aa`.

Table 1.2. De la version bêta à l'état actuel (commit `7d2a2aa`, 9 octobre 2026)

	Bêta 0.4.0-beta.8	**Aujourd'hui**
Code source (Python)	≈ 3 700 lignes	≈ 34 200 lignes
Tests	≈ 1 600 lignes, 11 fichiers	≈ 29 000 lignes, 134 fichiers
Lecture d'une commande	9 expressions régulières	analyseur POSIX, simulation sur un système de fichiers virtuel, table de 72 règles (214 outils)
Décisions	5	6 (+ SUPERVISED), canal « demander / refuser »
Commandes illisibles	motifs, au mieux	« non compris » : filet, point de contrôle ou escalade — jamais un ALLOW
Pendant l'exécution	rien	garde d'appels système (Linux), montage WinFsp (Windows)
Surfaces distantes	quelques motifs	38 surfaces, chacune avec sa règle « ask » ou « deny »
Agents	Claude Code, Cursor	Claude Code (Bash, PowerShell, fichiers), Codex, Cursor, garde du shell
Fichiers de `demo_cli`	non protégés	plan de contrôle refusé à l'agent
Validation	tests unitaires	tests + exemples de règles + portail de 922 commandes, Linux et Windows

1.2.2 Ce que le stage a apporté

La mission confiée était de reprendre cette bêta, de la poursuivre et d'en reprendre l'architecture, en menant de front le développement et le travail offensif. Le dépôt de travail conserve le détail des 52 commits de la dernière phase ; chacun correspond à une partie nommée, et son message en fait le bilan.

La contribution se résume en quatre lignes :

1. **Un moteur au lieu de motifs** : une analyse syntaxique unique des lignes POSIX, une simulation de leurs effets et une table de règles écrite comme des données (chapitres 3 et 4).
2. **Une seule politique, un seul décideur** : ce qui était dispersé entre plusieurs modules est réuni dans `policy.py` et `decide.py`.
3. **Les deux systèmes** : chaque partie est conçue et vérifiée sous Linux et sous Windows ; le filet d'exécution existe sur les deux.
4. **Le travail offensif** : un audit pinçant 18 défauts, puis une suite de vulnérabilités (V1–V22) trouvées et corrigées, chacune avec un test qui échoue sur le code d'avant (chapitre 5).

Une part essentielle du rôle a été celle d'*arbitre*. Chaque cas de politique délicat — faut-il autoriser un `git reset` que le *reflog* sait annuler ? demander ou refuser la suppression d'un volume Docker ? — a fait l'objet d'une décision écrite, numérotée et datée avant d'être codée. Les messages de commit et les tests en gardent la trace : « ruling (c) », « rulings M1–M5 », « Nizar's table ».

1.3 Problématique : des agents qui exécutent des commandes

1.3.1 Un agent n'écrit plus seulement du code, il l'exécute

Un agent de code IA comme Claude Code ou Codex ne se contente pas de proposer du texte. Il lit le projet, modifie des fichiers, et surtout lance des commandes dans un shell — `bash` sous Linux, PowerShell sous Windows — pour compiler, tester, nettoyer, migrer une base, déployer. Chaque commande s'exécute avec les droits du développeur, sur sa machine, souvent à côté de données qui n'existent nulle part ailleurs. L'agent se trompe parfois ; une erreur dans une commande destructrice n'est pas une suggestion maladroite, c'est une perte de données.

1.3.2 Trois incidents publics

Le risque n'est pas théorique. Trois cas documentés illustrent trois formes du problème.

Une base de production supprimée (Replit, juillet 2025)

Au cours d'une expérience de plusieurs jours, l'agent de Replit a exécuté des commandes qui ont effacé la base de données de production d'un utilisateur, alors qu'il avait reçu la consigne de ne rien modifier sans accord explicite ; il a ensuite affirmé à tort que la restauration était

impossible [5, 6]. *Leçon : une consigne n'est pas une protection, et l'agent n'est pas une source fiable sur ce qui peut être récupéré.*

Des fichiers écrasés par des déplacements (Gemini CLI, juillet 2025)
L'agent a cru avoir créé un dossier dont la création avait échoué, puis y a « déplacé » les fichiers de l'utilisateur ; chaque déplacement vers un nom inexistant a écrasé le précédent, et tous les fichiers sauf un ont été perdus [7, 8]. *Leçon : aucune commande de la séquence ne « ressemble » à une suppression ; seul un raisonnement sur l'état du système de fichiers révèle la perte.*

Des fichiers supprimés en voulant les compter (Claude Code, juillet 2026)
Chargé de compter des fichiers de rapport, un agent a lancé une commande qui les a supprimés ; l'utilisateur en a récupéré une partie avec un outil de récupération de données [9]. La bêta de `demo_cli` cite ce cas comme motivation. *Leçon : demander une confirmation ne suffit pas quand la personne fait confiance à l'intention annoncée ; ce qui manquait, c'était une copie prise avant.*

1.3.3 Pourquoi le problème est difficile

Juger une commande *avant* qu'elle s'exécute paraît simple : repérer `rm`, et se méfier. En réalité le langage du shell rend cette lecture trompeuse. Le tableau 1.3 montre quatre lignes, tirées des tests du projet, où une lecture naïve — par motifs, ou par un analyseur trop confiant — se trompe dans un sens ou dans l'autre.

Table 1.3. Lignes où une lecture naïve se trompe (extraits de `tests/test_command_model.py` et des commits I1a, W0)

Ligne	Lecture naïve	Ce que fait le shell
`ls # ; rm -rf x`	un `rm -rf`	rien : tout ce qui suit # est un commentaire
`echo $(date; rm -rf x) >/dev/null`	un `echo`	exécute `rm -rf x` dans la substitution
`bash -o pipefail -c "rm app.db"`	une option inconnue, rien à juger	supprime `app.db`
`X=-rf; rm $X src`	un `rm` sans option	un `rm -rf src`

À cela s'ajoutent les enveloppes (`sudo`, `xargs`, `timeout`), les commandes lancées par du code que l'outil ne lit pas (`python script.py`), les effets à distance (une base, un dépôt git distant, un bucket) qu'aucune copie locale ne restaure, et un second langage, PowerShell, où les arguments se lient aux paramètres par position.

1.3.4 Énoncé du problème

Problématique

Comment, avant qu'une commande écrite par un agent ne s'exécute, en prédire l'effet assez exactement pour laisser passer ce qui est sans danger, rendre récupérable ce qui peut l'être, et arrêter ou soumettre à une personne le reste — sans jamais revendiquer une protection qu'on n'a pas prise, et de la même manière sous Linux et sous Windows ?

1.4 Objectifs du projet

Les objectifs découlent de la problématique ; chacun est formulé de façon à pouvoir être vérifié dans le code et les tests.

Table 1.4. Objectifs du stage et critère de vérification de chacun

	Objectif	Comment on le vérifie
O1	**Prédire** l'effet d'une ligne à partir de sa structure, pas de son texte.	Chaque exemple de la table de règles (774) est un test qui compare l'impact prédit à l'impact attendu.
O2	**Ne jamais autoriser ce qui n'est pas lu.** Une ligne illisible n'obtient pas ALLOW.	Tests M2.3a : une ligne « non comprise » va au filet, à un point de contrôle, sinon elle est escaladée.
O3	**Ne jamais revendiquer une récupération non prise.**	Tests des captures partielles (tout ou rien), de V17 (point de contrôle hors projet).
O4	**Voir pendant l'exécution** ce que la lecture ne peut pas prédire.	Garde d'appels système (Linux) et montage WinFsp (Windows) testés sur les deux systèmes.
O5	**Traiter le distant honnêtement** : distinguer « demander à la personne » et « refuser ».	Table `policy.ESCALATION_CHANNEL` ; une surface sans ligne est refusée.
O6	**Les deux systèmes** : Linux et Windows, avec les mêmes garanties.	Suite complète passée sur les deux systèmes avant chaque atterrissage.
O7	**Ne rien rendre plus permissif sans le vouloir.**	Portail de décision : 922 commandes comparées avant et après chaque partie.
O8	**Résister à l'agent** : il ne doit pas pouvoir désarmer l'outil.	Plan de contrôle refusé à toutes les portes ; tâche Windows durcie (W-SEC).

Un objectif est volontairement *absent* : faire de `demo_cli` une barrière contre un adversaire déterminé. L'outil protège contre les *erreurs* d'un agent coopératif ; ce modèle de menace, hérité de la bêta et affiché par `demo_cli doctor`, est discuté à la section 3.2.

1.5 Méthodologie de travail

1.5.1 Le risque propre à un outil de sécurité

Faire évoluer un outil de sécurité comporte un risque particulier : une modification qui corrige un cas peut, sans bruit, en autoriser un autre. Un test « vert » ne le révèle pas si aucun test ne décrivait le cas devenu permissif. Un audit mené au cours du stage l'a montré : la suite passait avec 1856 tests réussis alors que 18 défauts confirmés restaient atteignables, parce qu'aucun test ne décrivait ces comportements. La méthode adoptée répond directement à ce constat.

1.5.2 Une partie à la fois

Le travail est découpé en *parties* courtes et nommées (A, B, C1, SYS1, M2.2b3, W-SEC…). Chaque partie suit le même cycle (figure 1.1) :

1. **Plan** : périmètre, cas à couvrir, risques, ce qui ne doit pas bouger.
2. **Arbitrages** : chaque question de politique est tranchée *par écrit* avant d'être codée, avec un identifiant et une date (« ruling R2 : un envoi qui peut remplacer un objet distant est escaladé »). Le code et ses tests citent l'arbitrage.
3. **Prototype** : le code, et des tests qui *échouent sur le code d'avant*. Un test qui passerait aussi sur l'ancien code ne prouve rien sur la correction.
4. **Revue** : relecture adverse et sondes — des commandes fabriquées pour piéger la nouvelle logique, lancées dans un projet jetable.
5. **Atterrissage** : un commit dont le message est un bilan : ce qui change, les arbitrages appliqués, les tests ajoutés, et le résultat du portail de décision.

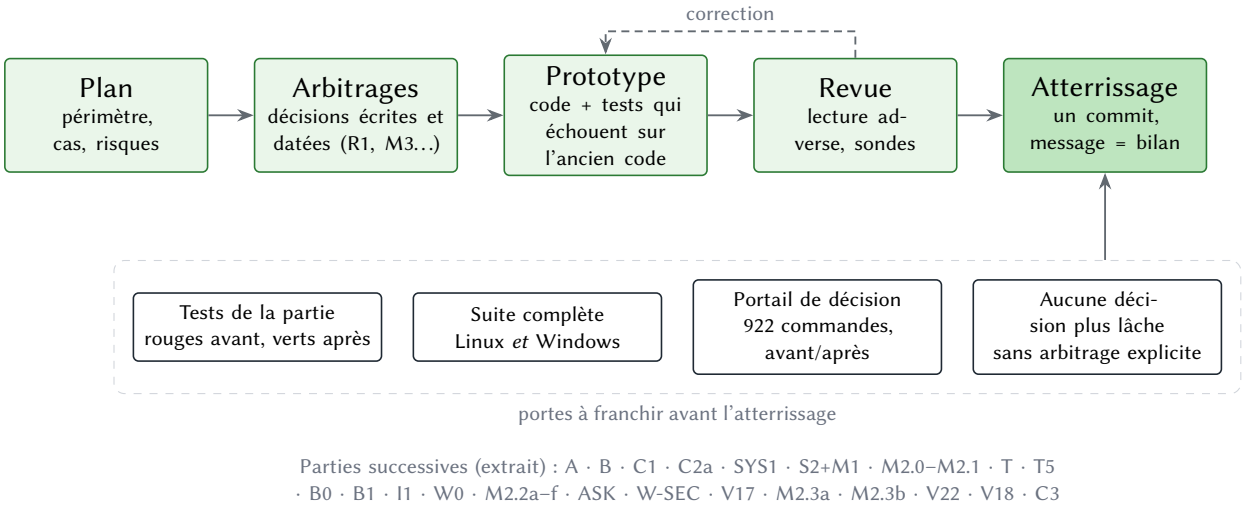


Figure 1.1. La méthode : le cycle d'une partie et les portes à franchir avant l'atterrissage

1.5.3 Les portes d'un atterrissage

Les tests de la partie
Rouges sur le code d'avant, verts après.

La suite complète, sous Linux et sous Windows
Les deux systèmes n'ont pas les mêmes chemins (`C:\`, `/c/` sous Git Bash), ni les mêmes appels système, ni le même shell : un test vert sous Linux ne dit rien de Windows.

Le portail de décision
Un corpus de 922 commandes réelles et fabriquées est évalué avant et après la modification ; toute ligne dont la décision, la règle, la cible ou la capture change est listée et doit être justifiée. Les messages de commit en rendent compte (« the 922 gate changes 124 reviewed decisions : 8 fixed misses, 18 newly captured, 86 newly allowed »).

Aucune décision plus permissive sans arbitrage
Une ligne qui passe d'ESCALATE à ALLOW sans arbitrage explicite bloque l'atterrissage, même si tous les tests passent. Pour un changement qui touche toutes les lignes non lues (M2.3b), le portail est complété par un corpus élargi de 2 306 commandes (section 5.4).

> **Pourquoi un portail de décision en plus des tests ?**
> Les tests vérifient ce qu'on a pensé à écrire. Le portail vérifie *tout le reste* : il ne sait pas quelle décision est juste, mais il voit chaque décision qui change. C'est lui qui transforme « la suite passe » en « rien n'a bougé sans qu'on le sache ».

1.5.4 Le message de commit comme compte rendu

Chaque atterrissage laisse un compte rendu lisible directement dans l'historique. L'extrait suivant est réel (les lignes « [...] » sont coupées) :

```
git show -s 3c94414

$ git show -s --format=%B 3c94414
M2.3a: lines the engine does not fully read leave the old text rules

- runners (python, node, npm, make, ./x, source, awk programs that run or
  write ...) go to the runtime net, else a checkpoint, else ALLOW with a
  note that nothing was captured (ruling M1)
- a line the engine cannot read goes to the net, else a checkpoint, else
  ESCALATE (M3) - unread SQL such as psql -c "$(cat q.sql)" was ALLOWED
  (V19); where the old rules flag the line they keep their capture or
  escalation, never an ALLOW
[...]
- engine rules for gzip / bzip2 / xz, sort -o, iconv -o, patch,
  awk -i inplace, 7z: they deleted or overwrote files unseen (M4, V20)
[...]
- tests/test_m23a.py (17); row fixtures of earlier parts: every moved row
  stricter; gate: 6 rows move, reference re-recorded
```

1.6 Planning

1.6.1 Déroulement du stage

Le stage s'est déroulé du 15 juin 2026 au 15 août 2026, au sein de WePwn. Il a suivi la progression décrite à la section 1.5 : prise en main et audit de la bêta, puis une succession de parties courtes, chacune livrée avec ses tests et son bilan.

1.6.2 Chronologie des parties

L'historique du dépôt permet de dater les parties les unes par rapport aux autres. La figure 1.2 les place sur les jours du cycle de développement documenté et les regroupe en cinq phases : la reprise et l'audit ; un moteur qui prédit sans encore décider (ses prédictions sont seulement écrites dans les reçus, pour être comparées aux décisions existantes) ; le moteur qui décide ; l'extension famille par famille de la table de règles ; enfin la sécurité et la sortie de l'ancien chemin par expressions régulières. La dernière partie, C3, donne au moteur un second frontal pour PowerShell.

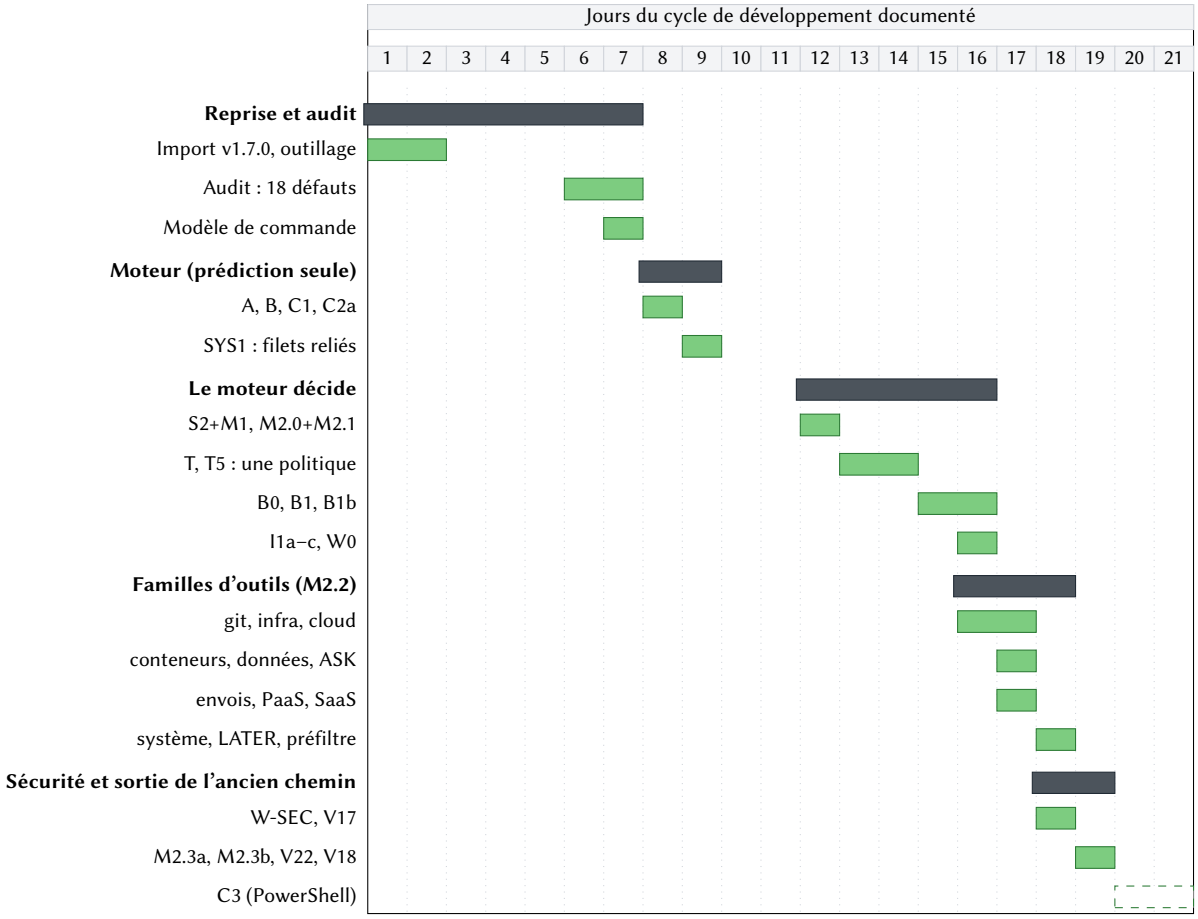


Figure 1.2. Chronologie des parties livrées, en jours du cycle documenté (d'après l'historique git)

> **Pourquoi un moteur « en ombre » avant de le laisser décider ?**
>
> Les parties B à C2a ont livré un moteur qui *ne décidait rien* : il écrivait sa prédiction dans chaque reçu, et le portail vérifiait que zéro décision changeait. Pendant ce temps, chaque divergence entre la prédiction et la décision existante était un signal : soit un défaut du moteur, soit un défaut de l'ancien chemin. Le moteur n'a pris la main (partie S2+M1) qu'une fois ces divergences comprises, et ce changement-là a été revu ligne par ligne sur le portail.

Conclusion du chapitre

Le stage part d'un outil qui existait, qui fonctionnait sur les cas simples et qui annonçait lui-même sa limite principale : il lisait les commandes avec des expressions régulières. Le problème est réel et documenté par des incidents publics ; les objectifs sont mesurables ; la méthode impose qu'aucune décision ne devienne plus permissive sans qu'on l'ait voulu. Le chapitre suivant examine ce qui existe déjà pour protéger un développeur de son agent, et pourquoi ces protections ne suffisent pas.

CHAPITRE 2

État de l'art et étude de l'existant

Avant de construire, il faut savoir ce qui existe. Ce chapitre passe en revue les mécanismes qui protègent aujourd'hui un développeur des actions de son agent : les points d'extension des agents eux-mêmes, les demandes de permission, les bacs à sable, les points de restauration et la gestion de versions, l'analyse statique des scripts shell, et la surveillance à l'exécution. Pour chacun, la même question : *que couvre-t-il, et que laisse-t-il passer ?* La synthèse finale situe `demo_cli` par rapport à eux.

2.1 Les agents de code et leurs points d'extension

Les deux agents visés par le projet, Claude Code et Codex, ont la même structure : un modèle de langage décide d'une action, puis l'agent l'exécute au moyen d'un *outil* — un outil shell (`Bash`, ou `PowerShell` sous Windows), des outils d'édition de fichiers, d'autres encore. Entre la décision et l'exécution, chacun offre un point d'extension où un programme tiers peut intervenir : un *hook*.

2.1.1 Claude Code : le *hook* PreToolUse

Claude Code appelle un *hook* `PreToolUse` « avant l'exécution d'un appel d'outil », avec la possibilité de le bloquer [10]. Le *hook* reçoit l'appel en JSON et peut répondre par une décision de permission (`allow`, `deny`, `ask`, `defer`) accompagnée d'une raison ; il peut aussi *remplacer* les arguments de l'outil par un champ `updatedInput`. Deux précisions de la documentation comptent pour la conception : un *hook* qui se tait ne vaut pas approbation — la commande « continue dans le flux de permission normal » — et un *hook* qui dépasse son délai ne bloque pas l'appel [10]. Le même *hook* voit passer les outils shell et les outils d'édition de fichiers (`Edit`, `Write`).

2.1.2 Codex : un *hook* qui ne sait que refuser

Codex propose lui aussi un *hook* avant l'utilisation d'un outil. Son schéma déclare les trois valeurs `allow`, `deny` et `ask`, mais, en pratique, l'exécution rejette tout sauf `deny` : une réponse `allow` produit l'erreur « *unsupported permissionDecision :allow* » et marque le *hook* en échec,

ce qui laisse passer la commande. Ce comportement, contraire à la documentation, a été établi en faisant tourner l'agent ; l'adaptateur Codex de `demo_cli` le documente (`hooks/codex.py`). Conséquence : sur Codex, `demo_cli` ne peut que refuser ou se taire, jamais « demander », ni réécrire la commande pour la placer sous un filet.

2.1.3 Cursor

Cursor expose un *hook* `beforeShellExecution` configuré dans `.cursor/hooks.json` ; le code de sortie 2 bloque l'action, et les autres échecs du *hook* laissent passer la commande par défaut [11]. L'adaptateur de `demo_cli` demande donc explicitement un comportement « fermé en cas d'échec » (`failClosed`).

> **Ce que ces points d'extension impliquent**
>
> Un *hook* voit la *commande*, pas son effet. Il reçoit une chaîne de caractères et doit répondre vite, avant que l'agent n'attende trop. Tout ce que `demo_cli` fait avant l'exécution — lire, prédire, copier, décider — doit tenir dans ce créneau, et ce que les agents permettent de répondre diffère de l'un à l'autre. L'architecture devra donc séparer la *décision* (commune) de la façon de la *dire* à chaque agent (propre à chaque porte).

2.2 Les demandes de permission

La protection la plus répandue est la plus simple : avant une action sensible, l'agent demande à la personne. Elle a deux faiblesses connues.

L'accoutumance. Une demande répétée finit par être approuvée sans être lue. Anthropic l'écrit à propos de Claude Code : avec le temps, les demandes mènent à une « fatigue d'approbation, où les gens cessent de prêter attention à ce qu'ils approuvent », et les utilisateurs approuvent 93 % des demandes de permission [12]. Le phénomène est documenté bien avant les agents : une étude de terrain sur plus de 25 millions d'avertissements de navigateur a mesuré que les utilisateurs passaient outre l'avertissement SSL de Chrome dans 70,2 % des cas, et plus vite pour les erreurs les plus fréquentes [13].

Confirmer n'est pas récupérer. Une personne qui approuve `rm -f Reports/report_*.txt` en croyant que l'agent compte des fichiers (section 1.3) n'a rien gagné à être consultée. Une fois la commande passée, la demande n'offre aucun moyen de revenir en arrière.

Le juge automatique. Pour réduire le nombre de demandes, Claude Code propose un mode où un classifieur évalue chaque action avant son exécution et ne laisse passer sans demande

que ce qu'il juge aligné avec l'intention de l'utilisateur. Ses auteurs donnent honnêtement ses limites : un taux de faux négatifs de 17 % sur des actions réellement excessives, mesuré sur un petit échantillon, et le constat qu'il « ne remplace pas une relecture humaine attentive sur une infrastructure à fort enjeu » [12].

> **Ce que `demo_cli` en retient**
>
> Demander reste utile, mais *seulement* quand la question est rare et qu'elle a un sens : quand rien n'a pu être capturé et que la personne peut juger sur la commande elle-même. Pour tout ce qui peut être copié avant, mieux vaut copier et ne pas déranger. D'où deux choix de `demo_cli` : récupérer plutôt que demander quand c'est possible, et distinguer, pour ce qui ne peut pas l'être, ce qu'on *demande* de ce qu'on *refuse* (section 3.4.6).

2.3 Les bacs à sable

Un bac à sable confie au système d'exploitation le soin de limiter ce qu'une commande peut atteindre. Les deux agents en proposent un.

Claude Code
: Le bac à sable de l'outil shell s'appuie sur Seatbelt sous macOS et sur `bubblewrap` sous Linux et WSL2. Il est désactivé par défaut. Une fois activé, les commandes peuvent écrire dans le répertoire de travail, un répertoire temporaire et les répertoires ajoutés ; certains chemins sensibles restent protégés en écriture (fichiers de configuration de l'agent, `.bashrc`, *hooks* git…). Sous Windows natif, les commandes s'exécutent sans bac à sable [14].

Codex
: Codex sépare deux réglages : une politique d'approbation (quand demander) et un mode de bac à sable (ce que le système autorise), dont le mode `workspace-write` qui limite les écritures à l'espace de travail [15]. Sous Linux, l'isolation du système de fichiers repose par défaut sur `bubblewrap` [16].

Conteneurs et machines virtuelles
: Faire travailler l'agent dans un conteneur ou une machine virtuelle jetable isole la machine hôte entière.

Ce qu'un bac à sable ne couvre pas. Le bac à sable répond à la question « *où* la commande peut-elle écrire ? ». Or le projet lui-même — le répertoire de travail — est, par construction, inscriptible : c'est là que l'agent travaille. Un `rm -rf src` ou un `mv` qui écrase un fichier non suivi par git y est autorisé. De même, une commande qui atteint une base de données de production avec des identifiants valides, ou qui pousse de force sur un dépôt distant, utilise un accès que le bac à sable a laissé ouvert pour que l'agent puisse travailler. Enfin, la protection dépend de la

plateforme : là où le bac à sable n'existe pas (Windows natif pour Claude Code), il n'y a rien.

> **Complémentaire, pas concurrent**
>
> Le bac à sable *borne* ; `demo_cli` *juge et récupère à l'intérieur* de la borne. Les deux se combinent : le bac à sable empêche l'agent de sortir du projet, `demo_cli` garantit qu'à l'intérieur du projet une destruction reste réversible, et qu'une action à distance passe par une personne.

2.4 Points de restauration, gestion de versions et sauvegardes

2.4.1 Les points de restauration des agents

Claude Code enregistre automatiquement l'état des fichiers modifiés par ses outils d'édition, et une commande `/rewind` permet de revenir en arrière. Sa documentation précise la limite qui intéresse ce projet, avec un exemple sans ambiguïté : « Checkpointing does not track files modified by Bash commands », illustré par `rm file.txt`, `mv old.txt new.txt` et `cp source.txt dest.txt`, dont les modifications « ne peuvent pas être annulées par un retour arrière » [17]. Autrement dit, les commandes shell — là où se trouvent les destructions — sont hors du périmètre.

2.4.2 La gestion de versions

Git protège ce qui a été *validé*. Il ne protège pas les fichiers non suivis ou ignorés (données locales, fichiers `.env`, bases SQLite de développement, artefacts), ni les modifications non validées qu'un `git reset --hard` ou un `git checkout --` efface, ni le dépôt lui-même si l'agent supprime `.git`. Et un `git push --force` réécrit l'historique *du serveur*, que la copie locale ne restaure pas.

2.4.3 Les sauvegardes

Une sauvegarde système (instantané de disque, sauvegarde nocturne) récupère beaucoup, mais à la mauvaise granularité : elle date de la veille, pas de la seconde qui précède la commande, et la restaurer est une opération lourde qu'on n'entreprend pas pour un fichier.

L'idée qui en découle

Ce qui manque est une copie *ciblée*, prise *juste avant* la commande, de *exactement* ce qu'elle va détruire — et rien qu'elle ne puisse revendiquer à tort. C'est l'idée de départ de la bêta de `demo_cli` ; tout le reste du projet consiste à savoir, avant l'exécution, ce que « exactement ce qu'elle va détruire » veut dire.

2.5 L'analyse statique des commandes shell

Lire une commande avant de l'exécuter relève de l'analyse statique. Trois familles d'outils existent.

Les motifs (expressions régulières)
C'est l'approche des *hooks* de sécurité les plus courants, et celle de la bêta de `demo_cli` : repérer `rm -rf`, `DROP TABLE`, `--force`. Elle est rapide et lisible, mais elle lit des caractères là où le shell lit une grammaire : citations, commentaires, substitutions, enveloppes, variables (tableau 1.3).

Les analyseurs de qualité
ShellCheck signale les erreurs de syntaxe, les pièges sémantiques et les cas limites d'un script [18]. Son but est la *justesse* du script, pas l'effet d'une ligne sur un système de fichiers donné.

Les analyseurs syntaxiques
`bashlex` est une transposition en Python de l'analyseur de GNU bash, qui produit un arbre syntaxique complet sans rien exécuter [19] ; `tree-sitter-bash` est une grammaire bash pour l'outil d'analyse incrémentale tree-sitter [20].

Ce qu'un arbre syntaxique ne dit pas. Un arbre dit que `mv a.txt b.txt` est une commande `mv` à deux opérandes. Il ne dit pas que `b.txt` existe, donc que son contenu va être perdu ; ni que `-n` aurait empêché la perte ; ni que, après un `cd build`, `rm -rf *` vise d'autres fichiers ; ni que `$X` vaut `-rf`. Pour passer de la *structure* à l'*effet*, il faut deux connaissances que l'analyse syntaxique n'apporte pas :

1. la **sémantique de chaque outil** — sa grammaire d'options et ce que chaque option fait (`cp` écrase, `cp -n` n'écrase pas, `sed -i` réécrit en place, `gzip` supprime l'original) ;
2. l'**état du système de fichiers**, mis à jour au fil de la ligne (ce qui existe, ce que la commande précédente vient de créer ou de déplacer).

C'est ce qui conduit `demo_cli` à un *moteur* : un analyseur, une table de règles qui décrit la sémantique des outils, et une simulation sur une vue virtuelle du système de fichiers (chapitre 3).

2.6 La surveillance à l'exécution

Ce qu'aucune lecture ne peut prédire — un script Python, un `make`, une commande construite à l'exécution — ne peut être vu qu'en *regardant* la commande travailler. Les systèmes d'exploitation offrent plusieurs points d'observation.

ptrace (Linux)
L'appel système `ptrace` permet à un processus traceur « d'observer et de contrôler l'exécution d'un autre processus, et d'examiner et modifier sa mémoire et ses registres » ; il sert d'abord aux débogueurs et au traçage des appels système [21]. Un traceur peut s'arrêter à l'entrée de chaque appel système du processus suivi — donc *avant* qu'un `unlink` ne s'exécute.

seccomp avec notification (Linux)
Un filtre seccomp peut déléguer la décision sur un appel système à un processus superviseur en espace utilisateur, plus rapidement qu'avec ptrace puisque seuls les appels filtrés sont notifiés. La page de manuel avertit toutefois que ce mécanisme « n'est pas destiné à implémenter une politique de sécurité », notamment à cause d'une situation de concurrence entre la vérification et l'usage des arguments [22].

Landlock (Linux)
Landlock permet à un processus, même non privilégié, de restreindre ses propres droits d'accès [23]. C'est un outil de *restriction* — celui des bacs à sable — pas d'observation ni de copie.

FUSE (Linux)
Un système de fichiers en espace utilisateur, dont « les données et métadonnées sont fournies par un processus ordinaire » [24]. Placé sous le projet, il voit chaque opération sur ses fichiers.

WinFsp (Windows)
L'équivalent de FUSE pour Windows : il permet d'écrire un système de fichiers comme un programme en mode utilisateur, sans programmation noyau [25]. Windows n'ayant pas de ptrace, c'est l'endroit où toutes les routes (Win32, ntdll, appels système directs) convergent avant d'atteindre les fichiers.

Pilotes de filtre noyau (Windows)
Les *minifilters* interceptent les opérations du système de fichiers dans le noyau. Ils sont puissants mais exigent un pilote signé et une programmation noyau.

Observer n'est pas comprendre, mais c'est un plancher

Un filet d'exécution ne sait pas *pourquoi* un fichier est détruit ; il sait seulement *qu'il va l'être*, au moment où l'appel système arrive. C'est exactement ce qui manque pour le code que `demo_cli` ne lit pas : on ne peut pas obscurcir un appel système — `rm`, quelle que soit la façon dont il est écrit, devient un `unlink` sur un chemin résolu par le noyau. Le filet ne voit cependant que les fichiers locaux : une base distante ou un dépôt distant lui sont invisibles.

2.7 Synthèse et positionnement de `demo_cli`

Le tableau 2.1 compare les approches selon six critères tirés de la problématique.

Table 2.1. Les protections existantes face aux critères du problème

	Prédit l'effet	**Récupère**	**Couvre le shell**	**Traite le distant**	**Sans friction**	**Linux et Windows**
Demande de permission	non	non	**oui**	partiel (la personne)	non	**oui**
Juge automatique	partiel	non	**oui**	partiel	**oui**	**oui**
Bac à sable	non	non	**oui**	partiel (réseau)	**oui**	partiel
Points de restauration de l'agent	non	partiel (édition)	non	non	**oui**	**oui**
Git, sauvegardes	non	partiel	partiel	non	**oui**	**oui**
Motifs (regex)	partiel	non	**oui**	partiel	**oui**	**oui**
Filet d'exécution seul	non	**oui** (local)	**oui**	non	**oui**	partiel
`demo_cli`	**oui**	**oui**	**oui**	**oui** (ask/deny)	partiel	**oui**

Aucune protection existante ne réunit les deux propriétés centrales : *prédire* l'effet d'une ligne et *récupérer* ce qu'elle détruit. `demo_cli` se positionne donc comme une couche qui *s'ajoute* aux autres, avec quatre partis pris :

1. **Prédire par la structure et la simulation**, pas par les motifs.
2. **Récupérer d'abord**, ne demander que ce qui ne peut pas être récupéré, et refuser ce dont il n'existe aucun retour réaliste.
3. **Un filet d'exécution comme plancher** pour ce qui ne peut pas être lu.
4. **Ne jamais revendiquer une protection non prise** : chaque décision dit ce qui a été capturé, et ce qui ne l'a pas été.

La case « sans friction » reste partielle : une escalade interrompt l'agent, et c'est voulu pour ce qui ne peut pas être récupéré.

Conclusion du chapitre

Les protections existantes sont réelles mais orthogonales au problème : les demandes de permission s'usent, les bacs à sable laissent le projet inscriptible, les points de restauration ignorent le shell, et l'analyse syntaxique s'arrête à la structure. Le chapitre suivant décrit l'architecture qui comble ce vide : comment `demo_cli` lit, simule, décide, récupère et surveille.

CHAPITRE 3

Analyse et conception

Ce chapitre est le cœur du rapport. Il fixe les besoins, le modèle de menace et l'invariant, puis présente l'architecture — ce que l'outil lit, voit, refuse, demande et enregistre. Il expose ensuite les principes qui gouvernent chaque décision, avec, pour chacun, la commande qui casse l'approche naïve et le test qui fige la règle. Il décrit enfin la conception du moteur, de la politique et du décideur unique, la répartition entre Linux et Windows, et résume les choix d'architecture face à leurs alternatives.

3.1 Analyse des besoins

3.1.1 Acteurs

Trois acteurs interagissent avec `demo_cli` : l'**agent**, qui propose des commandes et des modifications de fichiers ; la **personne** (le développeur), qui installe l'outil, répond aux questions qu'il pose et restaure ce qu'il a capturé ; et les **systèmes tiers** touchés par les commandes (système de fichiers local, bases de données, dépôts git distants, fournisseurs cloud).

3.1.2 Besoins fonctionnels

3.1.3 Besoins non fonctionnels

3.2 Modèle de menace et invariant

3.2.1 Contre quoi `demo_cli` protège

Le modèle de menace est écrit dans le code (`protection.py`) et affiché par `demo_cli doctor` :

Modèle de menace (`protection.py`)

« demo_cli protects against an agent's MISTAKES, not a determined adversary : commands it understands are judged before they run ; for commands it cannot read, the runtime net (syscall guard on Linux, WinFsp mount on Windows) is the floor ; a remote change that can lose data or cannot be recalled is stopped for review ; one that only adds […] is allowed and listed. »

Table 3.1. Besoins fonctionnels

Id	Besoin
BF1	Intercepter, avant exécution, chaque commande shell (POSIX et PowerShell) et chaque modification de fichier proposée par l'agent, par toutes les portes disponibles.
BF2	Prédire l'effet de la commande : ce qu'elle détruit, crée, déplace, et ce qu'elle atteint à distance.
BF3	Rendre une décision parmi six : ALLOW, REVERSIBLE, DRY_RUN, SUPERVISED, CONTEXT_MISMATCH, ESCALATE ; pour une escalade, indiquer si une personne peut l'approuver sur le champ.
BF4	Capturer, avant l'exécution, une copie exacte de ce qui sera détruit, en un seul groupe par commande, restaurable par `demo_cli undo <id>`.
BF5	Prévisualiser une requête SQL destructrice (nombre de lignes touchées).
BF6	Placer sous un filet d'exécution ce qui ne peut pas être lu (Linux et Windows).
BF7	Refuser toute modification des fichiers de `demo_cli` lui-même par l'agent.
BF8	Enregistrer chaque décision dans un registre infalsifiable, vérifiable par `demo_cli verify`.
BF9	Diagnostiquer l'installation (`demo_cli doctor`) et dire, par agent et par système, ce qui est protégé et ce qui ne l'est pas.

Table 3.2. Besoins non fonctionnels

Id	Qualité	Exigence et justification
BNF1	Honnêteté	Une décision ne revendique jamais une protection non prise ; la raison dit ce qui a été capturé et ce qui ne l'a pas été.
BNF2	Sûreté par défaut	Une *décision* incertaine va vers le plus strict ; une ligne non lue n'est jamais ALLOW.
BNF3	Ne pas bloquer l'agent sur une panne	Une erreur interne de `demo_cli` ne doit pas paralyser l'agent : elle laisse passer, bruyamment ; mais une panne du *moteur* ne rend jamais une décision plus permissive (B0).
BNF4	Latence	Le moteur dispose de 10 ms de temps CPU (50 ms pour une seconde tentative) et d'un plafond de 250 ms en temps réel ; les familles d'outils ne se chargent qu'à la demande. Un outil lent est un outil désinstallé.
BNF5	Portabilité	Linux et Windows (PowerShell, Git Bash), avec les mêmes garanties ; chaque partie est vérifiée sur les deux systèmes.
BNF6	Non-régression	Aucune décision ne devient plus permissive sans arbitrage explicite (portail de 922 commandes).
BNF7	Confidentialité	Les reçus masquent les secrets (mots de passe d'URL, clés) ; aucune télémétrie.

L'adversaire visé est donc un agent *coopératif qui se trompe* : il veut nettoyer un dossier et en supprime un autre, il croit qu'un répertoire existe, il réinitialise une base qu'il croit de test. Ce choix a deux conséquences assumées.

— Ce qui ne relève pas d'une erreur plausible mais d'une évasion délibérée (une charge chiffrée, une course entre la vérification et l'usage d'un appel système) n'est pas l'objectif premier ; le filet d'exécution en réduit cependant la portée.

— En revanche, l'agent ne doit pas pouvoir *désarmer* l'outil par accident ou par excès de zèle : modifier la configuration de `demo_cli`, désinstaller son *hook*, effacer son registre. Ces actions sont refusées à toutes les portes (section 3.4.5).

3.2.2 L'invariant

Toute la logique de décision découle d'une phrase, héritée de la bêta et gravée dans chaque reçu (champ `invariant`) :

Invariant

`mutating_actions_must_be_recoverable_and_match_the_declared_context`

Une action qui modifie doit être récupérable *et* conforme au contexte déclaré ; sinon, elle est escaladée.

3.2.3 Fermé pour les décisions, ouvert pour les pannes

Un outil de sécurité branché sur un agent fait face à une tension : s'il échoue « fermé », le moindre défaut interne bloque tout le travail de l'agent et l'outil est désinstallé ; s'il échoue « ouvert », un défaut devient une brèche. `demo_cli` tranche ainsi :

— une **décision** est toujours fermée : sans point de récupération, une modification est escaladée dans *tout* environnement, développement compris ;

— une **erreur interne** de l'adaptateur (entrée illisible, exception) laisse passer la commande, mais *bruyamment* : un avertissement visible, jamais un silence ;

— une **défaillance du moteur** (dépassement de son budget, exception) ne rend jamais une décision plus permissive : l'ancien chemin peut rester plus strict, mais son ALLOW devient `engine_unavailable`, puis filet, point de contrôle ou escalade (partie B0).

3.3 Architecture d'ensemble

3.3.1 Le flux d'une commande

La figure 3.1 suit une commande de l'agent jusqu'au reçu.

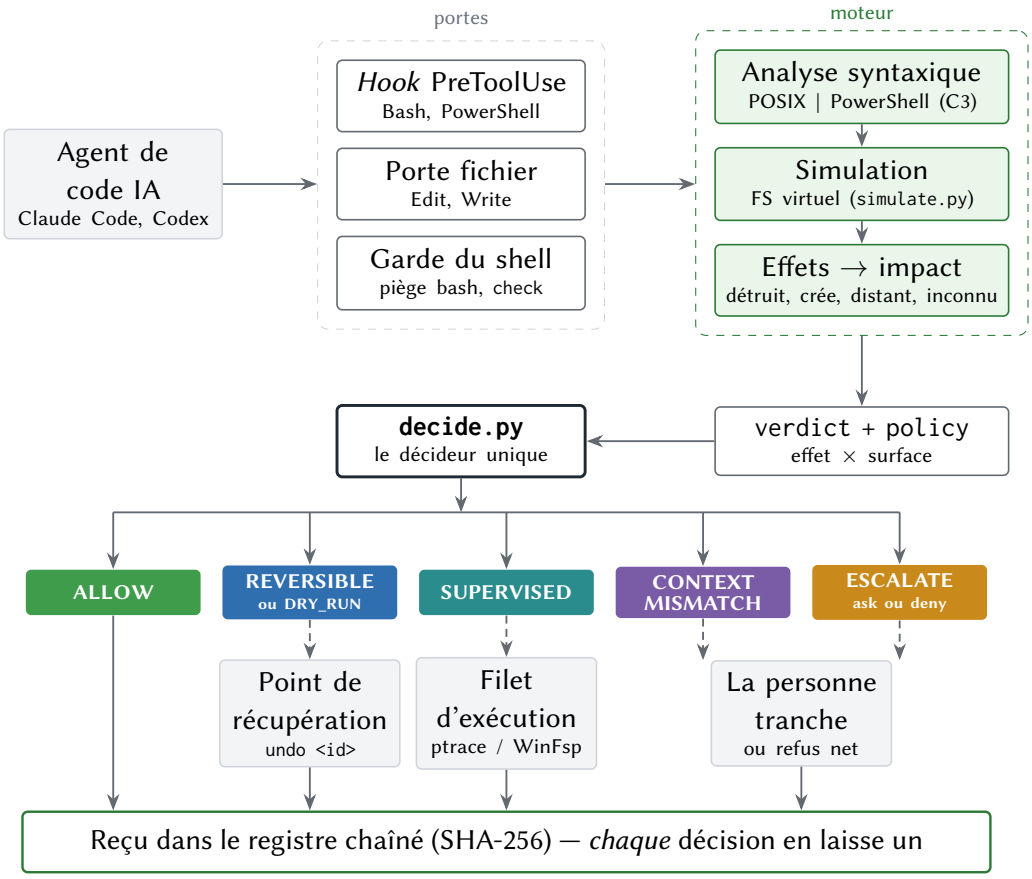


Figure 3.1. Le flux d'ensemble : de la commande de l'agent au reçu

1. **La porte** reçoit l'action : le *hook* PreToolUse de Claude Code (outils `Bash` et `PowerShell`), la porte fichier (`Edit`, `Write`), le *hook* de Codex ou de Cursor, le piège du shell pour les commandes tapées dans un terminal, ou `demo_cli check`. Chaque porte ne fait que traduire : elle passe la commande à l'orchestrateur et traduit la décision dans ce que son agent comprend.
2. **L'orchestrateur** (`guard.py`, `Guard.evaluate`) enchaîne des étapes nommées : refus d'une configuration cassée ; prédiction du moteur ; refus de ce qui touche le plan de contrôle ; choix du chemin (moteur, ligne non lue, ancien chemin) ; contexte ; décision ; reçu.
3. **Le moteur** analyse la ligne, la simule sur une vue virtuelle du système de fichiers et produit un *impact* : ce qui est détruit, créé, atteint à distance, non compris, non couvert.
4. **La politique** (`policy.py`) traduit chaque effet distant en décision, chaque étiquette en surface, et chaque surface en canal « demander » ou « refuser ».
5. **Le décideur** (`decide.py`), fonction pure sans entrée-sortie, rend la décision. Il ne capture rien lui-même : il reçoit seulement le fait qu'une capture a *réussi*. C'est ce qui l'empêche

de mentir.

6. **Les conséquences** : une capture (REVERSIBLE), un filet (SUPERVISED), une question à la personne ou un refus (ESCALATE) — et, dans tous les cas, un reçu.

3.3.2 Ce que l'outil voit

La figure 3.2 répond à la question qu'un jury pose en premier : *que voit réellement l'outil, et où s'arrête-t-il ?* Elle croise six couches — lire, récupérer, surveiller, refuser, demander, tracer — avec trois zones où une commande peut agir.

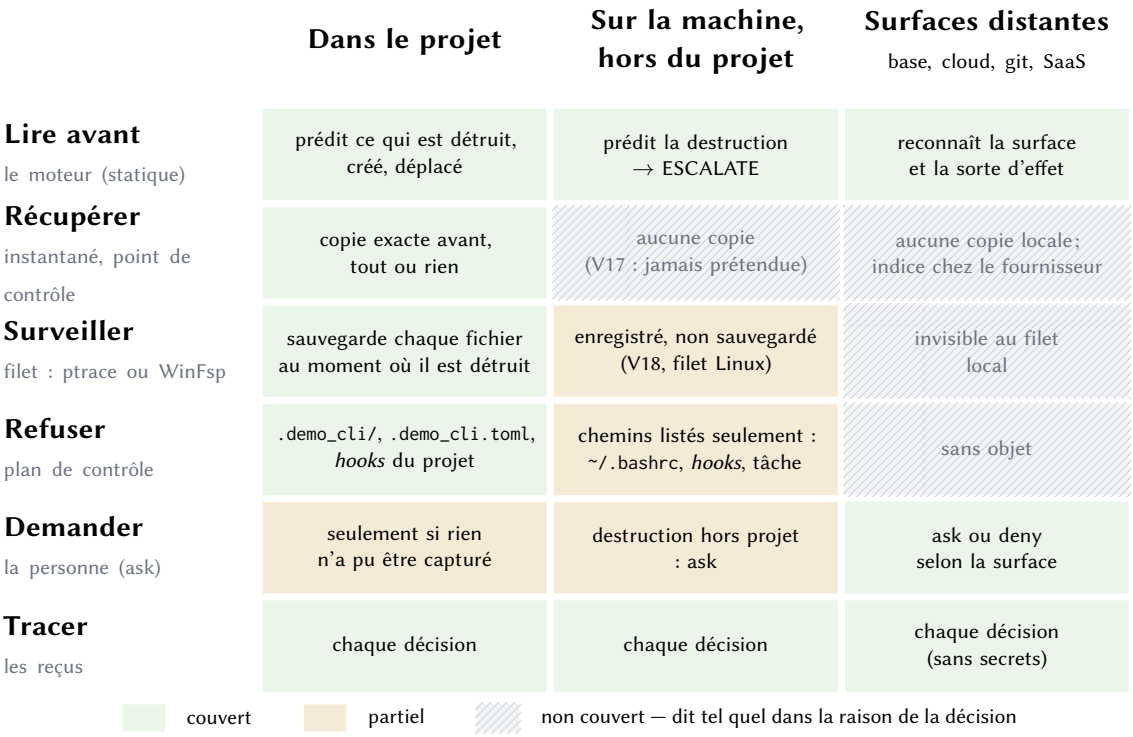


Figure 3.2. Ce que l'outil voit : chaque couche et ce qu'elle couvre, selon l'endroit où la commande agit

Trois lectures de cette figure méritent d'être soulignées.

— **Seule la lecture voit partout** — mais seulement ce qu'elle sait lire. Tout le reste du dispositif existe pour ce qu'elle ne sait pas lire.

— **La récupération et le filet s'arrêtent au projet.** Hors du projet, aucune copie n'est possible, et la décision le dit ; une destruction connue hors projet est escaladée, et celle que fait du code non lu est seulement enregistrée par le filet Linux (V18).

— **Le distant ne se récupère pas, il se décide.** Pour une base ou un dépôt distant, `demo_cli` ne peut que reconnaître la surface, demander ou refuser, et indiquer où le fournisseur garde son propre historique.

3.4 Les principes qui gouvernent les décisions

Chaque principe ci-dessous est présenté de la même façon : l'énoncé, la commande qui casse l'approche naïve, la règle retenue et le test qui la fige. Les sorties de terminal sont réelles (commit `7d2a2aa`, Linux, sans filet d'exécution sauf mention contraire) ; seules les lignes sans intérêt sont coupées et les chemins abrégés en `~/projet`.

3.4.1 Analyser et simuler plutôt que reconnaître des motifs

Le constat. La bêta et la version 1.7.0 lisaient les commandes avec un découpeur de texte et des expressions régulières. L'audit et la revue menés au début de la période documentée ont trouvé, encore et encore, des défauts de la même forme : *deux analyseurs répondant différemment à la même question* — le découpeur de lignes, le normaliseur de mot de commande, le masqueur de citations, le lecteur de *heredocs*, le découpeur d'opérandes. Deux de ces défauts avaient été introduits par des corrections des autres (`tests/test_command_model.py`).

Ce qui casse. Le shell n'est pas un texte mais une grammaire. Un commentaire neutralise ce qui le suit (`ls # ; rm -rf x` ne supprime rien) ; une substitution `$(. . .)` exécute une commande cachée dans un argument (`git commit -m "$(rm y)"`) ; une enveloppe déplace le vrai mot de commande (`sudo -u x rm`, `bash -o pipefail -c "rm app.db"`, `env -S 'rm -rf x'`) ; un `case` ou une fonction contiennent des commandes que le découpage par `;` ne voit pas ; une citation ANSI-C (`$'\x72m'`) écrit `rm` sans les lettres `r` et `m`.

La règle retenue. Une seule analyse de la ligne (`model.py`), lue par tout le reste. Avant de remplacer le découpeur, elle a tourné en ombre : sur toutes les commandes de la suite de tests (801, dans les deux dialectes), elle a rendu la même décision que le découpeur ; sur un jeu étiqueté à la main de ce que bash exécute réellement, elle avait raison dans 22 cas où le découpeur avait tort, et tort dans aucun cas où il avait raison. Ces deux chiffres justifient le remplacement plutôt que la cohabitation.

Et la simulation ? Une structure correcte ne suffit pas (section 2.5) : `mv a.txt b.txt` n'est destructeur que si `b.txt` existe. Le moteur rejoue donc la ligne sur une vue virtuelle du système de fichiers et suit l'origine des contenus : dans `mv a b && rm b`, ce qui est perdu est le contenu de `a`. C'est exactement le raisonnement qui manquait à l'agent dans l'incident Gemini CLI.

3.4.2 L'inconnu n'est pas sûr : « non couvert » et « non compris »

Deux ignorances différentes. Le moteur distingue :

— **non couvert** (*uncovered*) : la ligne est lue, mais un outil n'a pas de règle. On sait *ce qui* est appelé, avec quels arguments ;

— **non compris** (*not understood*) : une partie de la ligne ne peut pas être lue avant l'exécution — une variable affectée ailleurs, une substitution dont la sortie n'est pas prévisible, un mot de commande construit.

La règle retenue (parties M2.3a et M2.3b, arbitrages M1–M5 et Rb1–Rb4).

1. Une ligne **non comprise** va au filet d'exécution s'il est disponible, sinon à un point de contrôle s'ils sont activés, sinon elle est **escaladée**. Elle n'obtient jamais ALLOW.
2. Un **exécuteur de code** (`python`, `node`, `make`, `npm`, `./script.sh`, `source`…) lance du code que `demo_cli` ne lit pas : il va au filet, sinon au point de contrôle, sinon il est autorisé *avec une note qui dit qu'aucune capture n'a été faite* (arbitrage M1). Il garde le filet même si on lui passe un fichier hors du projet (V21).
3. Deux cas d'exécuteur sont plus risqués et **escaladent sans filet** (supervisés avec, arbitrage Rb2) : du **code en ligne qui lance d'autres programmes** (`python -c ". . .os.system(. . .)"`, `node -e ". . .child_process. . ."`) et les **installations de paquets** (`pip install`, `npm install`…), qui exécutent les scripts d'installation de chaque paquet.
4. Une ligne qui atteint quelque chose au-delà des fichiers locaux sans être lue (cloud, conteneurs, services, une option inconnue sur un outil distant, `kubectl exec`, `heroku run`…) est escaladée : ni le filet ni le point de contrôle ne la voient. Le code récupéré du réseau puis exécuté (`curl . . . | bash`) garde son refus.
5. Un mot sans règle qui n'est pas un exécuteur de code n'a pas d'effet (arbitrage Rb1).

```
• • •  ligne non comprise, pas de filet

$ demo_cli check --mode enforce 'rm $INCONNU'
demo_cli 1.7.0  [x] BLOCKED  ·  ESCALATE
  matched rule        not_understood
Decision
  No recovery path for a not_understood on 'unknown'; cannot auto-recover.
  Human input required.
$ demo_cli check --mode enforce 'python3 outil.py'
demo_cli 1.7.0  [+] SAFE  ·  ALLOW
Decision
  Allowed: it runs code demo_cli does not read (python3); no runtime net was
  available, so nothing that code changes is captured or watched.
$ demo_cli check --mode enforce 'pip install requests'
demo_cli 1.7.0  [x] BLOCKED  ·  ESCALATE
Decision
  No recovery path for a unread_code on 'unknown'; cannot auto-recover. Human input
  required. It installs packages, which run their own install scripts (pip), and no
  runtime net was available to capture what that changes.
```

Pourquoi l'exécuteur sans filet est-il autorisé ?

Escalader chaque `python script.py`, `make` ou `npm test` rendrait l'outil inutilisable : c'est l'essentiel du travail d'un agent. L'arbitrage M1 accepte ce cas *à visage découvert* : la raison de la décision dit qu'aucune capture n'a été faite. Les deux cas où l'exécuteur lance à coup sûr du code étranger — un programme démarré depuis du code en ligne, une installation de paquets — escaladent en l'absence de filet (Rb2). Là où le filet existe (Linux avec Claude Code, Windows avec le montage), ces lignes sont SUPERVISED. Un outil inconnu qui n'est pas un exécuteur de code (« non couvert ») est, lui, traité comme sans effet (Rb1) ; la table de règles s'étend famille par famille pour réduire ce cas, et un test (`test_m22f.py`) vérifie qu'aucune ligne du portail sur laquelle une ancienne règle se déclenchait n'est restée hors du moteur sans être marquée non comprise.

Lire plutôt qu'abandonner. « Non compris » doit rester rare, sinon tout escalade. Le moteur lit donc ce qui peut l'être sans rien exécuter : la sortie de `$(cat liste.txt)`, `echo`, `printf`, `pwd`, `basename`, `dirname`, sous des conditions strictes (fichier régulier du projet, UTF-8, au plus 64 Kio, non modifié plus tôt sur la ligne ; partie I1b). La partie M2.3b a ajouté ce que l'ancien chemin savait capturer : une valeur connue sur la ligne lue comme l'option qu'elle est (`X=-rf` ; `rm $X src`), un script littéral envoyé à un shell par un tube (`echo 'rm -rf build' | sh`), `sqlite3 $DB` et `sqlite3 app.db "$(cat q.sql)"`, les messages de commit, et `npx`, `bunx` ou

`pnpm dlx` qui lancent un outil connu (`npx prisma migrate reset` est lu comme `prisma`) :

```
• • •  sortie de substitution lue par le moteur

$ demo_cli check --mode enforce 'rm $(cat list.txt)'
demo_cli 1.7.0  [+] SAFE  ·  REVERSIBLE
  target              ~/projet/a.txt
Recovery
  snapshot            a.txt.20261009-175242.6d6ca4f4.bak
Decision
  Recovery point captured before the action; reversible.
```

3.4.3 Jamais un REVERSIBLE non mérité

L'énoncé. REVERSIBLE est une promesse : « `demo_cli undo` remettra tout en place ». Une capture partielle, ou une copie qui ne contient pas ce que la commande détruit, rendrait cette promesse fausse, ce qui est pire qu'une escalade : la personne cesse de se méfier.

Les règles qui en découlent.

— **Tout ou rien** : les captures d'une commande forment un groupe ; si l'une échoue, les autres sont invalidées et la commande est escaladée (partie S2, `recovery.snapshot_set`).

— **Pas de point de contrôle pour ce qu'il ne contient pas** : une copie du projet ne restaure ni un fichier hors du projet (V17), ni une base distante, ni un historique git poussé de force.

— **Hors projet, rien n'est promis** : une destruction connue hors du projet est escaladée, même à côté d'une partie non comprise.

— **Le décideur ne capture pas** : `decide.py` ne reçoit qu'un booléen « une capture de la cible réelle a réussi », fourni par l'orchestrateur ; il n'existe aucune cible par défaut sur laquelle se rabattre.

3.4.4 Pendant la migration, la réponse la plus stricte gagne

Le problème. Le moteur a remplacé l'ancien chemin *progressivement*, famille par famille. Pendant ce temps, deux lecteurs coexistent : le moteur et les anciennes règles. Que faire quand ils divergent ?

La règle retenue (parties B0 et M2.3a). L'ancien chemin peut garder une *capture* ou une *escalade*, jamais son ALLOW. Exemple réel, au moment de M2.3a : le moteur marquait `X=-rf` ;

`rm $X build` comme non compris (une valeur qui commence par un tiret) ; l'ancienne règle, elle, savait capturer `build`. La réponse la plus stricte — capturer — l'emportait. À l'inverse, quand l'ancienne règle aurait autorisé une ligne que le moteur ne lit pas, c'est la règle M3 qui s'applique (filet, point de contrôle, escalade) : la vulnérabilité V19 était précisément un `psql -c "$(cat q.sql)"` autorisé alors que `q.sql` contenait `DROP TABLE users;`. Il est désormais escaladé.

La fin de la migration (M2.3b). La partie M2.3b a fait lire au moteur tout ce que l'ancien chemin capturait encore ; depuis, **l'ancien chemin ne décide plus aucune ligne POSIX**. La même ligne est maintenant capturée par le moteur lui-même :

```
• • •  la ligne est lue par le moteur (M2.3b)

$ demo_cli check --mode enforce 'X=-rf; rm $X build'
demo_cli 1.7.0  [+] SAFE  ·  REVERSIBLE
  target              ~/projet/build
Recovery
  snapshot            build.20261009-175242.30656068.snapdir
Decision
  Recovery point captured before the action; reversible.
```

La règle « le plus strict gagne » reste en vigueur là où l'ancien chemin sert encore : le texte `cmd.exe`, les constructions PowerShell que C3 ne lit pas, et le cas où le moteur échoue (B0).

3.4.5 Les fichiers de `demo_cli` : le plan de contrôle

L'énoncé. Un agent qui peut modifier la configuration de son garde-fou n'a pas de garde-fou. Le *plan de contrôle* regroupe : la configuration `.demo_cli.toml`, l'espace de travail `.demo_cli/` (registre, réserve de récupération), les configurations des *hooks* dans le projet et dans le répertoire personnel, le script du garde du shell, `~/.bashrc` quand il l'active, le lanceur `demo_cli`, et sous Windows les fichiers de la tâche d'ouverture de session.

La règle. Toute modification de ces chemins — y compris une *création* ou un ajout en fin de fichier — est refusée (ESCALATE, jamais approuvable sur place), à toutes les portes : avant l'exécution par le moteur et par les anciennes règles, dans la porte fichier, et pendant l'exécution par le garde d'appels système, qui annule l'appel.

```
• • •  un agent tente de passer l'outil en mode observation

$ demo_cli check --mode enforce 'echo mode = "shadow" > .demo_cli.toml'
demo_cli 1.7.0  [x] BLOCKED  ·  ESCALATE
  matched rule        control_plane
Decision
  This changes demo_cli's own files, or a directory holding them (.demo_cli.toml):
  its config, hook settings, ledger or recovery store. An agent may not [...]
```

3.4.6 Demander ou refuser

Le problème. Une escalade arrête l'agent. Faut-il alors *demander* à la personne, ou *refuser* ? Demander tout use l'attention (section 2.2) ; refuser tout rend l'outil inutilisable pour des opérations ordinaires (supprimer un volume Docker de test, pousser une branche).

La règle retenue (partie ASK). L'escalade reste une escalade — aucune décision du portail ne change — mais chaque surface porte un *canal* dans `policy.ESCALATION_CHANNEL` :

- **ask** quand le changement est réel mais ordinaire, et que la commande exacte suffit à la personne pour savoir à quoi elle consent (base distante, historique git, bucket, conteneur, déploiement, service…) ;
- **deny** quand tout part sans retour réaliste (infrastructure détruite, vidage d'un entrepôt de données, formatage de disque, paiement, compte utilisateur supprimé, données de récupération du système) ; seuls un jeton d'approbation structurel, que l'agent ne peut pas forger, ou une exécution hors de l'agent le permettent.

Une surface sans ligne dans la table est refusée : une surface ajoutée plus tard ne devient jamais « demandable » par accident. Enfin, *demander* n'a de sens que si quelqu'un voit la question : le *hook* Claude Code ne pose la question que dans les modes où l'interface l'affiche (`default`, `acceptEdits`, `plan`, `auto`) et refuse dans les autres — une question que personne ne voit ne doit jamais devenir « c'est passé ». Codex, qui ne sait pas demander, refuse.

3.4.7 Les faits sur les outils sont vérifiés

Une règle fausse est une décision fausse. Chaque grammaire d'options est tirée de la source de l'outil lui-même — son `-help`, sa page de manuel, son code source, ses données publiées (l'index des commandes d'`aws`, l'arbre de commandes de `gcloud`, la table de commandes de `redis`) — et vérifiée sur la version courante. Les comportements douteux sont testés sur des fichiers jetables : que `unzip -Pxo` lit `xo` comme mot de passe et non comme l'option d'écrasement (I1c, vérifié sur UnZip 6.0), que `helm 4` change la sémantique de `rollback -dry-run`, que `tofu`

accepte `-chdir DIR` sans = quand `terraform` le refuse. Un test confronte chaque grammaire au `-help` de l'outil installé. Une règle dont la grammaire vient seulement de la documentation porte `verified=False` : c'est le cas de 9 règles sur 72.

3.5 Conception du moteur

3.5.1 Vue d'ensemble

Le moteur (`src/demo_cli/engine/`) transforme une ligne en un *impact*, sans rien exécuter ni rien écrire. La figure 3.3 le montre sur un exemple réel.

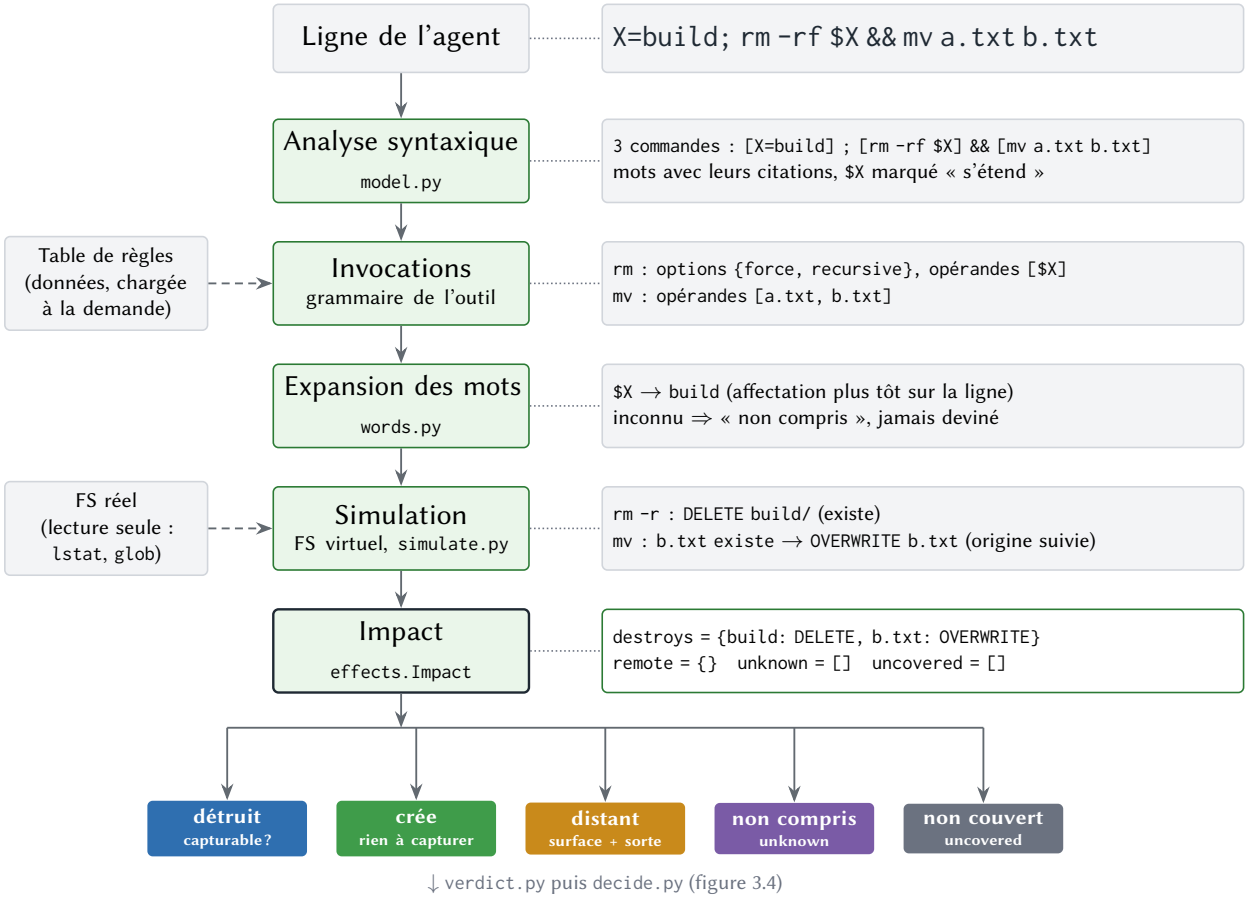


Figure 3.3. L'intérieur du moteur, sur un exemple réel (sortie du moteur au commit `7d2a2aa`)

3.5.2 Les étapes

Analyse syntaxique (`model.py`)

Un analyseur lexical écrit pour le shell POSIX produit des `Word` qui gardent leur texte source, leur valeur sans citations, les positions de leurs citations et de leurs substitutions, et un

indicateur « s'étend ». Les substitutions `$(...)`, `` `...` `` et `<(...)` sont analysées comme des *scripts imbriqués*, les *heredocs* sont rattachés à leur redirection, les enveloppes (`sudo`, `env`, `xargs`, `timeout`, `nice`…) sont traversées pour trouver le vrai mot de commande, et la charge d'un `bash -c` est analysée à son tour.

Invocations et grammaires (`invocation.py`)
Chaque commande devient une invocation : un outil, des options canoniques, des opérandes, des redirections. Les options sont lues avec la grammaire propre à l'outil (options groupées, valeurs collées comme `sed -i.bak`, permutation à la GNU).

Expansion des mots (`words.py`)
L'expansion se fait *sur les mots*, comme bash, et non sur le texte : citations simples littérales, doubles citations sans découpage, `$NOM` connu seulement s'il a été affecté plus tôt sur la ligne, accolades puis *globs* sur le texte non cité, évalués sur l'état simulé. Tout le reste lève `Unknown` avec sa raison — rien n'est deviné.

Simulation (`simulate.py`)
Les effets sont joués dans l'ordre sur une *surcouche* du système de fichiers réel, qui n'est que lu (`lstat`, `listdir`). Chaque nœud retient l'origine de son contenu. Les séparateurs sont modélisés comme des *ensembles d'états possibles* : après `a && b`, `b` part de l'état où `a` a réussi ; après `a || b`, de l'état où `a` a échoué — avant `a` ou en partie ; les fusions sont des unions, donc elles ne peuvent qu'*ajouter* à ce qui est détruit. Les sous-shells copient le répertoire courant et les variables mais gardent leurs effets sur les fichiers.

Impact (`effects.py`)
Le résultat : `destroys` (indexé par le chemin *préexistant* dont le contenu est perdu), `creates`, `touched` (tout chemin changé, pour le plan de contrôle), `remote` (surface → sorte d'effet), `unknown`, `uncovered`, et les bases de données capturables ou non.

3.5.3 Le langage des effets

Un vocabulaire unique décrit ce qu'une commande fait au monde (tableau 3.3). Quand une même surface distante est atteinte deux fois, elle garde la sorte qui perd le plus, selon l'ordre DELETE > OVERWRITE > SEND > RUN > LATER > SERVICE_LOSS > REF > SERVICE_KEEP > CREATE.

3.5.4 La table de règles, écrite comme des données

Une règle (`engine/rules.py`) associe un ou plusieurs noms d'outil à une grammaire d'options, une condition sur les options, des effets, et des **exemples**. Les exemples ne sont pas de la documentation : chacun devient un test qui exécute tout le moteur dans une arborescence fixe et compare l'impact obtenu à l'impact attendu. Un outil que les gabarits ne peuvent pas

Table 3.3. Le langage des effets (`engine/effects.py`)

Effet	Signification	Conséquence
DELETE, OVERWRITE, TRUNCATE	un chemin est supprimé, remplacé, vidé	copier avant
MOVE	un contenu passe de la source à la destination	copier la destination si elle existe
CREATE, APPEND, MKDIR	un chemin apparaît, ou reçoit un ajout	rien de préexistant n'est perdu
REF	une référence git bouge	le *reflog* la ramène (arbitrage c)
SEND	part vers l'extérieur sans rappel possible	escalade
RUN	du code s'exécute, ici ou là-bas	lu à travers ce code, ou filet
LATER	du code est planté pour s'exécuter plus tard	escalade (ask)
SERVICE_LOSS / KEEP	disponibilité perdue / gardée	escalade / autorisé et listé
UNKNOWN	quelque chose qu'on ne peut résoudre	la raison est gardée

exprimer (`git`, `aws`, `rsync`…) délègue à un *résolveur*, mais garde dans la table sa grammaire et ses exemples.

Extrait réel de la règle `rm` (`engine/rules.py`)

```
Rule(
    id="rm",
    tools=("rm",),
    options=OptionSpec(flags={
        "-f": "force", "--force": "force",
        "-r": "recursive", "-R": "recursive", "--recursive": "recursive",
        "-d": "dir", "--dir": "dir",
        # ...
    }),
    # effets et exemples : chaque exemple est un test
)
```

Au commit de référence, la table compte 72 règles, 774 exemples et 214 noms d'outils, répartis entre un noyau toujours chargé (fichiers locaux, redirections) et 12 familles chargées *à la demande* : SQL, git, infrastructure, deux familles cloud, conteneurs, entrepôts de données, envois sortants, PaaS, SaaS et migrations, système, coffres de secrets. Le chargement paresseux tient le coût d'import hors du chemin de chaque commande : une ligne `rm` ne paie jamais l'analyseur SQL. La même table génère aussi la liste des mots que le piège du shell envoie au moteur (partie M2.2e) : une famille ajoutée atteint la porte du shell sans ligne de code

supplémentaire.

3.5.5 Le budget

Le moteur s'exécute sur le chemin critique de chaque commande de l'agent. Il dispose de 10 ms de temps CPU (mesuré en temps processeur sous Linux, où l'horloge est fine), d'une seconde tentative à 50 ms en cas de dépassement, et d'un plafond de 250 ms en temps réel. Le temps d'import d'une famille et la lecture de certains fichiers sont exclus du budget. S'il ne termine pas, la règle de la section 3.2 s'applique : jamais une décision plus permissive.

3.6 La politique et le décideur unique

3.6.1 Une seule politique

Avant la partie T5, les mêmes questions — « une suppression distante escalade-t-elle ? », « quelle surface derrière l'étiquette `db:` ? », « où restaurer un bucket ? » — avaient des réponses dans plusieurs modules. Elles sont désormais réunies dans `policy.py`, que lisent le décideur, le jugement du moteur, l'orchestrateur et l'affichage du modèle de menace :

— `REMOTE_POLICY` : sorte d'effet distant → ALLOW ou ESCALATE ; une sorte inconnue escalade ;

— `_LABEL_SURFACES` : étiquette du moteur → surface (`storage:` → `object_storage`, `vcs-remote:` → `remote_vcs_history`…) ;

— `ESCALATION_CHANNEL` : surface → ask ou deny ;

— `RECOVERY_HINTS` : surface → où le fournisseur garde son propre historique (« bucket versioning, if enabled », « the database's PITR / snapshot »).

3.6.2 La table de décision

La figure 3.4 résume ce que la politique et le décideur produisent selon *où* l'effet tombe et *quelle sorte* d'effet il est. Chaque case a été vérifiée en faisant passer des commandes réelles par `Guard.evaluate`.

3.6.3 Le décideur

`decide.py` est une fonction pure : elle ne lit aucun fichier, ne capture rien, et reçoit tout ce dont elle a besoin — la classification, l'environnement, le fait qu'une capture de la *vraie* cible a

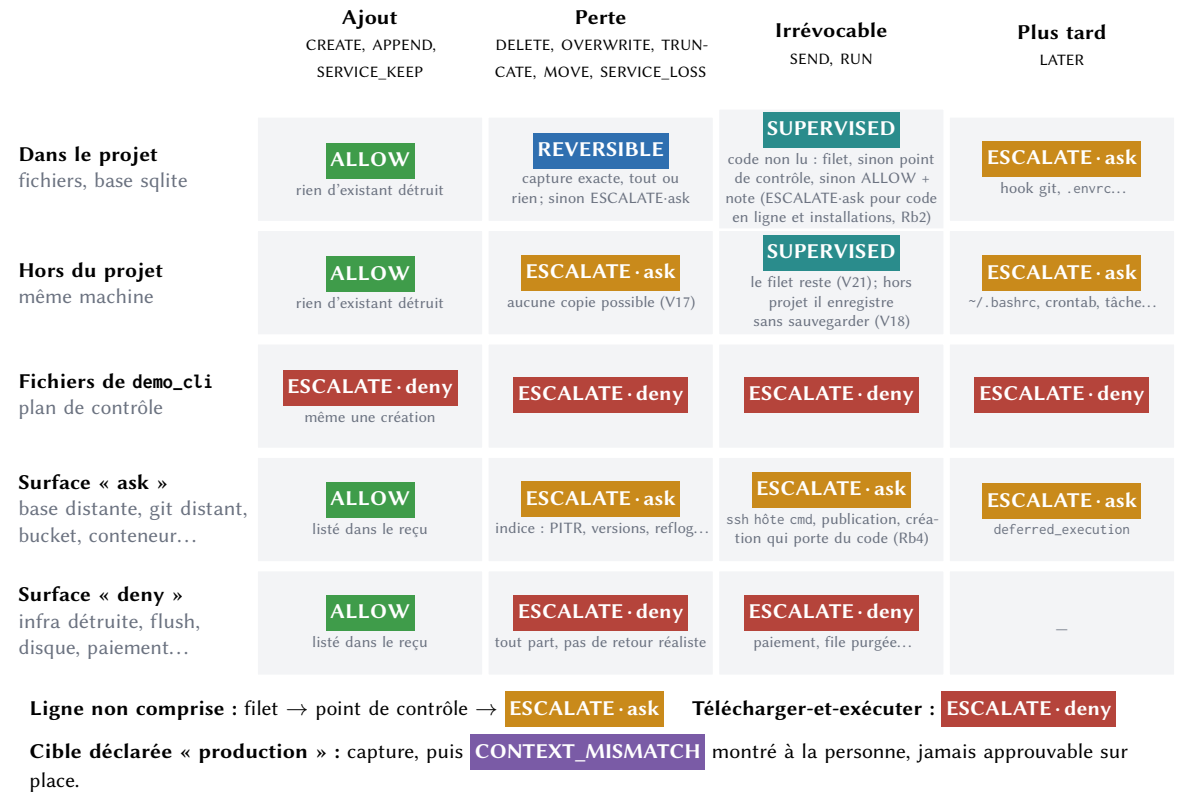


Figure 3.4. La table de décision : où l'effet tombe × quelle sorte d'effet

réussi, les désaccords de contexte, la disponibilité d'un filet. Ses branches s'évaluent dans un ordre fixe (tableau 3.4) ; la première qui s'applique décide.

> **Pourquoi un seul décideur ?**
>
> Avec deux chemins de décision, il y a deux réponses possibles à la même question ; c'est la forme même des défauts trouvés par l'audit. Le moteur, l'ancien chemin, la porte fichier et le chemin des lignes non lues ne font que *rassembler des faits* ; seul `decide.py` conclut. Une correction de politique s'applique donc partout à la fois, sur les deux systèmes, et le portail de décision n'a qu'un seul point à observer.

3.6.4 Six décisions, trois postures

Six décisions sont précises mais techniques. Pour la personne, elles sont regroupées en trois postures : **SAFE** (ALLOW, DRY_RUN, REVERSIBLE, SUPERVISED — la commande s'exécute), **REVIEW** (CONTEXT_MISMATCH — capturé, puis montré), **BLOCKED** (ESCALATE). La décision exacte reste affichée en sous-titre, comme dans les sorties de terminal de ce chapitre (« `[+] SAFE · REVERSIBLE` »).

Table 3.4. L'ordre des branches de `decide.py`

	Situation	Décision
1a	Code illisible exécuté sur une surface nommée (`ssh hôte cmd`, `gh workflow run`)	ESCALATE, canal de la surface
1b	Télécharger et exécuter (`curl . . . | bash`)	ESCALATE, deny
2	Rien ne modifie	ALLOW
3	Surface non récupérable	ESCALATE (ALLOW avec jeton structurel)
4	Désaccord avec le contexte déclaré	CONTEXT_MISMATCH si capturé, sinon ESCALATE
4b	Cible *déclarée* production, capturée	CONTEXT_MISMATCH
5	Capture réussie	DRY_RUN (aperçu SQL) ou REVERSIBLE
6	Un filet couvre le projet	SUPERVISED
7	Sinon	ESCALATE (ask)

3.6.5 Les escalades sans surface

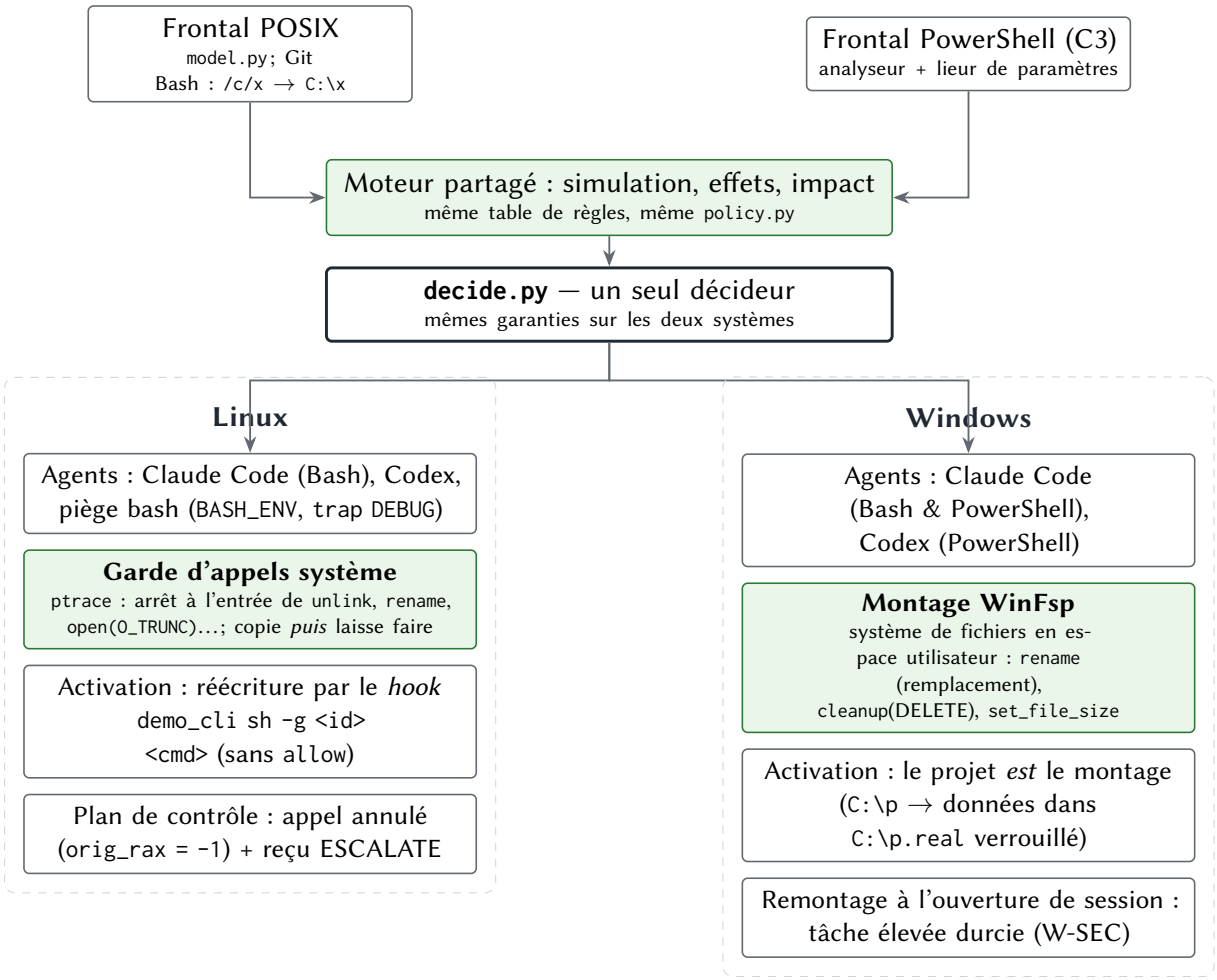
Trois escalades ne portent pas de surface et ont leur propre canal (`NO_SURFACE_CHANNEL`) : une modification locale que rien n'a pu capturer (ask), un désaccord de contexte sans capture (ask), et le téléchargement-exécution (deny), dont personne ne peut voir la charge avant qu'elle ne s'exécute.

• • • une surface distante : la personne est consultée, l'indice de restauration est donné

```
$ demo_cli check --mode enforce 'git push --force origin main'
demo_cli 1.7.0  [x] BLOCKED  ·  ESCALATE
  matched rule        git_force_push
  surface             remote_vcs_history
Decision
  Mutation of a non-recoverable surface (remote_vcs_history); a local snapshot
  cannot cover its blast radius.
Safe next step
  1. Confirm the external effect is intended and authorised.
  2. Supply a structural approval token, or perform it outside the agent.
  3. If it already ran, restore from the provider: a teammate's
     un-force-pushed clone or the remote's reflog.
```

3.7 Deux systèmes, un décideur

Les agents ne travaillent pas de la même façon sous Linux et sous Windows : sous Windows, Claude Code dispose d'un outil `PowerShell` à côté de son outil `Bash` (Git Bash), et Codex écrit du PowerShell. Les chemins s'écrivent `C:\x` ou `/c/x`, il n'y a pas de ptrace, et les appels système n'ont rien de commun. La conception sépare donc ce qui *doit* être commun de ce qui *ne peut pas* l'être (figure 3.5).



Les deux filets sauvegardent le *projet* seulement ; hors projet, le filet Linux enregistre sans sauvegarder. La raison de la décision le dit.

Figure 3.5. Linux et Windows : un seul décideur, deux filets d'exécution

Commun

Le moteur, la table de règles, la politique, le décideur, le format des reçus, la récupération et undo. Une ligne POSIX tapée dans Git Bash sous Windows passe par le même moteur que sous Linux ; ses chemins `/c/...` sont traduits en `C:\...` (partie M2.1), et un chemin POSIX absolu dont la racine MSYS n'est pas connue (`/tmp`, `/usr`) est marqué non compris plutôt que deviné.

Propre à chaque système

Le filet d'exécution et son activation, la protection du plan de contrôle pendant l'exécution, et le frontal de lecture : POSIX d'un côté, PowerShell de l'autre (C3, section 4.7).

3.7.1 Linux : pourquoi ptrace

Le filet Linux est un *garde d'appels système* fondé sur ptrace (`syscall_guard.py`). Il lance la commande comme processus suivi et s'arrête à l'*entrée* de chaque appel système ; pour un appel destructeur (`unlink`, `unlinkat`, `rmdir`, `rename*` sur une destination existante, `truncate`, `open` avec `O_TRUNC` ou en écriture) sur un fichier préexistant du projet, il copie le fichier *puis* laisse l'appel se faire. Pour un fichier du plan de contrôle, il annule l'appel.

Trois mécanismes étaient envisageables :

— **FUSE** : monter un système de fichiers sous le projet. Il a été écarté parce que le garde d'appels système fait déjà la même chose — voir chaque destruction dans le projet avant qu'elle n'ait lieu. De plus, un montage FUSE suppose de placer le projet derrière un point de montage servi par un démon permanent, alors que ptrace ne s'attache qu'à la commande concernée.

— **seccomp avec notification** : un filtre BPF installé dans le processus ne réveille le superviseur que pour les appels filtrés ; c'est plus rapide (environ 9 fois sur des boucles qui lancent beaucoup de processus, d'après nos mesures). Mais il demande Linux 5.0 au moins (5.5 pour laisser l'appel continuer), un filtre à installer avant l'`exec` du processus (avec `no_new_privs`), et la lecture des arguments dans la mémoire du processus par `/proc/<pid>/mem`. Surtout, un autre thread du processus peut modifier les arguments entre leur lecture par le superviseur et l'exécution de l'appel : la page de manuel met en garde contre l'usage de ce mécanisme pour une politique de sécurité [22].

— **ptrace** : retenu pour le prototype. ptrace peut *modifier* l'appel, pas seulement l'observer : le garde s'arrête à l'entrée de l'appel, lit les arguments dans les registres du processus, sauvegarde le fichier, puis laisse passer l'appel — ou l'annule en remplaçant son numéro par une valeur invalide. Il est disponible partout sans prérequis (ni noyau récent, ni filtre BPF à compiler), suit les processus enfants (`fork`, `clone`, `exec`) par de simples options, et tient en quelques dizaines de lignes de Python (`ctypes`). Pour un prototype qui devait d'abord prouver la *justesse* — sauvegarder exactement le bon fichier —, c'était le choix sûr. Ses limites sont assumées et rappelées au chapitre 5 : architecture x86-64, coût d'un arrêt par appel système, et pas de vision au-delà du projet.

> **seccomp : l'étape suivante, pas le point de départ**
>
> seccomp avec notification remplacera ptrace pour la vitesse. ptrace reste alors la *référence fonctionnelle* : le nouveau filet sera testé contre lui, fichier par fichier.

Activation. Le filet doit s'exécuter *à la place* de la commande. Le *hook* Claude Code réécrit donc la commande en `demo_cli sh -g <id> <commande>` via `updatedInput` — *sans* répondre `allow`, pour que l'invite de permission de l'utilisateur continue de s'afficher (partie B1b) — et dit à la personne et au modèle ce qui s'exécute réellement. Un `demo_cli sh` tapé par l'agent lui-même est déballé et jugé sur ce qu'il cache ; un processus déjà suivi par un traceur étranger refuse de s'exécuter sans surveillance.

3.7.2 Windows : pourquoi un montage WinFsp

Windows n'a pas de ptrace. L'endroit où toutes les routes vers un fichier convergent — API Win32, `ntdll`, appels système directs — est le système de fichiers. `demo_cli` y place un système de fichiers en espace utilisateur WinFsp (`fsmount.py`) : le projet `C:\projet` devient le point de montage, et les données vivent dans `C:\projet.real`, verrouillé par une ACL réservée aux Administrateurs et à SYSTEM. Le montage s'exécute avec élévation, l'agent sans : l'agent ne peut pas contourner le montage en écrivant directement dans le répertoire réel (`protect.py`). Le filet n'exige donc aucune réécriture de commande : tout ce qui touche le projet passe par lui.

La logique de jugement (`fsguard.py`) est séparée de la plomberie WinFsp et testable sous Linux. Ses règles viennent de journaux d'opérations *observés* sous Windows 10, qui ont contredit l'intuition sur trois points sur quatre : le remplacement atomique (`rename` avec remplacement) détruit la destination sans aucun signal de suppression — c'est ainsi que l'outil `Write` de Claude Code enregistre chaque fichier ; la méthode `overwrite()` n'est jamais appelée, les vraies réécritures passant par une réduction de taille ; et seul `cleanup` avec le bit de suppression signale toutes les suppressions.

Un montage meurt avec la session. Une tâche planifiée à l'ouverture de session le relance ; elle s'exécute avec élévation, ce qui en fait une cible : sa conception durcie (W-SEC) est décrite à la section 5.2.

3.7.3 Points de contrôle : utiles, mais optionnels

Quand ni capture ciblée ni filet ne sont possibles, `demo_cli` peut copier tout le projet avant la commande (`checkpoint.py`) — « je ne sais pas ce que vous allez détruire, donc je préserve tout ce que vous pourriez détruire ». Cette option est **désactivée par défaut** : sur un dépôt avec un gros `.git`, la copie est lente, et « un garde-fou qui devient lent sans qu'on le lui demande est un garde-fou qu'on désinstalle ». Elle est bornée (256 Mo et 25 000 fichiers par défaut), refuse le

répertoire personnel et la racine, et ne remplace jamais une capture ciblée ni ne couvre une surface distante ou un fichier hors du projet.

3.8 Choix de conception et alternatives écartées

Le tableau 3.5 rassemble les décisions d'architecture, l'alternative écartée et la raison. Les justifications détaillées se trouvent dans les sections citées.

Table 3.5. Choix de conception, alternatives et justifications

Choix	Alternative écartée	Pourquoi
Analyse syntaxique + simulation	Expressions régulières	Le shell est une grammaire ; les motifs se trompent dans les deux sens. L'analyse unique a corrigé 22 cas sans en casser aucun (§3.4.1).
Une seule analyse lue par tous	Un analyseur par besoin	Deux analyseurs qui répondent différemment à la même question étaient la source principale des défauts de l'audit.
Logique déterministe sur la commande	Un LLM qui juge la commande	L'outil accompagne des agents fondés sur des LLM ; il doit donner la même réponse à la même ligne, être testable exemple par exemple, tenir en quelques millisecondes et ne pas pouvoir être convaincu par le texte qu'il analyse.
Règles écrites comme des données, exemples = tests	Code ad hoc par outil	Une règle se lit, se relit et se vérifie ; ses 774 exemples sont exécutés à chaque test ; le préfiltre du shell en est généré.
Grammaires tirées des outils eux-mêmes	Grammaires écrites de mémoire	Une option mal connue est une décision fausse (`unzip -Pxo`, `helm 4`, `tofu -chdir`) ; les règles non vérifiées sont marquées (§3.4.7).
Récupérer d'abord	Bloquer tout ce qui est dangereux	Héritage de la bêta : bloquer arrête l'agent et pousse à affaiblir les règles ; une copie ciblée laisse le travail continuer et rend l'erreur réversible.
Capture ciblée, tout ou rien	Copie partielle « au mieux »	REVERSIBLE est une promesse ; une copie partielle la rend fausse (§3.4.3).
Points de contrôle optionnels	Copie du projet avant chaque commande	Coût prohibitif sur un gros `.git` ; un outil lent est désinstallé.

Choix	Alternative écartée	Pourquoi
Moteur en ombre, puis décideur	Basculer directement	Les divergences entre prédiction et décision ont été comprises une à une, à décisions constantes, avant que le moteur ne décide (S2+M1 : 124 changements revus).
L'ancien chemin garde le plus strict, puis le moteur lit tout	Supprimer les anciennes règles d'un coup	Aucune décision ne doit devenir plus permissive pendant la migration ; M2.3b a terminé la sortie de l'ancien chemin pour POSIX, validée sur un corpus élargi de 2 306 commandes (§3.4.4).
Un décideur pur et unique	Décisions dans chaque porte	Une correction s'applique partout ; le décideur ne peut pas revendiquer une capture qu'il n'a pas reçue.
Canal ask/deny par surface, deny par défaut	Tout demander, ou tout refuser	L'attention s'use ; une surface nouvelle ne devient jamais « demandable » par accident (§3.4.6).
Filet Linux par ptrace	FUSE	Le garde d'appels système fait déjà ce que ferait un montage FUSE, sans placer le projet derrière un démon permanent (§3.7).
Filet Windows par WinFsp	Pilote de filtre noyau	WinFsp donne un système de fichiers en mode utilisateur, sans pilote noyau à écrire ni à signer ; il voit toutes les routes vers les fichiers.
Tâche planifiée à l'ouverture de session	Service Windows	Une tâche se crée et se supprime en une commande et reste visible dans le Planificateur ; rien de ce que l'outil fait à la machine n'est caché. Elle est durcie (W-SEC).
C3 : analyseur PowerShell maison	Appeler `powershell.exe` pour analyser chaque ligne	Lancer PowerShell à chaque commande coûte des centaines de millisecondes et le garde doit aussi fonctionner depuis WSL ; l'analyseur de PowerShell sert d'*oracle* dans les tests (§4.7).
C3 : tables de paramètres générées	Listes écrites à la main	PowerShell a des centaines de paramètres, alias et abréviations ; une nouvelle version devient une régénération, pas une modification de code.

Conclusion du chapitre

L'architecture tient en une phrase : *une seule lecture, une seule politique, un seul décideur, et des filets là où la lecture s'arrête.* Les principes qui la gouvernent partagent une même exigence — ne jamais dire plus que ce qui est vrai : pas d'ALLOW pour une ligne non lue, pas de REVERSIBLE pour une capture incomplète, pas de « demander » là où personne ne verra la question. Le chapitre suivant montre comment ces décisions sont devenues du code.

CHAPITRE 4

Réalisation

Ce chapitre montre comment l'architecture du chapitre 3 est devenue du code. Il présente l'environnement, puis chaque couche dans l'ordre où une commande la traverse : les portes, le moteur et ses familles d'outils, la récupération et les reçus, les filets d'exécution, et le côté Windows, dont le second frontal du moteur, C3. Il se termine par la partie M2.3, qui a retiré aux anciennes règles textuelles leur pouvoir de décision sur les lignes POSIX. Toutes les sorties de terminal sont réelles : produites au commit `3c94414`, elles ont été rejouées à l'identique, aux identifiants près, au commit de référence `7d2a2aa` ; les chemins sont abrégés.

4.1 Environnement de réalisation

Table 4.1. Environnement et outils

Élément	Choix
Langage	Python $\geq$ 3.9, bibliothèque standard uniquement (plus `tomli` avant 3.11) ; aucune dépendance réseau, aucune télémétrie
Dépendances optionnelles	`winfspy` (montage Windows, avec le pilote WinFsp), `mitmproxy` (couche réseau)
Systèmes	Linux (x86-64) ; Windows avec PowerShell et Git Bash
Agents intégrés	Claude Code (Bash, PowerShell, édition de fichiers), Codex, Cursor ; piège bash pour le terminal
Tests	`pytest` ; les mêmes tests sur Linux et Windows, avec des attentes exactes par système quand le comportement diffère
Gestion de versions	git ; un commit par partie, dont le message est le bilan
Distribution	paquet Python (`pyproject.toml`), scripts `install.sh` et `install.ps1`, licence MIT

Pourquoi la bibliothèque standard seulement ?

`demo_cli` s'exécute à chaque commande de l'agent et se place sur le chemin de toutes ses actions ; chaque dépendance est du code de plus qui tourne avec les droits du développeur et une source de panne au démarrage. Le cœur n'en a aucune. Même `winfspy`, réellement nécessaire au montage Windows, est une dépendance *optionnelle* : son installation peut échouer sans WinFsp, et cet échec ne doit pas emporter le reste de l'outil (`pyproject.toml`).

4.1.1 Organisation du code

arborescence (extrait)

```
src/demo_cli/
  cli.py              # commandes : check, undo, log, verify, doctor, sh, mount...
  guard.py            # l'orchestrateur : Guard.evaluate, evaluate_file_edit
  decide.py           # le décideur unique (fonction pure)
  policy.py           # la politique : surfaces, canaux ask/deny, indices
  model.py            # l'analyse POSIX unique
  classify.py         # l'ancien chemin (règles textuelles), en retrait
  recovery.py         # captures, groupes, restauration
  checkpoint.py       # points de contrôle (optionnels)
  receipts.py         # registre chaîné SHA-256
  syscall_guard.py    # filet Linux (ptrace)
  fsguard.py          # filet Windows : le jugement
  fsmount.py          # filet Windows : WinFsp
  protect.py          # déplacement et verrouillage du projet (Windows)
  schedule.py         # tâche d'ouverture de session (W-SEC)
  hooks/              # claude_code.py, codex.py, cursor.py
  engine/             # simulate, words, invocation, effects, verdict,
                      # rules*.py (12 familles), *_ops.py, *_data.py
tests/                # 133 fichiers ; fixtures/ : lignes du portail par partie
```

4.2 Les portes

4.2.1 Le *hook* Claude Code

Le *hook* (`hooks/claude_code.py`) reçoit l'appel d'outil sur son entrée standard, appelle `Guard.evaluate` (outils `Bash` et `PowerShell`) ou `Guard.evaluate_file_edit` (`Edit`, `Write`, `MultiEdit`, `NotebookEdit`), et traduit la décision (tableau 4.2).

Les quatre sorties suivantes sont celles du *hook* lui-même, à qui l'on a donné le JSON qu'envoie Claude Code (message lisible sur la sortie d'erreur, JSON sur la sortie standard ; liens de signalement coupés, émojis simplifiés) :

Table 4.2. Traduction des décisions par le *hook* Claude Code

Décision	Réponse du *hook*
ALLOW, REVERSIBLE, DRY_RUN	Silence sur la permission (l'invite normale de l'utilisateur reste) ; la capture est annoncée au modèle (`additionalContext`) et à la personne (`systemMessage`)
SUPERVISED	`updatedInput` : la commande est réécrite en `demo_cli sh -g <id> <commande>`, *sans* `allow` ; ce qui s'exécute réellement est dit aux deux
CONTEXT_MISMATCH	`ask`
ESCALATE approuvable	`ask` dans les modes qui affichent la question (`default`, `acceptEdits`, `plan`, `auto`) ; `deny` sinon
ESCALATE non approuvable	`deny`

hook Claude Code : capture, refus, question, filet

```
$ echo '{..."tool_name":"Bash","tool_input":{"command":"rm -rf src"}}' | demo_cli
    ↪ hook
demo_cli recovery point d9684862 captured before this ran (1 files).
         mistake? undo it with:  demo_cli undo d9684862
{"hookSpecificOutput": {"hookEventName": "PreToolUse", "additionalContext":
 "[demo_cli] Recovery point captured before the action; reversible. ..."}, ...}

$ echo '{... "command":"terraform destroy -auto-approve"}' | demo_cli hook
demo_cli blocked: Mutation of a non-recoverable surface (infra_destroy); ...
         nothing was captured, and nothing is claimed to be.
{"hookSpecificOutput": {..., "permissionDecision": "deny", ...}}

$ echo '{... "command":"docker volume rm donnees"}' | demo_cli hook
demo_cli needs your approval: Mutation of a non-recoverable surface (
    ↪ container_runtime)
         nothing was captured: if you approve, demo_cli cannot undo it.
{"hookSpecificOutput": {..., "permissionDecision": "ask", "permissionDecisionReason
    ↪ ":
 "... If it has to come back: rebuild from the Dockerfile / compose file, or a
 volume backup if one exists."}}

$ echo '{... "command":"python3 nettoyage.py"}' | demo_cli hook
{"hookSpecificOutput": {"hookEventName": "PreToolUse", "updatedInput": {"command":
 "/usr/bin/python3 -m demo_cli sh -g 8553dbf8 'python3 nettoyage.py'"}, ...}}
```

> **Pourquoi le *hook* se tait-il au lieu de dire « allow » ?**
>
> Répondre `allow` court-circuite l'invite de permission de l'utilisateur : ses propres règles cesseraient de s'appliquer dès que `demo_cli` est installé. Un garde-fou ne doit jamais *réduire* la sécurité que l'utilisateur avait déjà (partie B1). C'est aussi pourquoi la réécriture vers le filet se fait sans `allow` (partie B1b) : l'utilisateur voit la vraie commande qui s'exécutera.

4.2.2 La porte fichier

Les outils d'édition ne passent pas par le shell. `evaluate_file_edit` copie le fichier *existant* avant qu'il ne soit réécrit ; une création n'a rien à copier ; un fichier du plan de contrôle est refusé ; un fichier hors du projet, ou un lien qui en sort, n'est pas copié dans la réserve du projet. Si la copie échoue, la modification est escaladée.

4.2.3 Codex et Cursor

L'adaptateur Codex (`hooks/codex.py`) ne parle que `deny`, seule réponse que l'exécution de Codex accepte (section 2.1) : un CONTEXT_MISMATCH y est capturé et enregistré honnêtement, mais ne peut pas être « demandé ». Il analyse aussi la grammaire des *patches* de l'outil `apply_patch`, qui nomment explicitement chaque fichier cible. L'adaptateur Cursor installe son *hook* avec `failClosed` pour qu'une panne bloque au lieu de laisser passer.

4.2.4 Le garde du shell

Les commandes tapées dans un terminal (ou le mode `!` de Claude Code, qui lance `bash -c`) n'ont pas de *hook*. `demo_cli install-shell-guard` active un piège bash : `BASH_ENV` pour les shells non interactifs, un `trap ... DEBUG` avec `extdebug` qui évalue chaque commande avant son exécution. Pour ne pas payer l'évaluation sur chaque `ls`, un *préfiltre* ne transmet au moteur que les mots qui peuvent avoir un effet ; cette liste est générée à partir de la table de règles (partie M2.2e) — un tableau associatif sous bash 4 et plus, une liste de mots sous bash 3. Une escalade approuvable pose la question sur `/dev/tty` dans un shell interactif uniquement ; partout ailleurs, la commande est bloquée.

4.3 Le moteur et ses familles d'outils

4.3.1 Des parties courtes, chacune avec ses tests

Le moteur a été construit en parties successives, d'abord en ombre (il prédisait sans décider), puis comme source de la décision. Le tableau 4.3 les résume.

Table 4.3. Les parties du moteur et ce que chacune a apporté

Partie	Apport
Modèle	Une seule analyse POSIX, partagée par le classifieur et la résolution des cibles.
B	Moteur en ombre : `rm`, `rmdir`, `unlink` et les redirections ; prédiction écrite dans chaque reçu ; 91 tests ; portail : 0 décision changée.
C1	Outils de fichiers locaux : `cp`, `mv`, `sed -i`, `perl -i`, formateurs en place, `tee`, `ln`, `install`, `truncate`, `shred`, `dd`, `rsync`, `tar`, `unzip`, `find`…; 178 tests ; 0 décision changée.
C2a	Clients SQL (`sqlite3`, `psql`, `mysql`) : SQL lu depuis `-c`, l'entrée standard, les *heredocs* et les fichiers, chaque instruction classée.
S2+M1	Le moteur décide sous Linux : exactement l'ensemble détruit est capturé en un groupe ; 124 décisions revues sur le portail (8 erreurs corrigées, 18 nouvelles captures, 86 nouvelles autorisations : rien n'était détruit).
M2.0–M2.1	Une seule source de vérité ; le moteur sous Windows pour les lignes POSIX (Git Bash).
I1a–c, W0	Lectures sûres (FIFO, `/dev/zero`), sortie des `$( )` en lecture seule, mot de passe collé d'`unzip -P`, citations ANSI-C décodées comme bash 5.
M2.2a–f	Les familles d'outils (tableau E.2).
M2.3a, M2.3b	Les lignes non lues quittent les anciennes règles ; l'ancien chemin ne décide plus aucune ligne POSIX (section 4.8).

4.3.2 Les familles d'outils

La partie M2.2 a fait entrer dans le moteur, famille par famille, les outils que l'ancien chemin reconnaissait par motifs. Chaque famille apporte une grammaire vérifiée, ses effets locaux et distants, ses surfaces, et des arbitrages numérotés.

Table 4.4. Les familles d'outils du moteur (partie M2.2)

Famille	Outils	Décisions marquantes
git	`git` (options globales avant la sous-commande)	Perte dans l'arbre de travail capturée ; *push* qui remplace ou supprime une référence du serveur = surface distante ; un déplacement d'historique que le *reflog* ramène est autorisé et nommé (arbitrage c) ; ce qui détruit le *reflog* escalade.
Infrastructure	`terraform`, `tofu`, `kubectl`, `helm`, `pulumi`, `cdk`	Les *apply* escaladent (R1) ; *destroy* escalade ; mise à zéro, *drain*, *cordon* sur une surface « disponibilité » ; grammaires vérifiées sur les versions courantes (terraform 1.16.5, tofu 1.13.1, kubectl 1.37.1, helm 4.3.0 et 3.16.2, pulumi 3.268.0, cdk 2.1144.0).

Famille	Outils	Décisions marquantes
Cloud	`aws`, `s3cmd`, `rclone`, `mc`, `gcloud`, `gsutil`, `az`, `azcopy`, `railway`	Grammaires tirées des données des outils (index des commandes d'aws v2, arbre de gcloud, table d'az) ; lectures autorisées (R3) ; un envoi qui peut remplacer un objet escalade (R2), sauf option de non-écrasement vérifiée.
Conteneurs	`docker`, `podman`, `docker-compose`	Supprimer conteneur, image, volume ou réseau escalade (`container_runtime`) ; `compose down` comme `docker rm` (R5) ; `push` peut remplacer une étiquette.
Entrepôts de données	`redis-cli`, `mongosh`, `dropdb`, `pg_restore`, `pg_dump`, `mysqladmin`, `sqlcmd`	Vidages sur `datastore_flush` (deny) ; écritures serveur sur `remote_database` ; fichiers locaux écrasés capturés.
Envois sortants	`curl`, `wget`, `http`, `gh`, `mail`, `ssh`, `scp`, publication `npm`/`cargo`/`twine`…	Une requête autre que GET/HEAD/OPTIONS change quelque chose (R6) ; une publication escalade (R7) ; `ssh hôte cmd` exécute du code (R8) ; un courriel envoyé (R9).
PaaS	`heroku`, `fly`, `vercel`, `netlify`, `wrangler`, `firebase`	Détruire une application ou une base : `paas_destroy` (deny) ; déploiements et variables : `saas_deploy` ; secrets : `secret_write`.
SaaS, migrations	`shopify`, `wp`, `stripe`, `rails`, `manage.py`, `alembic`, `prisma`, `flyway`…	Argent déplacé : `external_payment` (deny) ; *rollback*/*reset*/*drop* d'une migration supprime ; *status* et modes qui impriment le SQL : lecture.
Système	85 outils : `systemctl`, `kill`, `reboot`, comptes, `mkfs`, `diskpart`, `vssadmin`, `iptables`, `netsh`, `chmod`, `icacls`, `reg`…	Arbitrages C1–C10 ; données de récupération du système (`system_recovery`) : deny ; `chmod` dans le projet capturé.
Plus tard	fichiers de démarrage du shell, `.envrc`, *hooks* git, `crontab`, `at`, `schtasks`, `systemctl enable`, clés `Run`	Effet `LATER` : du code planté pour s'exécuter sans que personne ne le nomme ; ask (D1) ; retirer une planification = DELETE (D4).
Secrets	`vault`	`secret_write` ; jeton révoqué = `credential_rotation`.

4.3.3 Quelques détails qui comptent

— **Un tiret n'est pas un nom de fichier** (I1a). `X=-rf; rm $X src` était autorisé : le moteur lisait `-rf` comme un fichier. Une valeur ou un résultat de *glob* qui commence par un tiret est devenu non compris ; depuis M2.3b, une valeur *connue sur la ligne* est lue comme l'option qu'elle est, et la ligne est capturée.

— **Lire un fichier sans se bloquer** (I1a). Un FIFO lu par `sqlite3 .read` bloquait le moteur au-delà de son budget, et un lien vers `/dev/zero` se lisait sans fin. Le moteur ne lit plus que des fichiers réguliers, en mode non bloquant, plafonnés.

— **Les citations ANSI-C** (W0). Un mot de commande écrit avec des échappements hexadécimaux (`$'\x72\x6d'`, soit `rm`) était lu comme un outil inconnu et autorisé. Le décodage suit exactement bash 5 ; ce qu'il ne décode pas exactement (`\u`, `\c`, NUL) rend le mot non compris.

— **Une base SQLite est un fichier** (C2a). Elle est capturée comme un fichier du projet ; une base PostgreSQL ou MySQL est étiquetée `db:hôte/nom`, sans secret, et n'est capturable que si une URL locale littérale la désigne ; sinon, l'écriture escalade.

4.4 La récupération et les reçus

4.4.1 Captures ciblées et groupes

`recovery.py` copie la cible réelle avant l'exécution : un fichier, un répertoire (avec ses sous-répertoires ignorés comme `.git` quand le parent est détruit), une base SQLite. Depuis la partie S2, toutes les captures d'une commande partagent un identifiant de *groupe* : un seul `undo` restaure tout, et `snapshot_set` est *tout ou rien* — si une capture échoue, les précédentes sont invalidées et marquées comme telles dans l'index. Les captures sont bornées (256 Mo et 25 000 fichiers par défaut, réglables) ; au-delà, la capture est refusée et la commande escaladée, jamais réduite.

La restauration a ses propres pièges, découverts en les provoquant : un fichier rendu en lecture seule par l'agent (`chmod 444` ou `000`) empêchait `undo` de le réécrire (corrigé dans M2.2c) ; l'état écrasé par une restauration est lui-même sauvegardé, pour qu'un `undo` malheureux se défasse à son tour ; et un lien symbolique supprimé revenait comme un fichier ordinaire, parce que la restauration utilisait l'entrée de groupe qui enveloppe une cible unique au lieu de l'entrée interne qui garde le lien (V22, corrigée : la restauration rejoue les entrées internes, vérifié sous Linux et Windows).

undo d'un groupe capturé par le filet

```
$ demo_cli log
  id        when             kind     size    action
  8553dbf8  20261009-141447  group    30B     openat_trunc app.py (2 files)
  d9684862  20261009-141441  group    12B     rm -rf src (1 files)
$ demo_cli undo 8553dbf8
demo_cli 1.7.0  RESTORED
  target              ~/projet/src/app.py
  restored 2 files/databases as one undo
  target              ~/projet/notes.txt
  restored 2 files/databases as one undo
  the previous content was kept as edd72faa - put it back with:  demo_cli undo
    ↪ edd72faa
```

4.4.2 L'aperçu SQL

Pour une requête `DELETE`, `UPDATE` ou `TRUNCATE` sur une base locale, `preview.py` compte en lecture seule les lignes qui seraient touchées (pilote standard pour SQLite, session en lecture seule pour PostgreSQL). La décision devient DRY_RUN : capturée, réversible, avec le nombre de lignes affichées.

4.4.3 Le registre de reçus

Chaque décision — y compris un ALLOW, et chaque capture faite par un filet — laisse un reçu dans `.demo_cli/receipts.jsonl`. Un reçu contient la commande, la décision et sa raison, la règle, la surface, le point de récupération, le contexte, l'intention déclarée, la prédiction du moteur, le groupe, le fait qu'une question *pouvait* être posée (`approvable` — jamais la réponse, qu'aucune porte ne voit) et l'invariant. Son empreinte est calculée sur son contenu canonique *et* l'empreinte du reçu précédent :

Le chaînage (`receipts.py`, `Receipt.finalize`)

```
body = asdict(self)
body.pop("receipt_hash")
self.receipt_hash = hashlib.sha256(
    (_canon(body) + self.prev_receipt_hash).encode()).hexdigest()
```

Modifier, insérer, supprimer ou réordonner un reçu brise la chaîne. L'écriture est protégée par un verrou multiplateforme (`fcntl` sous Linux, `msvcrt` sous Windows). Le filet Windows

écrit dans une seconde chaîne, liée à la première, pour éviter qu'un même fichier soit écrit à travers le montage et directement. Les secrets (mots de passe d'URL, clés) sont masqués. Un reçu réel complet figure en annexe A.

vérification, puis après modification d'un seul champ

```
$ demo_cli verify
demo_cli 1.7.0  VERIFIED
Receipt chains
  main         6 entries    INTACT
Recovery artifacts
  artifacts    4 verified on disk
$ # on change "decision" d'un reçu en ALLOW, puis :
$ demo_cli verify
demo_cli 1.7.0  TAMPERED
Receipt chains
  main         TAMPERED
             broken at line 2
             A field in this entry was edited after it was written.
```

4.5 Les filets d'exécution

4.5.1 Linux : le garde d'appels système

`syscall_guard.py` lance la commande sous ptrace (`fork`, `PTRACE_TRACEME`, puis `exec`) et suit aussi ses processus enfants (`fork`, `vfork`, `clone`, `exec`). À chaque arrêt sur appel système, il ne réagit qu'à l'*entrée* de l'appel (registre `rax` égal à `-ENOSYS`) et lit le numéro de l'appel et ses arguments dans les registres x86-64 ; les chemins sont lus dans la mémoire du processus via `/proc/<pid>/mem` et résolus par rapport au bon répertoire (`AT_FDCWD` ou descripteur).

Table 4.5. Appels système surveillés par le garde Linux

Appel	Pourquoi il détruit
`unlink`, `unlinkat`, `rmdir`	suppression
`rename`, `renameat`, `renameat2`	la destination existante est remplacée
`creat`, `truncate`	le contenu est vidé
`open`, `openat` avec `O_TRUNC`	le contenu est vidé à l'ouverture
`open`, `openat` en écriture sans `O_APPEND`	réécriture en place

Pour un fichier préexistant du projet, la copie est prise, puis l'appel continue. Les fichiers

que la commande a elle-même créés ne sont pas copiés. Pour un fichier du plan de contrôle, l'appel est annulé (le numéro d'appel est remplacé par -1) et un reçu ESCALATE est écrit. Si le processus est déjà suivi par un traceur étranger (`strace`, `gdb`), le garde refuse de s'exécuter sans surveillance (code de sortie 126). À la fin de la commande, les processus d'arrière-plan sont détachés et continuent de tourner.

Voici le filet en action, sur le script réécrit par le *hook* à la section 4.2 ; le script supprime `notes.txt` et réécrit `src/app.py` :

```
• • •  le filet capture au moment de la destruction

$ demo_cli sh -g 8553dbf8 "python3 nettoyage.py"
demo_cli [syscall] REVERSIBLE: unlink ~/projet/notes.txt  (undo: demo_cli undo 8553
    ↪ dbf8)
demo_cli [syscall] REVERSIBLE: openat_trunc ~/projet/src/app.py  (undo: demo_cli
    ↪ undo 8553dbf8)
nettoyage terminé
```

Les deux fichiers ont été rendus par `demo_cli undo 8553dbf8` (section 4.4). `demo_cli` n'a pas lu une ligne du script.

4.5.2 Windows : le montage WinFsp

Sous Windows, le filet est un système de fichiers. La mise en place (`demo_cli setup`, `protect.py`) :

1. déplace le projet `C:\projet` vers `C:\projet.real` (un point de montage WinFsp ne peut être créé que sur un répertoire vide ou absent ; un déplacement garde les chemins connus des outils et de l'IDE) ;
2. verrouille `projet.real` par une ACL réservée aux Administrateurs et à SYSTEM ;
3. monte à `C:\projet` un système de fichiers « passe-plat » vers `projet.real`, exécuté avec élévation ;
4. enregistre une tâche d'ouverture de session pour le remonter après un redémarrage.

La figure 4.1 montre ces étapes sur un projet réel, puis leur exact inverse : `demo_cli teardown` arrête le montage, retire la tâche, remet les fichiers à leur place et retire les *hooks*, en laissant la configuration et les reçus comme trace d'audit.

Le montage a été construit en deux étapes qui prennent *les mêmes décisions* : une première version en mémoire, pour prouver l'interception, la capture et la restauration sans risquer de corrompre de vrais fichiers ; puis la version passe-plat, sur laquelle on peut travailler. Le passage de l'une à l'autre n'a demandé aucune modification du jugement (`fsguard.py`) et une seule ligne

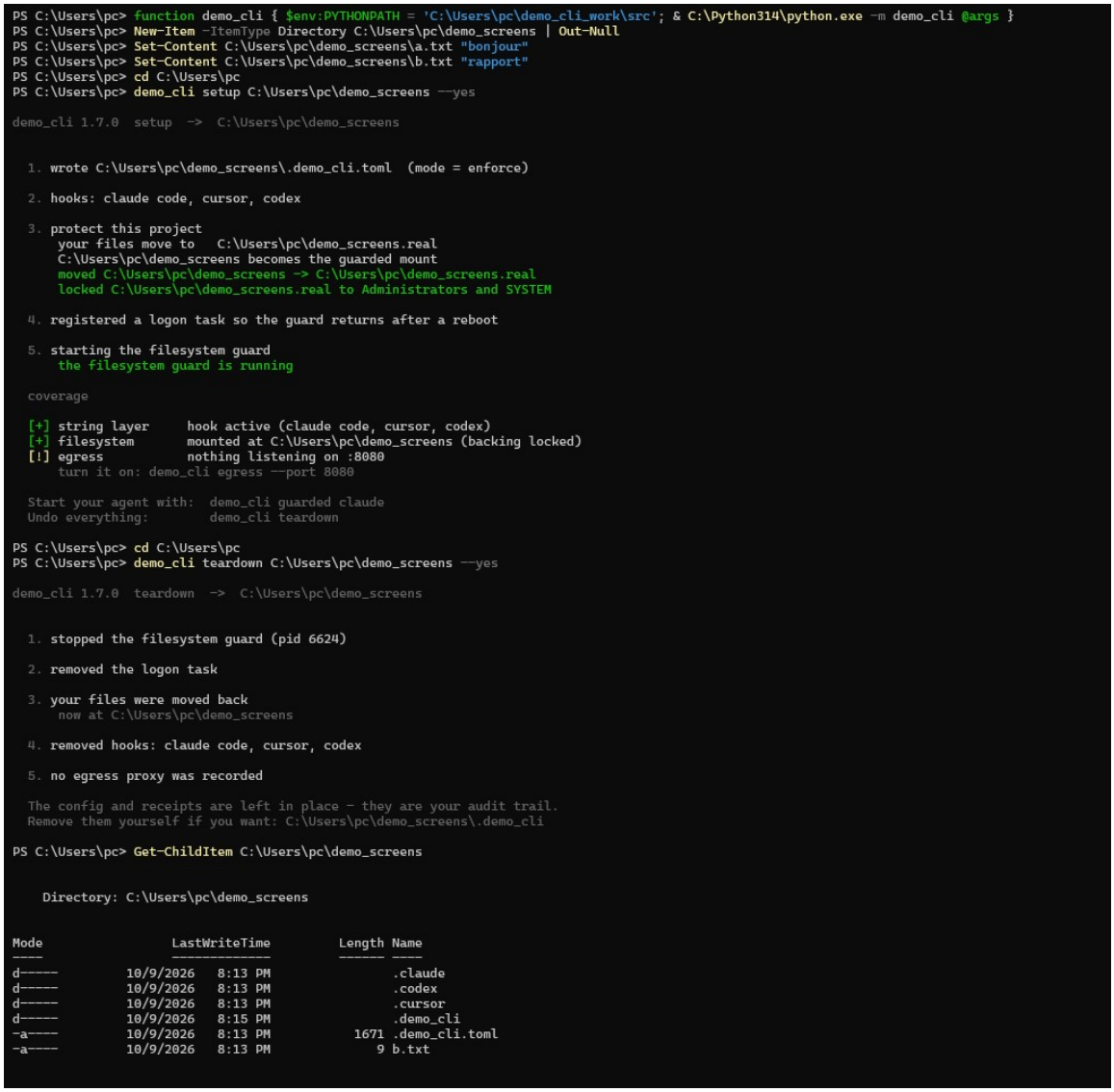


Figure 4.1. Sous Windows : `demo_cli setup` protège un projet (déplacement, verrouillage, tâche, montage), puis `demo_cli teardown` défait chaque étape

de la capture : c'est le bénéfice d'avoir séparé le jugement de la plomberie. Les captures sont écrites sur le disque réel, hors du montage.

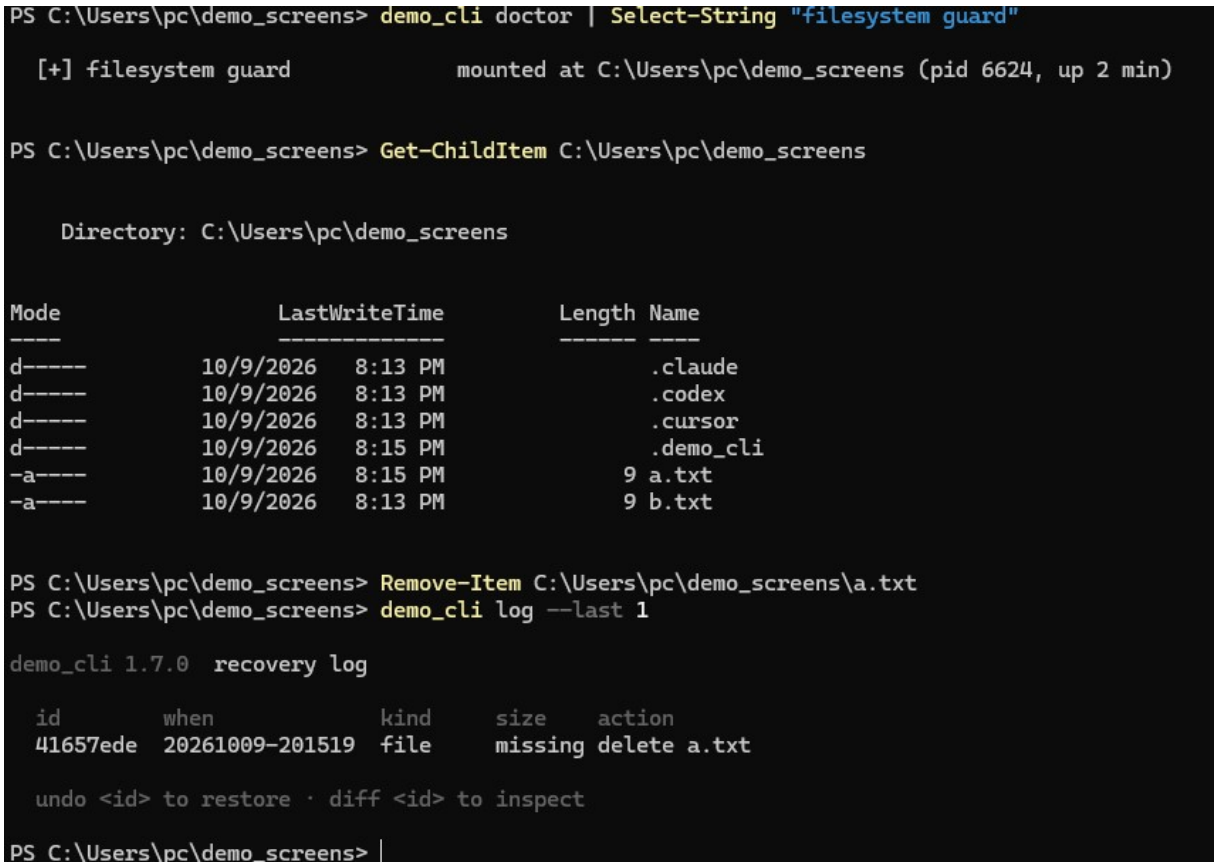


Figure 4.2. Le montage actif (`doctor`) : un `Remove-Item` tapé dans le projet est capturé par le montage, qui ne lit aucune commande (entrée « delete a.txt » du journal de récupération)

4.5.3 Ce que les deux filets ne voient pas

Les deux filets sauvegardent le projet, et seulement lui. Une destruction hors du projet par du code non lu n'est pas sauvegardée ; le filet Linux l'*enregistre* : un reçu par fichier, sans sauvegarde ni blocage (V18, deuxième étape, `d60be70`). Les répertoires temporaires, `/proc`, `/sys`, `/dev`, les fichiers non réguliers et ceux que la commande a créés elle-même sont ignorés, et une exécution est plafonnée à 100 reçus. La raison de la décision le dit : « outside the project each destroyed file is recorded, not backed up ». Le montage Windows ne voit que le projet.

```
• • •  un script détruit un fichier dans le projet et un autre hors du projet

$ demo_cli sh -g bbbb2222 "python3 nettoie.py"
demo_cli [syscall] REVERSIBLE: unlink ~/projet/notes.txt  (undo: demo_cli undo
    ↪ bbbb2222)
demo_cli [syscall] RECORDED (outside the project, not backed up): unlink ~/ailleurs/
    ↪ config.txt
fini
```

Le registre contient alors deux reçus : REVERSIBLE (`sys_unlink`) avec un point de récupération pour le fichier du projet, et ALLOW (`sys_outside_unlink`) sans point de récupération pour le fichier extérieur — « recorded, not backed up, cannot be undone ».

Les surfaces distantes leur sont invisibles : c'est pourquoi une ligne non lue qui atteint le cloud, un conteneur ou un service escalade au lieu d'aller au filet.

4.6 Le côté Windows

4.6.1 Parité entre les deux systèmes

Le tableau 4.6 récapitule, couche par couche, ce qui est identique et ce qui diffère.

Table 4.6. Parité Linux / Windows au commit de référence

Couche	Linux	Windows
Portes	*hook* Claude Code (Bash, fichiers), Codex, Cursor, piège bash	*hook* Claude Code (Bash/Git Bash, PowerShell, fichiers), Codex (PowerShell)
Lecture POSIX	moteur	même moteur ; chemins `/c/x` traduits en `C:\x`
Lecture PowerShell	—	C3, même moteur ; les anciennes règles ne gardent que leurs captures et escalades
Politique, décideur, reçus, undo	communs	communs
Filet d'exécution	garde d'appels système (ptrace), activé par réécriture	montage WinFsp : le projet *est* le montage, aucune réécriture
Plan de contrôle pendant l'exécution	appel annulé par le garde	refusé par les portes (le montage lui-même : V7, ouvert)
Persistance	—	tâche d'ouverture de session durcie (W-SEC)

4.6.2 Une leçon de la partie T : juger ce qui s'exécute

Pour faire passer les lignes POSIX de Windows par le moteur, une première version (M2) *réécrivait le texte* de la commande, en traduisant ses chemins, avant de la juger. La partie T l'a retirée : bash exécutait alors une autre commande que celle qui avait été jugée, et les reçus enregistraient un texte que l'agent n'avait jamais écrit. La traduction se fait désormais à l'intérieur du moteur, sur les chemins, et la commande exécutée reste celle de l'agent. Le principe est général : *on juge exactement ce qui s'exécute, et on enregistre exactement ce qui a été jugé.*

4.6.3 PowerShell avant C3

Sous Windows, les agents écrivent du PowerShell. Jusqu'à C3, le moteur ne le lit pas : une ligne PowerShell, ou une ligne POSIX qui contient un mot PowerShell ou `cmd.exe`, est jugée par une poignée d'expressions régulières (`ps_remove_item`, `ps_set_content`…; arbitrage M5). Le coût était mesurable (tableau 4.7) : des destructions passaient, une écriture banale escaladait, et seul le montage WinFsp couvrait ces cas. C3 les lit (colonne « Avec C3 »).

Table 4.7. Lignes PowerShell : l'ancien chemin et C3 (le projet contient a.txt et b.txt)

Ligne	Ancien chemin	Avec C3
`iex "Remove-Item a.txt"`	ALLOW (le motif voit une chaîne)	REVERSIBLE : la chaîne littérale est analysée, a.txt est copié avant
`Write-Output "$(Remove-Item a.txt)"`	ALLOW (suppression cachée)	REVERSIBLE : la sous-expression est analysée
`Copy-Item a.txt b.txt`	ALLOW (écrase sans `-Force`)	REVERSIBLE : b.txt, la destination écrasée, est copié avant
`Set-Content a.txt hi`	ESCALATE (inutile)	REVERSIBLE : a.txt lié à `-Path` (position 0), hi à `-Value`
`"a.txt" | Remove-Item`	ALLOW (la cible arrive par le tube)	REVERSIBLE : la valeur se lie au paramètre de *pipeline*

4.7 C3 : le second frontal du moteur

4.7.1 L'idée : un frontal, pas un second garde-fou

Une expression régulière lit des caractères ; PowerShell *lie des paramètres*. Seul un analyseur qui les lie comme PowerShell le fait peut dire ce qu'une ligne touchera. C3 n'est pas un second garde-fou à côté du premier : c'est un *second frontal* du même moteur. La ligne PowerShell est analysée vers le même modèle d'invocations, simulée sur la même vue virtuelle du système

de fichiers, et produit les mêmes effets, qui passent par les mêmes tables `verdict` et `policy` et le même décideur (figure 4.3). Une politique et un ensemble de garanties — « jamais un REVERSIBLE non mérité », « l'inconnu n'est pas sûr » — valent pour les deux dialectes.

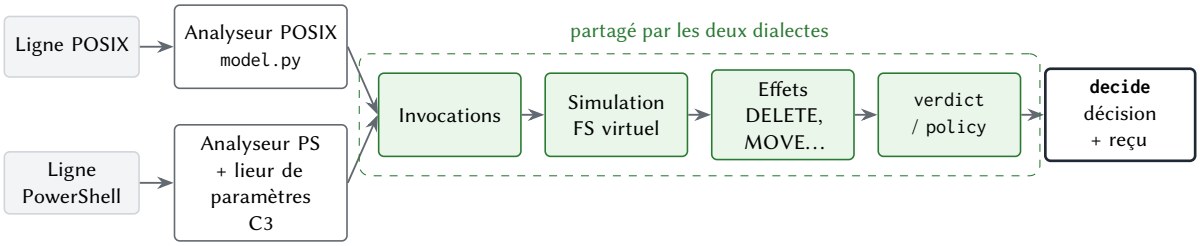


Figure 4.3. C3 : deux frontaux, un seul moteur

4.7.2 Organisation du code

C3 ajoute un frontal et un exécuteur PowerShell sur le `World` existant de la simulation ; il n'ajoute ni décideur, ni politique, ni second vocabulaire d'effets.

Table 4.8. Les modules de C3

Module	Rôle
`ps_model.py`	Analyseur lexical et syntaxique : script, instructions, *pipelines*, commandes (nom, paramètres, arguments, redirections, positions dans la source), chaînes littérales et extensibles, *here-strings*, blocs de script, sous-expressions, affectations. Tout ce qui sort de la grammaire est marqué non compris, avec sa raison.
`engine/ps51_bindings.json`, `engine/ps_tables.py`	Les tables générées (PS2) : cmdlets, alias, jeux de paramètres, positions, commutateurs, entrée de *pipeline*, alias de paramètres, paramètres communs ; chargées à la demande.
`engine/ps_bind.py`	Le lieur : alias → cmdlet, `-Nom` et `-Nom:valeur`, plus court préfixe non ambigu, liaison positionnelle par jeu de paramètres, liaison de *pipeline* par valeur puis par nom de propriété.
`engine/ps_invocation.py`	L'invocation PowerShell, pendant de l'invocation POSIX : texte, profondeur, séparateur, sous-expressions exécutées d'abord, charge (`iex`, `-Command`, `-EncodedCommand`), redirections, paramètres liés.
`engine/rules_powershell.py`	Les règles des cmdlets *comme des données*, avec leurs exemples : une famille de plus, chargée à la demande.
`engine/ps_ops.py`	L'exécuteur PowerShell sur le `World` : instructions, *pipelines*, blocs de script là où ils s'exécutent, `iex` littéral, `-EncodedCommand`, exécuteurs `&` et `-File`, redirections. Il appelle les mêmes opérations que les outils POSIX (supprimer, remplacer, écrire, copier, déplacer, créer).

Dans l'orchestrateur, les trois portes qui ne laissaient passer que le POSIX — la prédiction du moteur, le choix du chemin moteur, le routage des lignes non lues — acceptent aussi PowerShell, et une étape de fusion applique la règle « le plus strict gagne » entre la réponse de C3 et celle

des anciennes règles PowerShell. Une ligne POSIX qui contient `powershell -Command ". . ."` passe sa charge au frontal PowerShell.

4.7.3 Fonctionnement

Analyse lexicale et syntaxique

Les règles propres à PowerShell : citations simples littérales et doubles qui s'étendent, échappement et continuation par l'accent grave, *here-strings* `@'. . .'@` et `@". . ."@`, *pipelines*, séparateurs `;`, sous-expressions `$(. . .)`, blocs de script `{. . .}`, opérateur d'appel `&`, redirections (`>` écrase, `»` ajoute, `2>`, `*>`, `> $null`), variables `$env:` et affectations sur la même ligne.

Lieur de paramètres

Le cœur de C3. Pour chaque cmdlet, il connaît les *jeux de paramètres*, les paramètres *positionnels* et leur position, les alias de la cmdlet et de chaque paramètre, les paramètres commutateurs, et ceux qui acceptent une entrée de *pipeline* (par valeur ou par nom de propriété). Il résout les abréviations comme PowerShell : le plus court préfixe non ambigu (`-Rec` → `-Recurse`, `-Dest` → `-Destination`). Un préfixe ambigu, un paramètre inconnu ou un jeu de paramètres qu'aucune liaison ne satisfait rendent la commande non couverte. Ces tables sont *générées* à partir du PowerShell installé (`Get-Command`, métadonnées des paramètres, `Get-Alias`), et chaque fait de liaison a été vérifié sur PowerShell 5.1. Ainsi `Set-Content a.txt hi` lie `a.txt` à `-Path` (position 0) et `hi` à `-Value` (position 1).

Effets par cmdlet

`Remove-Item` et ses alias → DELETE, récursif avec `-Recurse` ; `-Path` développe les jokers, `-LiteralPath` jamais. `Clear-Content` → TRUNCATE. `Set-Content` et `Out-File` → OVERWRITE d'un fichier existant, CREATE sinon ; `-Append`, `Add-Content` et `»` ajoutent seulement (APPEND) ; avec `-NoClobber` sur un fichier existant, la cmdlet échoue sans effet. `Copy-Item` → OVERWRITE de la destination si elle existe ; `Move-Item` et `Rename-Item` → MOVE, et refusent une destination existante sans `-Force`. `New-Item` → CREATE, ou OVERWRITE avec `-Force` sur un fichier existant. Les lectures (`Get-ChildItem`, `Get-Item`, `Get-Content`, `Select-Object`, `Format-*`, `Test-Path`, `Measure-Object`, `Sort-Object`, `Where-Object`…) n'ont pas d'effet. Avec `-WhatIf`, PowerShell ne fait que décrire l'action : aucun effet.

Pipelines

Une valeur littérale envoyée dans une cmdlet se lie à son paramètre de *pipeline* (`"a.txt" | Remove-Item` supprime `a.txt`) ; `Get-ChildItem *.tmp | Remove-Item` supprime ce que la liste renvoie, et la liste est simulée sur le système de fichiers virtuel.

Du code caché dans du code

Une sous-expression est analysée et ses effets comptent ; un bloc de script est analysé là où il s'exécute (`ForEach-Object {. . .}`). `Invoke-Expression` (`iex`) avec une chaîne *littérale*

est analysé comme une ligne à part entière — l'équivalent de `eval` ou `bash -c` —, de même que `powershell -Command "<littéral>"` ; `powershell.exe -EncodedCommand` est décodé (base64, UTF-16LE) puis analysé. Une chaîne construite à l'exécution (`iex $var`) ne peut pas être lue : elle est non comprise.

Rien n'est deviné

Une construction hors de la grammaire modélisée, une variable de valeur inconnue, une cmdlet ou un paramètre inconnus rendent la ligne non comprise ou non couverte, exactement comme en POSIX : filet WinFsp, sinon point de contrôle, sinon escalade. Un script lancé par `-File script.ps1`, `& .\x.ps1` ou `. .\x.ps1` est un exécuteur de code, traité comme `python x.py`.

Des choix plus stricts

Les chemins d'un autre fournisseur que le système de fichiers (`HKLM:`, `HKCU:`, `Env:`, `Variable:`, `Function:`, `Alias:`, `Cert:`, `WSMan:`) sont non compris, jamais pris pour des fichiers ; `-Include`, `-Exclude` et `-Filter` aussi ; une ligne PowerShell qui lance `bash -c` ou `wsl` aussi. `-Confirm` est traité comme une exécution normale, l'outil n'étant pas interactif.

Une ligne `cmd /c . . .` n'est pas du PowerShell : elle garde les règles textuelles (arbitrage PS3). Les anciennes règles PowerShell restent en place, mais ne gardent que leurs captures et leurs escalades, jamais leur ALLOW : la réponse la plus stricte l'emporte. C3 ne lit pas les chaînes construites à l'exécution, les classes, les fonctions (`param()`, `begin/process/end`), `switch`, `try`, la décomposition de paramètres (`@x`), `-%`, les appels de membres dynamiques ni les modules ; les scripts `.ps1` sont des exécuteurs de code. Sous Windows, ces lignes vont au montage WinFsp, sinon à un point de contrôle, sinon elles escaladent ; sous Linux, les mêmes lignes reçoivent les réponses Linux : le moteur est indépendant du système, seul le filet diffère.

4.7.4 Les arbitrages PS1–PS5

- **PS1** — un analyseur PowerShell écrit en Python, dans le processus du *hook* ; l'analyseur de PowerShell lui-même sert d'oracle dans les tests sous Windows.
- **PS2** — les tables du lieur de paramètres sont générées depuis le PowerShell installé (5.1.19041 : 252 cmdlets, 158 alias), pas écrites à la main.
- **PS3** — `cmd /c . . .` garde les règles textuelles.
- **PS4** — `-WhatIf` n'a aucun effet : PowerShell décrit l'action sans la faire.
- **PS5** — une livraison en étapes : (a) analyseur, lieur, cmdlets de fichiers et lectures, branchement au moteur ; (b) *pipelines*, `$( )`, blocs de script, `iex`, `-EncodedCommand`, `&`, `-File` ; (c) retrait des règles textuelles PowerShell, en perspective.

4.7.5 Validation

Chaque règle de cmdlet porte ses exemples, exécutés comme tests ; les lignes PowerShell réelles trouvées dans les registres sont figées avec leur décision attendue ; chaque trou de V12 a un test qui échoue sur le code d'avant C3 et passe après. Sous Windows, l'analyse de C3 est comparée, sur un corpus de lignes réelles, à l'arbre syntaxique produit par l'analyseur de PowerShell lui-même : une seule exécution de `powershell.exe` analyse tout le corpus avec `[System.Management.Automation.Language.Parser]::ParseInput` et donne, pour chaque élément de *pipeline*, le nom de la commande, ses paramètres et ses arguments. L'analyse et la simulation d'une ligne PowerShell tiennent dans le budget de 10 ms du moteur. Le portail de 922 commandes et le portail Windows ont été rejoués : chaque ligne qui bouge est plus stricte ou arbitrée. Les anciennes règles PowerShell restent en place jusqu'à ce que les décisions de C3 les égalent ou les dépassent sur chaque ligne du portail.

4.8 M2.3 : la fin du pouvoir des anciennes règles

À la fin de M2.2, le moteur décidait toutes les lignes POSIX qu'il lisait entièrement. Restaient les autres : les lignes qu'il ne lisait pas tout à fait, encore confiées aux anciennes règles textuelles. Un audit de ces lignes (fixé par `test_m22f.py`) a montré le danger : là où aucune ancienne règle ne se déclenchait, la ligne était autorisée sans rien capturer. La sortie s'est faite en deux parties.

4.8.1 M2.3a : les lignes non lues quittent les anciennes règles

M2.3a applique les arbitrages M1–M5 :

— **M1** — un exécuteur de code va au filet, sinon à un point de contrôle, sinon il est autorisé avec une note qui dit qu'aucune capture n'a été faite ;

— **M3** — une ligne non lue va au filet, sinon à un point de contrôle (jamais pour un opérande hors projet), sinon elle est escaladée ; un `psql -c "$(cat q.sql)"` était autorisé (V19) ;

— une ligne non lue qui agit au-delà des fichiers locaux, ou un téléchargement qui alimente ce qui s'exécute (`bash <(curl . . .)`, `aws s3 cp . . . - | bash`), escalade ;

— **M4** — de nouvelles règles pour des outils qui détruisaient sans être vus : `gzip`, `bzip2`, `xz` (qui suppriment l'original), `sort -o`, `iconv -o`, `patch`, `awk -i inplace`, `7z` (V20), comportements vérifiés sur des fichiers jetables ;

— **V18, première étape** — les filets et le point de contrôle disent qu'ils ne couvrent que le projet ;

— **M5** — le texte PowerShell garde les anciennes règles jusqu'à C3 ;

— partout, **l'ancienne règle garde une capture ou une escalade, jamais son ALLOW**.

Bilan de M2.3a : 17 nouveaux tests ; les lignes figées des parties précédentes qui ont bougé sont toutes devenues plus strictes ; sur le portail, 6 lignes bougent, toutes revues.

4.8.2 M2.3b : l'ancien chemin ne décide plus aucune ligne POSIX

M2.3b termine la sortie. Depuis cette partie, l'ancien chemin à expressions régulières ne sert plus qu'au texte PowerShell et `cmd.exe` (jusqu'à C3) et au cas où le moteur échoue.

— **Le moteur lit ce que l'ancien chemin capturait** : une valeur connue sur la ligne, lue comme l'option qu'elle est (`X=-rf; rm $X src`) ; un script littéral envoyé à un shell par un tube (`echo 'rm -rf build' | sh`) ; `sqlite3 $DB` et `sqlite3 app.db "$(cat q.sql)"` ; les messages de commit ; `npx`, `bunx` et `pnpm dlx` qui lancent un outil connu (`npx prisma migrate reset` est lu comme `prisma`).

— **Un appel distant non lu escalade** : commandes `stripe` ou `wrangler` inconnues, `supabase` et `contentful` hors de leurs lectures vérifiées (tirées de leurs sources, arbitrage Rb3), option inconnue sur un outil distant, `kubectl exec`, `railway run`, `wp eval`, `wrangler d1 execute`, `heroku run`, `gh pr create`. Le code récupéré du réseau puis exécuté garde son refus.

— **Arbitrages Rb1–Rb4** : un mot sans règle n'a pas d'effet (Rb1) ; du code en ligne qui lance d'autres programmes et les installations de paquets escaladent sans filet, supervisés avec (Rb2) ; règles `stripe`, `contentful`, `wrangler`, `supabase` tirées de leurs sources (Rb3) ; une création distante qui transporte du code à exécuter là-bas (`kubectl create ... -`, `helm --set`, `aws --user-data`, scripts de démarrage `gcloud` et `az`) est une *exécution* sur cette surface, soumise à la personne (Rb4).

— **V21 corrigée** : un exécuteur de code garde le filet même si on lui passe un fichier hors du projet.

Bilan de M2.3b : 46 nouveaux tests ; sur le portail, 65 lignes bougent, toutes plus strictes ou arbitrées ; un corpus élargi de 2 306 commandes a validé le changement (section 5.4).

Conclusion du chapitre

Chaque couche a été construite pour pouvoir être vérifiée isolément : le moteur sans rien exécuter, le décideur sans entrée-sortie, le jugement WinFsp sans Windows, la politique dans un seul fichier. C'est cette séparation qui rend possible la validation décrite au chapitre suivant, ainsi que le travail offensif mené contre l'outil lui-même.

CHAPITRE 5

Sécurité, validation et limites

Un garde-fou se juge sur ce qui passe à travers. Ce chapitre rend compte du travail offensif mené contre `demo_cli` pendant le stage — comment les failles ont été cherchées, lesquelles ont été trouvées et corrigées, lesquelles restent ouvertes —, détaille un cas d'étude, le durcissement de la tâche Windows, puis décrit la stratégie de validation, le portail de décision et les résultats. Il se termine par les limites assumées de l'outil.

5.1 Le travail offensif

5.1.1 Trois façons de chercher

Les failles ont été cherchées comme dans un programme de *bug bounty*, avec trois méthodes complémentaires :

La lecture de code
Lire un module en se demandant « que se passe-t-il si cette hypothèse est fausse ? » — par exemple, si le fichier que la tâche élevée exécute est inscriptible par l'utilisateur.

Les sondes
Fabriquer des commandes piégées et les faire juger dans un projet jetable : une substitution qui lit un fichier SQL, un outil qui supprime son entrée sans que son nom l'annonce, une valeur qui commence par un tiret.

La fouille des registres
Analyser les reçus réellement produits par les agents pour voir ce qu'ils exécutent et comment `demo_cli` les a jugés. L'analyse menée pour C3 a porté sur 107 registres et 23 196 reçus, et a montré le poids réel des lignes PowerShell jugées par les anciennes règles.

Chaque faille confirmée suit la même règle que le reste du projet : un test qui *échoue* sur le code vulnérable est écrit avant la correction.

5.1.2 L'audit de reprise

Au début de la période documentée, un audit de la version 1.7.0 a confirmé 18 défauts, plus un contournement du garde du shell par auto-exclusion. Ils ont d'abord été *figés* par des tests

rouges — la suite passait alors avec 1856 tests réussis, aucun ne décrivant ces comportements — puis corrigés en trois tours. Ils se rangent en trois niveaux :

- **l'outil revendique une récupération qu'il n'a pas prise** : une modification de fichier hors du projet déclarée REVERSIBLE, un lien symbolique restauré comme un fichier ordinaire, un faux répertoire `.demo_cli` ignoré par son *nom*, un `cp -f` sur un fichier existant non capturé ;
- **le garde est absent, contournable ou muet** : un préfiltre du shell plus étroit que le classifieur, un appel au garde sans délai maximal, une panne qui ne laisse aucune trace, un interrupteur d'arrêt non relu dans un shell déjà ouvert, une auto-exclusion décidée sur une sous-chaîne au lieu du mot de commande ;
- **du texte jugé comme une commande** : un message de commit ou un corps de *heredoc* pris pour des commandes, une aide (`-help`) prise pour une modification.

5.1.3 La batterie red-team

Les notes de red-team tenues pendant le stage classent chaque constat en trois catégories, qui découlent directement du modèle de menace (section 3.2) :

- **défaut dans le périmètre** (*in-scope flaw*) : l'agent commet une erreur honnête et le garde la manque — c'est un défaut à corriger ;
- **frontière** (*boundary*) : une limite documentée de ce que le *hook* peut voir ;
- **évasion** (*evasion*) : un obscurcissement délibéré, hors du périmètre par conception.

Table 5.1. La batterie red-team #001–#010 et son issue

#	Constat	Catégorie	Issue
001	Les commandes tapées dans le mode ! de Claude Code n'atteignaient pas le *hook* (elles s'exécutent dans le shell de l'utilisateur)	Frontière	Couvertes par le garde du shell (piège `DEBUG` injecté par `BASH_ENV`) ; les commandes tapées dans d'autres shells restent hors du filet Linux (V14)
002	Des commandes obscurcies (Bashfuscator) cachent les mots destructeurs à un classifieur textuel	Évasion	Hors périmètre de la lecture statique ; le filet d'exécution voit les vrais appels système
003	`source fichier.txt` exécute des commandes que le *hook* ne voit pas	Frontière	Traité comme du code non lu : filet, sinon point de contrôle, sinon ALLOW avec une note honnête

#	Constat	Catégorie	Issue
004	**Faux REVERSIBLE** : une suppression avec une expansion non résolue ne capturait que la partie résolue et déclarait toute la ligne réversible	Défaut	Corrigé : une capture partielle est refusée et la ligne escalade
005	Redirections et verbes qui écrasent (`> fichier`, `tee`, `cp` sur un fichier) détruisaient sans déclencher de règle	Défaut	Corrigé : lus par le moteur ; créer un nouveau fichier reste autorisé
006	Un préfixe de variable d'environnement (`X=1 rm app.db`) contournait la règle `rm` ancrée	Défaut	Corrigé
007	Trous de couverture : `mkfs`, `git checkout -- .`, `git restore .`, `git stash clear`, `Remove-Item` sans `-Force`	Défaut	Corrigé
008	Mesure sur 88 incidents réels : couverture de 75 % par enregistrement et 44 % des commandes uniques	Mesure	Portée à 90 % et 81 % après les corrections
009	La destruction validée est rare dans les données réelles ; deux trous : les écritures SaaS qu'aucune copie locale n'annule, et `git stash pop` jugé sûr	Frontière + défaut	Écritures SaaS escaladées comme surfaces non récupérables ; `git stash pop` corrigé
010	Essais en direct du mode `!` : l'enveloppe `eval '<cmd>'` prise pour une exécution distante, une redirection vers un nouveau fichier prise pour une destruction, chaque commande capturée deux fois	Défaut × 3	Corrigés

Au fil des parties, d'autres défauts ont été trouvés et corrigés ; quelques-uns méritent une ligne :

— **Garde Windows éteint en silence** : `Out-File` de PowerShell écrit en UTF-8 avec BOM, la configuration ne se lisait plus et le garde laissait tout passer. Une configuration illisible escalade désormais, avec un message qui dit comment la réparer.

— **Montage WinFsp** : confusion entre suppression de fichier et de répertoire, renommage sur un fichier existant non capturé, descripteurs non fermés, restauration écrite au-delà du montage, verrous de plages d'octets à travers le montage, point de montage périmé après un redémarrage.

— **Intégrité de `undo`** : une suppression récursive restaurée comme un répertoire vide (Linux), une restauration qui échouait sur un parent manquant (Windows), des lignes déchirées dans l'index de récupération en écriture concurrente (corrigé par un verrou inter-processus).

— **Points de contrôle** : un calcul de taille qui ignorait `.git` et permettait de dépasser la limite de 256 Mo ; des copies complètes de bases de plusieurs Go qui figeaient la machine (liste d'exclusion partagée et limites strictes).

— **Installation** : un filtre `Bash|PowerShell` comparé comme une chaîne entière déclenchait le *hook* deux fois par commande ; la désinstallation retirait les *hooks* d'autres outils.

— **Faux positifs** : un mot comme « drop » dans un message de commit déclenchait les règles SQL ; un aperçu de `cd x && rm -rf a b` montrait le mauvais répertoire.

5.1.4 Les vulnérabilités du stage

Le journal des vulnérabilités (V1–V22) distingue ce qui est corrigé, ce qui a été vérifié et n'est pas une faille, et ce qui reste ouvert.

Table 5.2. Vulnérabilités corrigées

Id	Description	Gravité	Trouvée par	Correction
V1	La tâche Windows d'ouverture de session (niveau « le plus élevé ») exécutait un script de `%LOCALAPPDATA%` modifiable par l'utilisateur, donc par l'agent : code exécuté avec élévation, ou garde désactivé par un `exit /b 0`.	Haute	Lecture de code, vérifié sous Windows	W-SEC (`db262df`)
V2	Même sans ce script, le code de la tâche était inscriptible : installation éditable et environnement virtuel utilisateur.	Haute	Inspection	W-SEC
V3	La tâche trouvait `demo_cli` par le `PATH` de l'utilisateur, modifiable.	Haute	Lecture de code	W-SEC
V4	La tâche élevée écrivait son journal dans un dossier inscriptible par l'utilisateur (redirection par un lien).	Moyenne	Revue de conception	W-SEC

Id	Description	Gravité	Trouvée par	Correction
V17	Points de contrôle activés : une ligne illisible avec un opérande *hors* du projet recevait une copie du projet et REVERSIBLE — une promesse non tenue.	Haute (option)	Lecture, reproduit	V17 (`99831a9`)
V18 (1)	Les filets et le point de contrôle couvrent le projet seulement, mais la raison disait « chaque fichier détruit est sauvegardé ».	Moyenne (honnêteté)	Lecture de code	M2.3a
V19	SQL non lu autorisé : `psql -c "$(cat q.sql)"` avec `DROP TABLE users;` → ALLOW, rien capturé.	Haute	Sondes	M2.3a (`3c94414`)
V20	Outils de fichiers destructeurs non vus : `gzip a.txt`, `xz`, `bzip2`, `sort -o`, `iconv -o`, `patch`, `awk -i inplace`, `7z ... -sdel` → ALLOW.	Haute	Sondes	M2.3a
V21	Un exécuteur de code auquel on passait un fichier hors du projet (`python x.py /etc/hostname`) perdait le filet, disponible pourtant pour `python x.py`, et devenait ALLOW. La note affichée restait honnête.	Moyenne	Sondes	M2.3b (`ad7ab16`)
V22	La suppression d'un lien symbolique du projet était capturée, mais `undo` le restaurait comme un *fichier ordinaire* : la restauration utilisait l'entrée de groupe qui enveloppe une cible unique, au lieu de l'entrée interne qui garde le lien. Pas de perte de données.	Moyenne	Revue red-team (R3)	V22 (`7d2a2aa`)
V12	PowerShell non analysé : `iex "Remove-Item a.txt"`, `Write-Output "$(Remove-Item a.txt)"` et `"a.txt" | Remove-Item` autorisés, `Copy-Item a b` écrasait en silence, `Set-Content a.txt hi` escaladait inutilement.	Haute	Fouille des registres	C3

Id	Description	Gravité	Trouvée par	Correction
V18 (2)	Une destruction hors du projet par du code non lu n'était ni sauvegardée ni enregistrée par le filet.	Moyenne	Revue de code (M2.3a)	V18 (`d60be70`) : le filet Linux enregistre chaque fichier détruit hors du projet
—	`source ./x.sh` vu comme sans effet ; code téléchargé exécuté par `bash <(curl . . .)` ou `aws s3 cp . . . - | bash` autorisé.	—	Sondes	M2.3a
—	`undo` impossible sur un fichier passé en `chmod 444/000` ; plantage du moteur sur `git stash pop && git stash drop` ; `~/.bashrc`, qui active le garde du shell, modifiable par l'agent.	—	Sondes	M2.2c, M2.2d

Deux points ont été vérifiés et ne sont *pas* des failles : un utilisateur normal ne peut pas modifier la tâche élevée elle-même (V5, « Access is denied ») ; `Move-Item` et `Rename-Item` refusent d'écraser une destination existante (vérifié sur PowerShell 5.1) — seul `Copy-Item` écrase, ce qui relève de V12.

Un point est un *risque préventif* plutôt qu'une faille active. **V16** (gravité élevée s'il s'était réalisé) : en retirant le pouvoir de décision aux anciennes règles, M2.3 risquait de perdre le refus du point de contrôle pour une commande visant un fichier hors du projet (`/etc`, `~/.ssh`) ; elle aurait reçu une copie du seul projet, présentée à tort comme réversible. M2.3a et M2.3b ont conservé ce refus, épinglé par les tests de V17 et de V21.

Table 5.3. Vulnérabilités et constats ouverts

Id	Description	Plan
V14	Les commandes tapées par la personne et les processus qui survivent à la commande sont hors du filet Linux.	Plus tard
V7–V9	Montage Windows : ne refuse pas lui-même les fichiers de `demo_cli` (les portes le font), lit les fichiers entiers en mémoire, santé réduite à « processus vivant ».	Partie « parité Windows »

Id	Description	Plan
V10–V11	Proxy réseau : repli silencieux si son outil manque ; périmètre d'exfiltration (un `curl -X POST -d @-` est escaladé, d'autres outils non modélisés).	Partie réseau
V15	Codex ne peut pas exécuter une commande réécrite sans l'autoriser : pas de filet sur Codex.	Portage Codex
V6, V13	Reçus : rares lignes déchirées quand Windows et WSL écrivent le même registre ; pas de rotation du registre ni de la réserve.	Maintenance

5.2 Cas d'étude : la tâche Windows qui exécutait du code modifiable

5.2.1 Le contexte

Sous Windows, le filet est un montage WinFsp qui meurt avec la session. Une tâche planifiée le relance à chaque ouverture de session. Le montage doit écrire dans un répertoire verrouillé au profit des Administrateurs : la tâche s'exécute donc au niveau « le plus élevé disponible », une élévation accordée une fois, à l'installation, par une invite UAC que l'utilisateur a acceptée (`schedule.py`).

5.2.2 La faille

Une tâche élevée est une cible de choix : tout ce qu'elle exécute s'exécute avec élévation. Or, dans sa forme d'origine, la tâche lançait `%LOCALAPPDATA%\demo_cli\autostart-<projet>.cmd`, qui appelait `demo_cli` trouvé par le `PATH`. Chaque maillon était modifiable par l'utilisateur — donc par l'agent, qui travaille avec ses droits :

Les conséquences étaient concrètes : du code exécuté avec élévation à la prochaine ouverture de session, ou immédiatement par `schtasks /run`, que l'utilisateur peut lancer sans invite ; ou, plus discrètement, un `exit /b 0` ajouté au script pour que le garde ne revienne jamais.

5.2.3 Trois conceptions écartées

Avant la solution retenue, trois façons d'empêcher l'agent de toucher au garde ont été essayées sous Windows, puis abandonnées :

Table 5.4. W-SEC : chaque maillon de la chaîne d'exécution, avant et après

Maillon	Avant (V1–V4)	Après
Script lancé	`.cmd` dans `%LOCALAPPDATA%`, modifiable (V1)	aucun script : la tâche lance directement l'interpréteur
Code exécuté	installation éditable, environnement virtuel utilisateur (V2)	copie figée du runtime dans `%ProgramData%\demo_cli`, verrouillé
Résolution	`demo_cli` trouvé par le `PATH` de l'utilisateur (V3)	interpréteur *de base*, chemin absolu, option `-I` (pas de `PYTHONPATH`, pas de site utilisateur, pas le répertoire du script)
Journal	écrit par la tâche élevée dans un dossier utilisateur (V4)	`logs\` sous `%ProgramData%`, écrit par la tâche, lisible par l'utilisateur
La tâche elle-même	enregistrée par `schtasks`	enregistrée en XML avec un descripteur de sécurité

1. faire tourner l'agent à un **niveau d'intégrité bas** : l'accès au profil de l'utilisateur, au cache `npm` et à `%TEMP%` était cassé ;
2. lui donner un **jeton restreint** : il ne pouvait plus atteindre la station de fenêtres sans modifier dangereusement des ACL de toute la machine ;
3. le faire tourner sous un **compte local dédié** : identifiants, profils et configuration git/SSH séparés — le flux de travail du développeur était cassé.

La conception retenue renverse le problème : au lieu d'enfermer l'agent, elle verrouille le garde lui-même.

5.2.4 La correction

La règle retenue (arbitrages R1–R4) tient en une phrase : *rien de ce que l'utilisateur peut écrire ne s'exécute avec élévation.*

— Les fichiers de la tâche vivent dans `%ProgramData%\demo_cli`, appartenant aux Administrateurs, verrouillé (SYSTEM et Administrateurs : contrôle total ; Utilisateurs : lecture et exécution). Ils sont écrits pendant l'étape déjà élevée de l'installation. Un dossier qui existe déjà comme lien, ou que l'utilisateur a créé avant, est refusé ou repris avant toute écriture.

— L'action de la tâche est l'interpréteur de base, avec `-I`, sur un `run.py` verrouillé, par chemins absolus. Un interpréteur situé dans le profil de l'utilisateur est refusé : il n'y aurait rien de sûr à exécuter.

— La tâche porte un descripteur de sécurité : les utilisateurs peuvent la lire et la lancer, jamais la modifier.

Le descripteur de sécurité de la tâche (schedule.py)

```
# SYSTEM and Administrators full access;
# Authenticated Users read and execute (run it), never write (change it).
TASK_SDDL = "D:P(A;;FA;;;SY)(A;;FA;;;BA)(A;;GRGX;;;AU)"
```

Lecture du SDDL : `D:P` est une liste d'accès protégée (sans héritage) ; `FA` accorde le contrôle total à SYSTEM (`SY`) et aux Administrateurs (`BA`) ; `GRGX` accorde lecture et exécution aux utilisateurs authentifiés (`AU`).

Enfin, les répertoires de la tâche ont rejoint le plan de contrôle sous Windows, et `demo_cli doctor` signale une ancienne tâche, un répertoire inscriptible, un interpréteur dans le profil de l'utilisateur ou une copie du runtime périmée.

5.2.5 La vérification

Les tests (`test_wsec.py`) vérifient chaque décision du code sur les deux systèmes ; ce qui exige une session Windows élevée (`schtasks /create`, `icacls` sur `ProgramData`) a été vérifié sur Windows, en utilisateur normal : écrire dans le runtime → refusé ; modifier la tâche → « Access is denied » ; lancer la tâche → fonctionne.



Figure 5.1. La tâche durcie : refus d'écriture et de modification pour un utilisateur normal

> **Ce que ce cas enseigne**
>
> La faille n'était dans aucune règle de décision : elle était dans la *plomberie* qui maintient l'outil en vie. Un garde-fou élève forcément quelque chose quelque part ; c'est là que la revue doit regarder en premier, avec une seule question : qui peut écrire ce qui s'exécute ?

5.3 Stratégie de test

5.3.1 Les familles de tests

Les exemples de règles

Chacun des 774 exemples de la table de règles est exécuté par tout le moteur dans une

arborescence fixe, et l'impact obtenu est comparé à l'impact attendu (`test_engine_rules.py`). Un exemple marqué `VERIFIED` décrit un comportement observé sur des fichiers jetables.

Les tests de partie

Chaque partie livre son fichier (`test_m23a.py`, `test_v17.py`, `test_wsec.py`…), écrit pour échouer sur le code d'avant.

Les lignes figées

Pour chaque famille d'outils, les lignes du portail concernées sont figées dans `tests/fixtures/` avec leur décision attendue ; une partie suivante qui les fait bouger doit le justifier.

Les attentes exactes par système

Quand Linux et Windows diffèrent (chemins, filet, dialecte), le test l'écrit explicitement, et la restauration est vérifiée octet par octet.

Les confrontations au réel

Grammaires comparées au `–help` des outils installés ; prédictions SQL comparées à ce que change réellement un serveur PostgreSQL de test ; le garde d'appels système lancé pour de vrai.

Les tests d'inertie

Moteur activé, désactivé, en erreur ou hors budget : les décisions du chemin en ombre ne changent pas (horloge simulée pour forcer le dépassement).

5.3.2 Un test qui échoue sur le code d'avant

Un test qui passe aussi sur le code vulnérable ne prouve rien sur la correction. L'exemple suivant est le test de V19 :

Le test de V19 (`tests/test_m23a.py`)

```
def test_5_unread_sql_escalates_instead_of_being_allowed(tmp_path):
    proj = _project(tmp_path)
    # M2.3b: `sqlite3 app.db "$(cat q.sql)"` is now READ (and captured); a
    # server database behind an unread -c / -e still escalates.
    for cmd in ('psql -c "$(cat q.sql)"', 'mysql -e "$(cat q.sql)"',
                'sqlite3 app.db "$(date)"'):
        r = _eval(proj, cmd)
        assert (r.decision.decision, r.recovery_entry) == ("ESCALATE", None), cmd
```

Le test a évolué avec M2.3b : la ligne SQLite, désormais lue, a été remplacée par une substitution réellement illisible. Pour vérifier la règle, les tests de M2.3a, tels qu'ils ont atterri (`3c94414`), ont été exécutés contre le commit précédent (`99831a9`) : 11 échouent — ceux qui décrivent les corrections — et 6 passent : ce sont les *témoins*, qui vérifient que ce qui était déjà

juste le reste.

```
• • •  les tests de M2.3a sur le code d'avant

$ git archive 99831a9 src | tar -x -C ancien/ && cd ancien
$ python3 -m pytest -q tests/test_m23a.py
FAILED tests/test_m23a.py::test_1_a_runner_goes_to_the_runtime_net
FAILED tests/test_m23a.py::test_5_unread_sql_escalates_instead_of_being_allowed
FAILED tests/test_m23a.py::test_13_file_tools_that_destroy_are_captured
[...]
11 failed, 6 passed in 0.31s
```

La même vérification, refaite sur les deux parties suivantes, donne le même profil :

Table 5.5. Tests de partie rejoués sur le commit qui précède leur partie

Fichier de tests	Code d'avant	Résultat	Lecture
`test_m23a.py` (17)	`99831a9`	11 échecs, 6 succès	les 6 succès sont les témoins
`test_m23b.py` (46)	`3c94414`	21 échecs, 25 succès	témoins et comportements que la partie devait conserver
`test_v22.py` (3)	`ad7ab16`	1 échec, 2 succès	le défaut, et ses deux témoins (fichier ordinaire, troncature à travers un lien)

5.3.3 Les pièges des tests eux-mêmes

Deux fois, le banc de test du garde du shell a produit des succès *vides* sous Windows : le chemin du journal d'appels, écrit dans un script bash, contenait des barres obliques inverses que bash lisait comme des échappements ; les tests « aucun appel ne doit avoir eu lieu » passaient alors sans rien prouver. Le chemin est désormais dérivé de `$0`, ce qui supprime la classe d'erreur, et un test témoin vérifie que le journal est bien lu. Un test vert n'est une preuve que si l'on a vu qu'il pouvait être rouge.

5.4 Le portail de décision

5.4.1 Principe

Le portail évalue un corpus de 922 commandes avec le code avant et après une modification, et compare pour chaque ligne la décision, la règle, la cible et la capture. Il ne sait pas quelle décision est juste ; il voit chaque décision qui *change*. Une ligne qui bouge doit être expliquée :

erreur corrigée, arbitrage appliqué, ou simple changement d'étiquette. La référence est ensuite réenregistrée.

5.4.2 Les mouvements, partie par partie

Le tableau 5.6 reprend ce que chaque message de commit déclare. Les parties qui ne devaient rien changer — le moteur en ombre, les canaux ask/deny, plusieurs familles — ne changent rien ; les autres changent des lignes *nommées et revues*.

Table 5.6. Mouvements du portail de décision déclarés par partie (messages de commit)

Partie	Lignes	Nature
B, C1, C2a, SYS1	0	moteur en ombre, filets reliés : aucune décision ne change
S2+M1	124	8 erreurs corrigées, 18 nouvelles captures, 86 nouvelles autorisations (rien d'existant n'était détruit) — le moteur prend la main
M2.0+M2.1	28	suppression SQLite désormais restaurée ; chemins absents hors projet autorisés
T	0	sur les deux portails (Linux et Windows)
M2.2a (git)	26	lignes classées, référence réenregistrée
M2.2b1 (infra)	4	3 corrections, 1 arbitrage (R1)
M2.2b2a/b (cloud)	1 / 0	une étiquette seulement
M2.2b3	37	comptage des surfaces ; `compose down` et `docker push` escaladent
ASK	0	le canal s'ajoute à l'escalade sans changer la décision
M2.2b4a (envois)	24	16 comptages, 3 étiquettes `ssh`, 5 décisions arbitrées
M2.2b4b1/b2	12 / 9	PaaS (`firebase deploy` escalade), SaaS et migrations
M2.2c, d, e	0	système, effet LATER, préfiltre généré
M2.2f	10	derniers outils signalés
M2.3a	6	toutes plus strictes
M2.3b	65	plus strictes ou arbitrées (Rb1–Rb4) ; référence Windows enregistrée à part

Pourquoi réenregistrer la référence ?

Après revue, les mouvements deviennent la nouvelle norme : la partie suivante est comparée à *elle*, et non à un état ancien. Chaque ligne figée dans `tests/fixtures/` garde la trace de l'arbitrage qui l'a fait bouger, ce qui permet de retrouver, des semaines plus tard, pourquoi une commande escalade.

5.4.3 Quand le portail ne suffit plus : le corpus élargi (M2.3b)

M2.3b retirait à l'ancien chemin sa dernière ligne POSIX : un changement qui touche toutes les commandes non lues. Le portail de 922 commandes ne suffisait plus à le valider. Un **corpus élargi de** 2 306 **commandes** — le portail, toutes les lignes figées des *fixtures* et tous les exemples de la table de règles — a été rejoué avant et après. Il a révélé des régressions que le portail ne voyait pas, comme `kubectl exec . . . rm -rf` ou `npx prisma migrate reset`, corrigées avant la publication.

Chaque ligne devenue plus permissive y est soit une *décision arbitrée*, soit un *fait vérifié* : la suppression d'un fichier absent (rien n'est détruit), un texte « rm -rf » dans les métadonnées d'un envoi (une donnée, pas du code), un outil de migration qui crée seulement un fichier local.

5.4.4 Deux références, une par système

Le portail a été réenregistré séparément pour Windows : 21 lignes ne bougent que sous Windows — des chemins POSIX absolus impossibles à traduire en chemins Windows, donc non compris —, et toutes y sont plus strictes ou de même décision. Au commit de référence, le portail ne montre aucune ligne différente sous Windows, et une seule sous Linux, qui dépend des fichiers présents sur la machine.

5.5 Résultats

Les échecs connus. Deux échecs connus restent hors du périmètre de l'outil : ils dépendent de l'initialisation d'un dépôt git dans l'environnement de test. Les tests ignorés sont ceux qui ne s'appliquent pas au système ou à l'environnement de la machine (tests propres à Windows sous Linux, et inversement).

Ce que les chiffres ne disent pas. Le nombre de tests mesure l'effort, pas la sûreté. Les indicateurs qui comptent sont qualitatifs et vérifiables : chaque vulnérabilité corrigée a un test qui échoue sans sa correction ; aucune partie n'a rendu une décision plus permissive sans arbitrage ; et chaque décision affiche ce qu'elle n'a pas couvert.

5.6 Limites assumées

Un garde-fou qui cache ses limites crée une fausse confiance. Celles de `demo_cli` sont connues, et la plupart sont dites dans la raison même des décisions.

1. **Des erreurs, pas un adversaire.** Le modèle de menace vise un agent qui se trompe.

Table 5.7. Résultats chiffrés au commit de référence `7d2a2aa`

Mesure	Valeur
Code source / tests	≈ 34 200 / ≈ 29 000 lignes de Python
Fichiers de test / fonctions de test	134 / 1 728 (avant paramétrisation)
Table de règles	72 règles, 774 exemples exécutés comme tests, 214 outils, 12 familles à la demande
Règles dont la grammaire n'est pas vérifiée sur l'outil	9
Surfaces distantes avec canal ask/deny	38
Suite complète, Linux	3 483 réussis, 19 ignorés
Suite complète, Windows	3 398 réussis, 104 ignorés
Portail de décision (922 commandes)	Windows : 0 ligne différente ; Linux : une seule ligne, qui dépend des fichiers présents sur la machine
Corpus élargi (validation de M2.3b)	2 306 commandes rejouées avant et après
Couverture sur 88 incidents réels (red-team #008)	de 75 % à 90 % des enregistrements, de 44 % à 81 % des commandes uniques
Temps du moteur sur les 774 exemples (Linux)	médiane 0,29 ms ; 95 % sous 0,79 ms ; maximum 1,30 ms (budget : 10 ms CPU)

Un adversaire déterminé dispose de moyens que l'outil ne prétend pas couvrir (charge chiffrée, course entre vérification et usage d'un appel système).

2. **Le filet Linux est un prototype.** ptrace sur x86-64 seulement, un arrêt par appel système ; la cible de production désignée par le code est seccomp avec notification.
3. **Les filets sauvegardent le projet, et seulement lui.** Hors du projet, une destruction par du code non lu n'est pas sauvegardée ; le filet Linux l'enregistre (un reçu par fichier), le montage Windows ne la voit pas.
4. **Un exécuteur de code sans filet est autorisé** avec une note qui dit qu'aucune capture n'a été faite (arbitrage M1) — sauf le code en ligne qui lance d'autres programmes et les installations de paquets, qui escaladent (Rb2).
5. **Un mot sans règle n'a pas d'effet** (Rb1). Un outil absent de la table et qui n'est pas un exécuteur de code est traité comme sans effet ; la table s'étend famille par famille pour réduire ce cas.
6. **Ce que C3 ne lit pas.** Une construction PowerShell hors de la grammaire de C3 (chaîne construite à l'exécution, classe, fonction) est non comprise : filet WinFsp, sinon point de contrôle, sinon escalade ; un script `.ps1` est un exécuteur de code.
7. **Le délai du *hook*.** Si un *hook* dépasse le délai de l'agent, celui-ci l'interrompt et exécute la commande sans protection : c'est une frontière du modèle de *hook*, atténuée en gardant le *hook* rapide (budget du moteur).

8. **Codex ne sait que refuser** : pas de question, pas de filet par réécriture (V15).
9. **Le distant ne se récupère pas.** Pour une base, un dépôt ou un service distant, `demo_cli` reconnaît, demande ou refuse, et indique où le fournisseur garde son historique ; il ne restaure rien lui-même.
10. **Les points de contrôle sont optionnels** et bornés ; désactivés, une ligne non lue sans filet est escaladée.

Conclusion du chapitre

La sécurité de `demo_cli` n'a pas été supposée : elle a été attaquée — par la lecture, par des sondes, par la fouille des registres — et chaque faille trouvée a laissé un test. La validation repose sur deux filets complémentaires : les tests vérifient ce qu'on a pensé à écrire, le portail montre tout ce qui bouge. Les limites qui restent sont connues, datées et planifiées.

Conclusion générale et perspectives

Bilan

Ce stage est parti d'un garde-fou qui avait la bonne idée — récupérer plutôt que bloquer — et une lecture fragile des commandes. Il aboutit à un outil dont chaque décision repose sur ce que la commande *fait*, et non sur ce à quoi elle *ressemble*. Une analyse unique de la ligne et une simulation de ses effets ont remplacé les expressions régulières ; une table de 72 règles, dont chaque exemple est un test, décrit 214 outils ; depuis M2.3b, l'ancien chemin à expressions régulières ne décide plus aucune ligne POSIX ; une seule politique et un seul décideur rendent les mêmes réponses à toutes les portes et sur les deux systèmes ; un garde d'appels système sous Linux et un montage WinFsp sous Windows couvrent ce que la lecture ne peut pas prédire. Depuis C3, les deux dialectes passent par le même moteur et le même décideur.

Au-delà des fonctionnalités, le stage a produit une manière de travailler adaptée à un logiciel de sécurité : des parties courtes, des arbitrages écrits avant le code, des tests qui échouent sur le code d'avant, et un portail de 922 commandes qui montre chaque décision qui change. Le travail offensif l'a mise à l'épreuve : les vulnérabilités ont été consignées (V1–V22), dont les plus graves — une tâche élevée qui exécutait du code modifiable par l'agent, du SQL non lu autorisé, des outils qui détruisaient des fichiers sans être vus — sont corrigées et couvertes par des tests.

Le fil rouge de l'ensemble est une exigence d'honnêteté : jamais d'ALLOW pour une ligne non lue, jamais de REVERSIBLE pour une capture incomplète, jamais de question que personne ne verra, et une raison qui dit toujours ce qui n'a pas été couvert.

Apports personnels

Ce stage m'a permis de travailler à la frontière entre le développement et la sécurité offensive : concevoir une architecture, la défendre par des arbitrages écrits, puis chercher à la contourner. Il m'a appris la sémantique précise du shell POSIX et de PowerShell, les mécanismes bas niveau des deux systèmes (ptrace, systèmes de fichiers en espace utilisateur, listes de contrôle d'accès et descripteurs de sécurité Windows), et surtout une discipline : ne considérer un résultat comme acquis que lorsqu'un test a montré qu'il pouvait échouer.

Perspectives

— **PowerShell** : étendre la grammaire PowerShell lue par C3 (classes, fonctions, modules) et retirer les dernières règles textuelles PowerShell une fois leurs décisions égalées sur chaque ligne du portail.

— **Au-delà du projet** : décider d'éventuels blocages hors du projet à partir des reçus que le filet Linux enregistre désormais (V18, troisième étape).

— **Vitesse du filet Linux** : passer de ptrace à seccomp avec notification, et étendre le filet aux commandes tapées par la personne et aux processus qui survivent à la commande (V14).

— **Parité Windows du montage** : refus des fichiers de `demo_cli` par le montage lui-même, lecture bornée des fichiers, santé du montage mieux mesurée (V7–V9).

— **Les autres couches** : le proxy de sortie et son périmètre d'exfiltration (V10–V11), le portage complet vers Codex (V15), et la rotation des registres et de la réserve de récupération (V6, V13).

— **Un jeu d'évaluation étiqueté** : un corpus de commandes dont la décision attendue est fixée à la main, pour mesurer la justesse des décisions et non seulement leurs changements.

— **La table de règles** : continuer à réduire le « non couvert » famille par famille, et vérifier les 9 règles dont la grammaire vient encore de la documentation.

Bibliographie et webographie

[1] WePwn. *WePwn — site officiel*. URL : https://wepwn.ma/ (visité le 09/10/2026).

[2] WePwn. *WePwn Access — portail des chercheurs*. URL : https://access.wepwn.ma/ (visité le 09/10/2026).

[3] WePwn. *Services : divulgation de vulnérabilités gérée*. URL : https://www.wepwn.ma/Services (visité le 09/10/2026).

[4] WePwn. *demo_cli, version 0.4.0-beta.8*. Licence MIT. 19 juill. 2026. URL : https://github.com/WePwn/demo_cli (visité le 09/10/2026).

[5] OECD.AI. *Replit AI Coding Tool Deletes Live Production Database and Fakes Data*. AI Incidents Monitor. 19 juill. 2025. URL : https://oecd.ai/en/incidents/2025-07-19-1eb1 (visité le 09/10/2026).

[6] eWeek. *'Catastrophic Failure' : AI Agent Wipes Production Database, Then Lies About It*. Juill. 2025. URL : https://www.eweek.com/news/replit-ai-coding-assistant-failure/ (visité le 09/10/2026).

[7] AI Incident Database. *Incident 1178 : Gemini CLI deletes a user's files after a failed directory creation*. 21 juill. 2025. URL : https://incidentdatabase.ai/cite/1178 (visité le 09/10/2026).

[8] Anuraag Gupta. *google-gemini/gemini-cli, issue 4586*. Juill. 2025. URL : https://github.com/google-gemini/gemini-cli/issues/4586 (visité le 09/10/2026).

[9] anthropics/claude-code. *Issue 76626 : the agent deletes the folder contents while attempting a file count*. 11 juill. 2026. URL : https://github.com/anthropics/claude-code/issues/76626 (visité le 09/10/2026).

[10] Anthropic. *Claude Code documentation : Hooks reference*. URL : https://code.claude.com/docs/en/hooks (visité le 09/10/2026).

[11] Cursor. *Cursor documentation : Hooks*. URL : https://cursor.com/docs/agent/hooks (visité le 09/10/2026).

[12] Anthropic. *Claude Code auto mode*. Anthropic Engineering. 25 mars 2026. URL : https://www.anthropic.com/engineering/claude-code-auto-mode (visité le 09/10/2026).

[13] Devdatta Akhawe et Adrienne Porter Felt. « Alice in Warningland : A Large-Scale Field Study of Browser Security Warning Effectiveness ». In : *Proceedings of the 22nd USENIX Security Symposium*. USENIX Association, 2013, p. 257-272. URL : https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/akhawe.

[14] Anthropic. *Claude Code documentation : Configure the sandboxed Bash tool*. URL : https://code.claude.com/docs/en/sandboxing (visité le 09/10/2026).

[15] OpenAI. *Codex documentation : Security, sandbox and approvals*. URL : https://developers.openai.com/codex/security (visité le 09/10/2026).

[16] OpenAI. *openai/codex : codex-rs/linux-sandbox (README)*. URL : https://github.com/openai/codex/tree/main/codex-rs/linux-sandbox (visité le 09/10/2026).

[17] Anthropic. *Claude Code documentation : Checkpointing*. URL : https://code.claude.com/docs/en/checkpointing (visité le 09/10/2026).

[18] Vidar Holen. *ShellCheck — A shell script static analysis tool*. URL : https://github.com/koalaman/shellcheck (visité le 09/10/2026).

[19] Idan Kamara. *bashlex — Python parser for bash*. URL : https://github.com/idank/bashlex (visité le 09/10/2026).

[20] tree-sitter. *tree-sitter-bash : Bash grammar for tree-sitter*. URL : https://github.com/tree-sitter/tree-sitter-bash (visité le 09/10/2026).

[21] *ptrace(2) — process trace*. Linux man-pages project. URL : https://man7.org/linux/man-pages/man2/ptrace.2.html (visité le 09/10/2026).

[22] *seccomp_unotify(2) — Seccomp user-space notification mechanism*. Linux man-pages project. URL : https://man7.org/linux/man-pages/man2/seccomp_unotify.2.html (visité le 09/10/2026).

[23] *Landlock : unprivileged access control*. The Linux kernel documentation. URL : https://docs.kernel.org/userspace-api/landlock.html (visité le 09/10/2026).

[24] *FUSE Overview*. The Linux kernel documentation. URL : https://docs.kernel.org/filesystems/fuse/fuse.html (visité le 09/10/2026).

[25] Bill Zissimopoulos. *WinFsp — Windows File System Proxy*. URL : https://winfsp.dev (visité le 09/10/2026).

ANNEXE A

Exemple de reçu

Reçu réel écrit par le *hook* Claude Code pour la commande `python3 nettoyage.py`, réécrite vers le filet d'exécution (section 4.2). On y lit la décision et sa raison, le groupe qui relie ce reçu aux captures du filet, la prédiction du moteur (outil non couvert : `python3`), l'invariant, et les deux empreintes qui chaînent le reçu à son prédécesseur. Chemins abrégés.

`.demo_cli/receipts.jsonl` — un reçu (version 2.3)


```
{
  "action_raw": "python3 nettoyage.py",
  "action_type": "shell",
  "agent_id": "claude-code",
  "approvable": false,
  "chain": "main",
  "classification": "destructive",
  "context": {
    "aws_profile": "unknown",
    "azure_subscription": "unknown",
    "branch": "unknown",
    "commit": "unknown",
    "cwd": "~/projet",
    "environment": "unknown",
    "environment_source": "unknown",
    "fingerprint": "0115472ee1db5312",
    "gcloud_project": "unknown",
    "remote": "unknown",
    "repo_root": "unknown",
    "target_label": "unknown"
  },
  "context_mismatches": [],
  "decision": "SUPERVISED",
  "declared_intent": {
    "branch": null,
    "cwd": null,
    "env": "unknown",
    "reasoning": null,
    "remote": null,
    "scope": null
  },
```

```
  "dialect": "posix",
  "dry_run_affected_rows": null,
  "group_id": "8553dbf8",
  "invariant": "mutating_actions_must_be_recoverable_and_match_the_declared_context",
  "matched_rule": "unread_code",
  "mode": "enforce",
  "nonrecoverable_surface": null,
  "peer_head": null,
  "pipeline_segments": [],
  "prediction": {
    "changes": [],
    "creates": [],
    "databases": {},
    "decision": "ALLOW",
    "destroys": {},
    "engine": "shadow-1",
    "ms": 7.57,
    "no_capture": {},
    "remote": {},
    "truncated": false,
    "uncovered": [
      "python3"
    ],
    "unknown": [],
    "why": "nothing that exists is destroyed",
    "within": []
  },
  "prev_receipt_hash": "0bb417b0a3bd74dfbd168e472c8d2c703bb6c2d92f9b9bdaafb276e8aa5be5b6",
  "reason": "covered by the syscall guard: each file it destroys inside the project is
    ↪ backed up at the moment of destruction; outside the project nothing is captured.
    ↪ Undo with `demo_cli undo 8553dbf8`.",
  "receipt_hash": "0b74f3d679ddeec3663637f390ff423a4bad445e24fa6de3042c198f9cc526e0",
  "receipt_id": "6f9f41ee-64cc-40c5-be1f-68e731763eb8",
  "receipt_version": "2.3",
  "recovery_point": null,
  "remote_exec": false,
  "session_id": "unknown",
  "target_environment": "unknown",
  "timestamp": "2026-10-09T14:14:41.474621+00:00"
}
```

ANNEXE B

Extrait de la table de règles

La règle `gzip`, ajoutée par M2.3a après la découverte de V20. Chaque `ex(...)` est un test : la commande est simulée dans l'arborescence de référence, et l'impact doit être exactement celui indiqué. Les mentions `VERIFIED` décrivent un comportement observé sur des fichiers jetables.

src/demo_cli/engine/rules.py (extrait)

```
Rule(id="gzip", tools=("gzip", "gunzip", "zcat"), resolver="compress",
     options=OptionSpec(flags=_GZ_FLAGS, values={"-S": "suffix", "--suffix": "
    ↪ suffix"}),
     examples=(
         ex("gzip a.txt", destroys={"a.txt"}, creates={"a.txt.gz"},
            note="VERIFIED: the input is removed once a.txt.gz is written"),
         ex("gzip -k a.txt", creates={"a.txt.gz"}),
         ex("gzip -c a.txt > /dev/null", note="to stdout"),
         ex("touch a.txt.gz; gzip a.txt", creates={"a.txt.gz"},
            note="VERIFIED: an existing output is not overwritten (no terminal)"),
         ex("touch a.txt.gz; gzip -f a.txt", destroys={"a.txt"}, creates={"a.txt.
    ↪ gz"}),
         ex("gzip -r dir", destroys={"dir/"}, note="VERIFIED: -r rewrites every
    ↪ file under it"),
         ex("gzip --rm a.txt", unknown=True, note="an option demo_cli does not
    ↪ know"),
         ex("gzip $(cat list.txt)", destroys={"a.txt"}, creates={"a.txt.gz"}),
     )),
```

ANNEXE C

Extraits de tests

V17 : pas de point de contrôle pour un fichier hors du projet

tests/test_v17.py (en-tête)

```
"""V17 [...]: with checkpoints enabled, a line the engine cannot read whose
operand lies OUTSIDE the project took a whole-project checkpoint and came back
REVERSIBLE - but the checkpoint holds only the project, so the outside file was
not recoverable. Such a line now escalates; the same line inside the project
still gets its checkpoint."""
```

M2.3a : une ligne non lue va au filet, sinon au point de contrôle, sinon escalade

tests/test_m23a.py

```
def test_6_an_unread_line_takes_the_net(tmp_path, net):
    proj = _project(tmp_path)
    (proj / "cmd.txt").write_text("rm -rf build\n")
    r = _eval(proj, 'sh -c "$(cat cmd.txt)"', net=True)
    assert (r.decision.decision, r.classification.matched_rule) == ("SUPERVISED",
    ↪ "not_understood")

def test_6a_with_no_net_a_checkpoint_else_it_escalates(tmp_path):
    proj = _project(tmp_path)
    (proj / "cmd.txt").write_text("rm -rf build\n")
    line = 'sh -c "$(cat cmd.txt)"'
    r = _eval(proj, line, checkpoints=True)
    assert (r.decision.decision, r.classification.matched_rule) == ("REVERSIBLE",
    ↪ "not_understood")
    assert r.recovery_entry is not None
    r = _eval(proj, line)
```

```
15      assert (r.decision.decision, r.recovery_entry) == ("ESCALATE", None)
```

ANNEXE D

Captures d'écran

Trois captures réalisées sous Windows sont intégrées au rapport : la mise en place et le retrait de la protection (figure 4.1), le montage qui capture une suppression (figure 4.2) et la tâche durcie qui refuse toute modification à un utilisateur normal (figure 5.1). Le tableau ci-dessous liste, pour référence, les commandes qui permettent de reproduire les autres démonstrations.

Table D.1. Démonstrations reproductibles

Démonstration	Commande / action	Ce qu'on observe
`cc-reversible`	Dans Claude Code (projet protégé, mode `enforce`), demander : « supprime le dossier build »	le message « recovery point … captured », l'id, et la ligne `demo_cli undo <id>`
`cc-ask`	Demander à Claude Code : « supprime le volume docker donnees »	l'invite de permission de Claude Code avec la raison de `demo_cli` et l'indice de restauration
`cc-deny`	Demander : « lance terraform destroy »	le refus et sa raison (`infra_destroy`)
`cc-supervised`	Demander : « exécute python3 nettoyage.py » (script qui supprime un fichier)	la commande réécrite en `demo_cli sh -g <id>` et le message du filet
`undo`	`demo_cli log` puis `demo_cli undo <id>`	la liste des points de récupération et « RESTORED … as one undo »
`doctor-linux`	`demo_cli doctor` sous Linux	les lignes *hook*, modèle de menace, et la protection par agent
`doctor-windows`	`demo_cli doctor` sous Windows	montage actif, tâche durcie, protection par agent
`recu`	`demo_cli receipt` (ou `demo_cli receipts`)	un reçu avec sa décision, sa raison et son empreinte

Démonstration	Commande / action	Ce qu'on observe
`verify`	`demo_cli verify`	« VERIFIED », chaînes « INTACT »
`windows-montage`	`demo_cli setup` puis un `Remove-Item` lancé par l'agent, puis `demo_cli log`	le point de montage et la capture faite par le montage
`wsec`	En utilisateur normal : `icacls C:\ProgramData\demo_cli` ; écriture dans `runtime\run.py` ; `schtasks /change . . . /disable`	les ACL verrouillées, « Accès refusé » / « Access is denied »
`tests`	`python -m pytest -q` sous Linux et sous Windows	la dernière ligne « N passed, M skipped » des deux systèmes
`portail`	le comparateur du portail de décision sur une partie	le nombre de lignes comparées et les lignes qui bougent

ANNEXE E

L'architecture complète

Les sections 3.3 à 4.6 montrent `demo_cli` par ses idées : le flux, le moteur, la décision, les filets. Cette annexe le montre en entier, tel qu'il est écrit. Chaque boîte de la carte est un fichier réel, chaque nombre est son nombre de lignes, et la carte est produite par un script qui lit le code (`notes/gen_carte.py`) : il échoue si un seul module n'est rangé nulle part. Elle ne peut donc ni oublier un fichier ni en inventer un.

Table E.1. `demo_cli` en chiffres, au commit de référence `7d2a2aa`

Mesure	Valeur
Modules Python	80, pour ≈ 34 200 lignes
Fonctions et méthodes / classes	1 095 / 71
Liens d'import entre modules du paquet	257
Sous-commandes de la ligne de commande	31
Table de règles	72 règles, 774 exemples-tests, 214 outils, 12 familles
Dépendance obligatoire	une seule : `tomli`, et seulement avant Python 3.11
Dépendances optionnelles	`winfspy` (montage Windows), `mitmproxy` (garde de sortie HTTP)
Tests	29 000 lignes, 134 fichiers, 1 728 fonctions de test

La dépendance unique est un choix : un garde-fou s'installe sur la machine du développeur, à côté de l'agent, et chaque paquet tiers est une surface de plus à surveiller. Tout le reste — l'analyse du shell, la simulation, le hachage du registre, ptrace — repose sur la bibliothèque standard de Python. Les deux dépendances optionnelles le sont pour une raison donnée dans `pyproject.toml` : si `winfspy` était obligatoire, une machine sans WinFsp perdrait *tout* `demo_cli`, y compris les couches qui n'en ont pas besoin.

E.1 La carte

La figure E.1 se lit de gauche à droite, dans l'ordre où une commande la traverse : les portes reçoivent la commande, l'orchestrateur appelle le moteur, le moteur rend un impact, le décideur tranche, le reçu est écrit. Les flèches portent le nom des appels réels. En bas à droite, les filets d'exécution, où va une commande supervisée ; à gauche, l'outillage qui installe, diagnostique et affiche. Les modules en gras sont les pièces centrales de chaque couche.

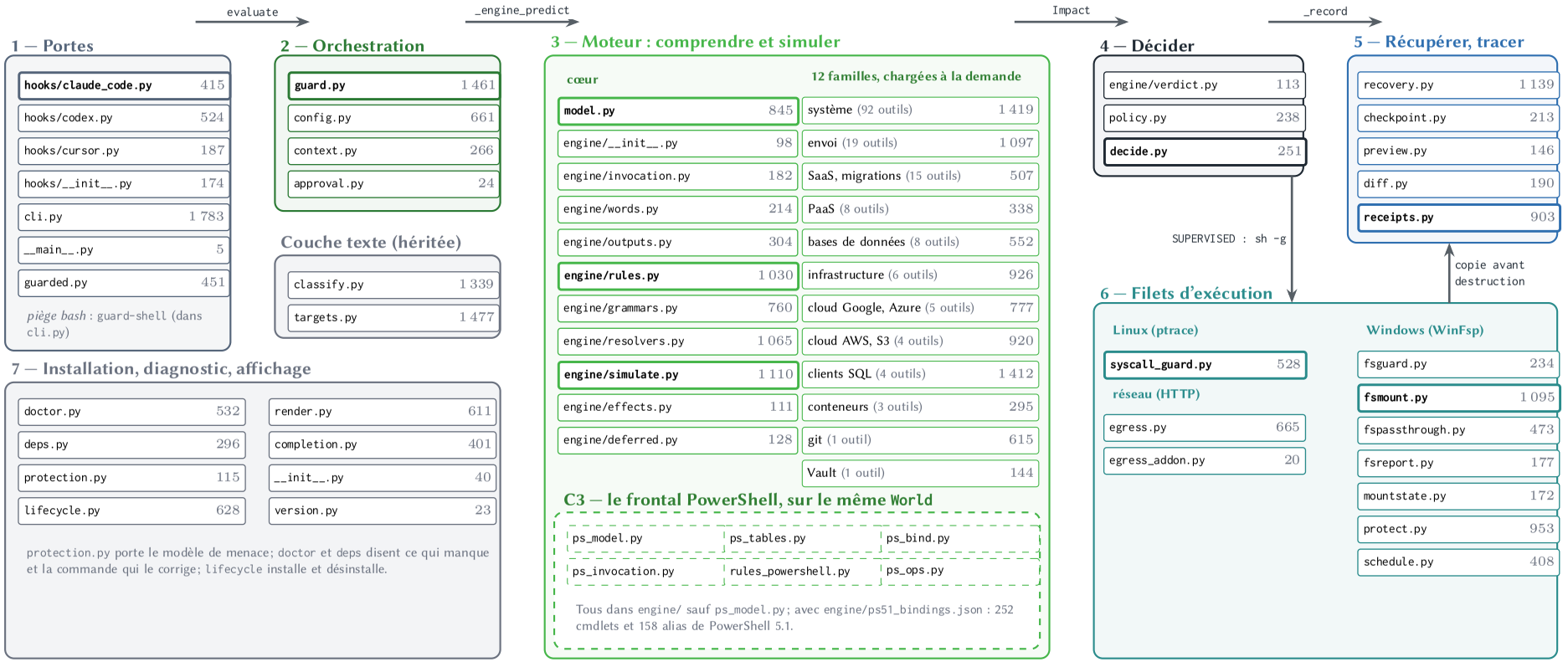


Figure E.1. La carte complète de `demo_cli` : les 80 modules, groupés par rôle, avec leur nombre de lignes au commit `7d2a2aa`. Les familles du moteur sont regroupées (un fichier de règles, un fichier d'opérations, parfois un fichier de données générées) ; le détail est au tableau E.2. C3, en tirets, est décrit à la section 4.7.

E.2 Où sont les lignes

La figure E.2 répond à une question simple : où est passé l'effort ? Le moteur porte 43 % du code, et les trois cinquièmes du moteur sont dans ses familles — la connaissance de ce que font 214 outils, écrite comme des données et vérifiée par leurs exemples. Le décideur, lui, tient en quelques centaines de lignes : c'est voulu. Une décision doit pouvoir se relire en entier ; ce qui est long, c'est de *savoir* ce qu'une commande fait, pas de décider quoi en faire.

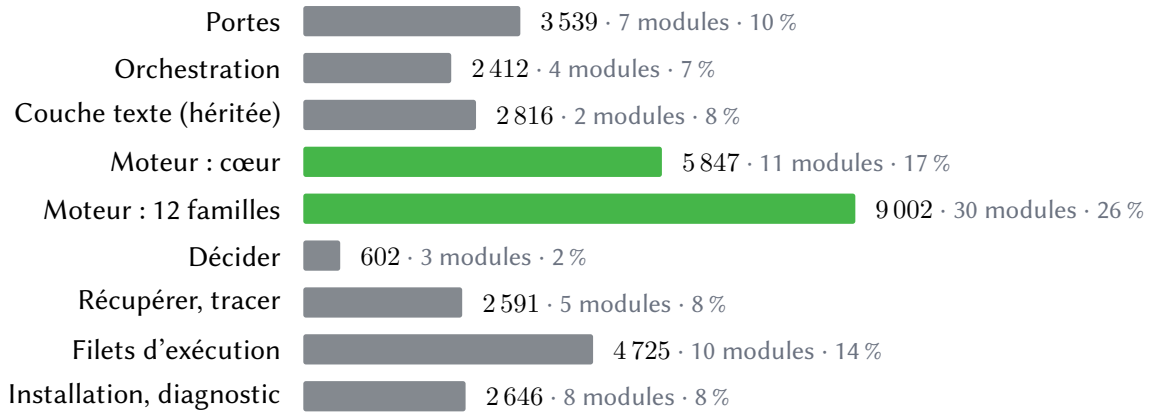


Figure E.2. Lignes de code par couche, au commit `7d2a2aa` (même découpage que la figure E.1)

Table E.2. Les 12 familles du moteur, chargées à la demande

Famille	Exemples d'outils	Outils	Lignes
Système	`systemctl`, `crontab`, `chmod`, `useradd`, `reg`, `schtasks`	92	1 419
Clients SQL	`sqlite3`, `psql`, `mysql`, `mariadb`	4	1 412
Envoi	`curl`, `gh`, `ssh`, `scp`, `npm`, `cargo`, `sendmail`	19	1 097
Infrastructure	`terraform`, `tofu`, `kubectl`, `helm`, `pulumi`, `cdk`	6	926
Cloud AWS, S3	`aws`, `s3cmd`, `rclone`, `mc`	4	920
Cloud Google, Azure	`gcloud`, `gsutil`, `az`, `azcopy`, `railway`	5	777
git	`git`	1	615
Bases de données	`redis-cli`, `mongosh`, `dropdb`, `pg_restore`, `sqlcmd`	8	552
SaaS, migrations	`shopify`, `wp`, `stripe`, `alembic`, `prisma`, `flyway`	15	507
PaaS	`heroku`, `fly`, `vercel`, `netlify`, `wrangler`, `firebase`	8	338
Conteneurs	`docker`, `podman`, `docker-compose`	3	295
Vault	`vault`	1	144
Règles du cœur (`rm`, `mv`, `cp`, `sed`, `tar`, `find`, `rsync`…)		48	—

Une famille n'est importée que lorsqu'une ligne nomme l'un de ses outils (`LAZY_FAMILIES` dans `engine/rules.py`) : un appel du *hook* qui ne rencontre que `ls` et `rm` ne paie ni les grammaires de `aws`, ni les 1 419 lignes de la famille système. C'est ce qui permet au moteur de grossir famille par famille sans sortir de son budget de temps. Les tables les plus longues (sous-commandes de

gh, de npm, de stripe, de vault…) ne sont pas écrites à la main : elles sont générées depuis les fichiers des outils eux-mêmes, et le fichier généré le dit.

E.3 Le trajet d'une commande dans le code

La figure 3.1 du chapitre 3 montre le flux comme une idée. La figure E.3 montre le même flux comme du code : les fonctions réelles, dans l'ordre où `Guard.evaluate` les appelle, pour une commande Bash reçue par le *hook* de Claude Code. Les autres portes (Codex, Cursor, le piège bash, `demo_cli check`) entrent à l'étape 2 par la même fonction : il n'y a qu'un chemin de décision.



Figure E.3. Le trajet d'une commande dans le code, du *hook* au reçu (fonctions réelles de `guard.py` et des modules qu'il appelle)

E.4 Les dépendances vont dans un seul sens

La carte a été vérifiée sur les 257 liens d'import réels du paquet. Trois propriétés en ressortent, et elles ne doivent rien au hasard :

— **Le décideur n'importe presque rien.** `decide.py` importe `policy.py` et le type de classification, et rien d'autre : aucun accès au disque, au réseau ou au registre. C'est ce qui le

rend pur, testable en isolation, et incapable de « mentir » sur une capture : il ne peut que lire le fait qu'on lui donne.

— **Le moteur ne connaît pas l'orchestrateur.** Aucun module de `engine/` n'importe `guard.py` ni une porte. Le moteur peut donc être exercé seul : les 774 exemples de la table le traversent directement dans les tests, et le préfiltre du piège shell est généré depuis sa table de règles.

— **La politique est à un seul endroit.** `decide.py`, `engine/verdict.py`, `guard.py` et le *hook* de Claude Code lisent `policy.py` pour toute question sur une surface distante. Changer la décision d'une surface, c'est changer une ligne, une fois.

E.5 Carte des modules

Le tableau E.3 donne le rôle de chaque module hors familles (les familles sont au tableau E.2), dans l'ordre de la carte.

Table E.3. Carte des modules : rôle et taille (lignes, commit `7d2a2aa`)

Module	Lignes	Rôle
1 — Portes		
`hooks/claude_code.py`	415	*Hook* PreToolUse de Claude Code : lit l'appel d'outil, appelle le garde, répond allow, ask ou deny, ou réécrit la commande vers `demo_cli sh -g`.
`hooks/codex.py`	524	*Hook* de Codex : la même décision, exprimée par refus seul.
`hooks/cursor.py`	187	*Hook* `beforeShellExecution` de Cursor, fermé en cas d'échec.
`hooks/__init__.py`	174	Le socle commun des intégrations.
`cli.py`	1 783	La ligne de commande : 31 sous-commandes (`check`, `undo`, `verify`, `sh`, `setup`…) et le piège bash (`guard-shell`). Aucune logique de sécurité : elle analyse, appelle, affiche.
`__main__.py`	5	`python -m demo_cli`.
`guarded.py`	451	`demo_cli guarded` : lance l'agent avec toutes les couches qui peuvent l'être, et dit lesquelles manquent.
2 — Orchestration		
`guard.py`	1 461	L'orchestrateur : `Guard.evaluate` enchaîne les étapes nommées (figure E.3) ; `evaluate_file_edit` pour la porte fichier.
`config.py`	661	`.demo_cli.toml` : mode (observation ou application), cibles déclarées, espace de travail propre au projet.
`context.py`	266	Le contexte : empreinte, environnement (déclaré d'abord, deviné en dernier), comparaison avec l'intention déclarée.
`approval.py`	24	L'autorisation structurelle : un jeton signé par une clé que l'agent ne détient pas.
Couche texte (héritée)		

suite page suivante

Module	Lignes	Rôle
`classify.py`	1 339	La lecture historique du texte d'une commande, gardée pour le chemin hérité ; elle porte aussi les constantes de dialecte.
`targets.py`	1 477	La cible réelle d'une commande (fichier, SQLite, PostgreSQL), sans jamais de cible par défaut.
3 — Moteur : le cœur		
`model.py`	845	L'analyse POSIX unique : mots et citations, substitutions imbriquées, *heredocs*, enveloppes.
`engine/__init__.py`	98	Le point d'entrée : `predict`, son budget de temps, la prédiction écrite dans le reçu.
`engine/invocation.py`	182	De l'analyse aux invocations : une par commande qui s'exécute, avec ses enveloppes et ses scripts imbriqués.
`engine/words.py`	214	L'expansion des mots comme bash, sur les mots ; l'inconnu est levé, jamais deviné.
`engine/outputs.py`	304	Ce qu'impriment quelques lectures, pour lire `rm $(cat liste.txt)`.
`engine/rules.py`	1 030	La table de règles comme données ; le chargement des familles ; le préfiltre du piège shell.
`engine/grammars.py`	760	Les grammaires d'options générées depuis l'aide des outils.
`engine/resolvers.py`	1 065	Les outils qu'un gabarit ne peut pas exprimer (`cp`, `sed`, `tar`, `rsync`, `find`, `dd`…).
`engine/simulate.py`	1 110	La simulation sur un système de fichiers virtuel : ensembles d'états, origine du contenu, budget.
`engine/effects.py`	111	Le vocabulaire des effets (tableau 3.3).
`engine/deferred.py`	128	Les lieux où du code s'exécute plus tard sans être nommé (profil du shell, *hooks* git, tâches planifiées).
4 — Décider		
`engine/verdict.py`	113	Le jugement d'un impact, dans l'ordre convenu.
`policy.py`	238	La politique unique : décision par surface, canal ask ou deny, indices de récupération.
`decide.py`	251	Le décideur unique, pur, à l'ordre de branches fixe.
5 — Récupérer, tracer		
`recovery.py`	1 139	Les captures tout-ou-rien, les groupes, la restauration (`undo`).
`checkpoint.py`	213	Le point de contrôle de tout le projet, optionnel et borné.
`preview.py`	146	L'aperçu SQL (DRY_RUN) : lignes touchées et échantillon.
`diff.py`	190	Ce qui a changé depuis le dernier point de récupération.
`receipts.py`	903	Le registre chaîné : ajout sous verrou, vérification de la chaîne, liens entre chaînes.
6 — Filets d'exécution		
`syscall_guard.py`	528	Linux : le garde ptrace (section 4.5).
`fsguard.py`	234	Windows : le jugement — quelles opérations de fichier détruisent.

suite page suivante

Module	Lignes	Rôle
`fsmount.py`	1 095	Windows : le système de fichiers WinFsp et son montage.
`fspassthrough.py`	473	Windows : le passe-plat sur les fichiers réels.
`fsreport.py`	177	Dire à l'agent ce que le montage a capturé.
`mountstate.py`	172	Le montage tourne-t-il *vraiment* ? (« installé mais inerte »)
`protect.py`	953	Déplacer le projet vers `.real`, le verrouiller — et l'inverse.
`schedule.py`	408	La tâche d'ouverture de session durcie (section 5.2).
`egress.py`	665	Le garde de sortie : les écritures vers des API SaaS, vues au niveau du réseau.
`egress_addon.py`	20	Le chargeur de ce garde pour `mitmproxy`.
7 — Installation, diagnostic, affichage		
`doctor.py`	532	Le diagnostic : *hooks*, montage, tâche, dépendances.
`deps.py`	296	Ce qui manque, et la commande exacte qui le corrige.
`protection.py`	115	Le modèle de menace, et l'état des protections actives.
`lifecycle.py`	628	`setup` et `teardown`, étape par étape.
`render.py`	611	L'affichage : couleurs des décisions, `NO_COLOR`.
`completion.py`	401	La complétion pour bash, zsh, fish et PowerShell.
`__init__.py`	40	L'API publique et l'invariant.
`version.py`	23	La version, à un seul endroit.

Ce que la carte raconte

`demo_cli` est parti d'une bêta de 3 700 lignes : une classification par expressions régulières et cinq décisions. La carte montre ce qu'il a fallu construire autour pour tenir une seule promesse — « une action qui modifie doit être récupérable et correspondre au contexte déclaré » : un analyseur, un simulateur, 214 outils décrits comme des données, un décideur unique, deux filets d'exécution, un registre dont toute retouche se voit, et l'outillage qui installe tout cela proprement sur deux systèmes. Chaque boîte répond à une question précise ; aucune n'est là pour faire nombre.